



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/26



Gruppo 17

Nome: BitByBit

Email: swe.bitbybit@gmail.com

Specifica Tecnica



Stato:	In lavorazione
Versione:	0.9.0
Responsabile:	Gabriele Scaggiante
Redattori:	Gabriele Scaggiante
	Dennis Parolin
	Ferdinando Fracasso
	Giovanni Visentin
	Marco Sanguin
	Riccardo Manisi
Verificatori:	Riccardo Manisi
	Dennis Parolin
	Marco Sanguin
	Giovanni Visentin
	Gabriele Scaggiante
	Ferdinando Fracasso
Destinatari:	Miriade srl
	Prof. Tullio Vardanega
	Prof. Riccardo Cardin
	Gruppo BitByBit



Registro delle modifiche

Versione	Data	Autore	Descrizione	Verificatore
0.9.0	2026-05-02	Riccardo Manisi	Correzioni a sezioni Architettura^G logica, e di Deployment^G	Gabriele Scaggianti
0.8.0	2026-05-02	Giovanni Visentin	Sistemazione della tabella dei requisiti	Dennis Parolin
0.7.1	2026-05-07	Ferdinando Fracasso	Aggiornato sezione Librerie	Riccardo Manisi
0.7.0	2026-04-29	Riccardo Manisi	Redatte le sezioni Architettura classi microservizio contatti fidati , Architettura classi microservizio luoghi sicuri	Gabriele Scaggianti
0.6.1	2026-04-27	Gabriele Scaggianti	Aggiunte librerie nella sezione Librerie	Ferdinando Fracasso
0.6.0	2026-04-26	Gabriele Scaggianti	Completata la sezione Architettura chatbot introducendo la sezione Architettura logica chatbot e aggiornando la sezione dedicata alle API	Dennis Parolin
0.5.3	2026-04-24	Ferdinando Fracasso	Aggiornato la sezione Architettura funzionalità diari	Gabriele Scaggianti
0.5.2	2026-04-23	Gabriele Scaggianti	Corretta la sezione Architettura chatbot dopo l'introduzione di due tabelle DynamoDB	Giovanni Visentin



0.5.1	2026-04-20	Dennis Parolin	Aggiornati i pattern presenti nella la sezione Design pattern utilizzati , aggiunti i pattern Dependency Injection , Command e Adapter .	Gabriele Scaggiante
0.5.0	2026-04-18	Gabriele Scaggiante	Nella sezione Architettura backend , aggiornata Architettura^G Auth, Luoghi Sicuri e Materiale Informativo; redatte sezioni Diario, Chatbot e DMS.	Dennis Parolin
0.4.0	2026-04-17	Riccardo Manisi	Redatta la sezione Architettura InvioSOS	Gabriele Scaggiante
0.3.0	2026-04-17	Riccardo Manisi	Redatta la sezione Architettura auth	Ferdinando Fracasso
0.2.1	2026-04-12	Marco Sanguin	Redatta la sezione Architettura luoghi sicuri	Riccardo Manisi
0.2.0	2026-04-12	Marco Sanguin	Redatta la sezione Architettura funzionalità Dead man's switch	Ferdinando Fracasso
0.1.9	2026-04-12	Dennis Parolin	Creata la sezione Architettura Back End con relative sottosezioni Autenticazione , Luoghi sicuri e Materiale informativo	Giovanni Visentin
0.1.8	2026-04-11	Marco Sanguin	Aggiunta la sezione Stato dei requisiti funzionali	Ferdinando Fracasso



0.1.7	2026-04-10	Marco Sanguin	Redatta la sezione Architettura Area Informativa	Ferdinando Fracasso
0.1.6	2026-04-09	Ferdinando Fracasso	Redatta la sezione Design pattern utilizzati	Giovanni Visentin
0.1.5	2026-04-06	Ferdinando Fracasso	Redatta la sezione Architettura funzionalità diari	Marco Sanguin
0.1.4	2026-04-05	Giovanni Visentin	Aggiunta della funzionalità dei contatti fidati	Dennis Parolin
0.1.3	2026-04-04	Gabriele Scaggianti	Iniziata la sezione Architettura frontend , completata Architettura chatbot	Marco Sanguin
0.1.2	2026-04-02	Dennis Parolin	Redatta la sezione Architettura logica e Architettura di deployment	Marco Sanguin
0.1.1	2026-03-31	Gabriele Scaggianti	Aggiunta di EventBridge, SAM, CloudFormation e Docker nella sezione tecnologie.	Dennis Parolin
0.1.0	2026-03-30	Gabriele Scaggianti	Creazione del documento. Scrittura dell'introduzione e delle tecnologie utilizzate.	Riccardo Manisi



Indice

1	Introduzione	21
1.1	Scopo del documento	21
1.2	Riferimenti	21
1.2.1	Riferimenti normativi	22
1.2.2	Riferimenti informativi	22
2	Tecnologie	23
2.1	Framework	23
2.1.1	Flutter	23
2.2	Linguaggi di programmazione	23
2.2.1	Dart	23
2.2.2	Python	24
2.3	Servizi AWS	24
2.3.1	Lambda	24
2.3.2	API Gateway	24
2.3.3	DynamoDB	24
2.3.4	S3	25
2.3.5	Cognito	25
2.3.6	Bedrock	25
2.3.7	SES	25
2.3.8	CloudWatch	25
2.3.9	EventBridge	26
2.3.10	SAM	26
2.3.11	CloudFormation	26
2.4	Librerie	26
2.4.1	http	26
2.4.2	image_picker	27
2.4.3	path_provider	27
2.4.4	provider	27
2.4.5	url_launcher	27
2.4.6	flutter_dotenv	28
2.4.7	flutter_web_auth_2	28
2.4.8	flutter_map	28
2.4.9	geolocator	28
2.4.10	get_it	29
2.4.11	latlong2	29
2.4.12	path	29
2.4.13	flutter_secure_storage	29



2.4.14	intl	29
2.4.15	just_audio	30
2.4.16	plugin_platform_interface	30
2.4.17	file_picker	30
2.4.18	url_launcher_platform_interface	30
2.4.19	command_it	30
2.4.20	listen_it	31
2.4.21	mocktail	31
2.4.22	shared_preferences	31
2.4.23	connectivity_plus	31
2.4.24	audio_session	31
2.4.25	python-ulid	32
2.4.26	rank-bm25	32
2.4.27	boto3	32
2.5	Tecnologie di analisi statica	32
2.5.1	SonarQube	32
2.6	Tecnologie di analisi dinamica	33
2.6.1	Pytest	33
2.6.2	Moto	33
2.7	Altro	33
2.7.1	Ollama	33
2.7.2	Docker	33
3	Architettura	34
3.1	Architettura logica	34
3.1.1	Frontend Mobile	34
3.1.2	Backend Serverless	35
3.2	Design pattern utilizzati	36
3.2.1	Model-view-viewmodel	36
3.2.1.1	Descrizione del pattern:	36
3.2.1.2	Ragioni per l'utilizzo:	36
3.2.1.3	Riferimenti nel progetto:	37
3.2.2	Observer	38
3.2.2.1	Descrizione del pattern:	38
3.2.2.2	Ragioni per l'utilizzo:	39
3.2.2.3	Riferimenti nel progetto:	39
3.2.3	Proxy	39
3.2.3.1	Descrizione del pattern:	40
3.2.3.2	Ragioni per l'utilizzo:	40
3.2.3.3	Riferimenti nel progetto:	40
3.2.4	Singleton	40



3.2.4.1	Descrizione del pattern:	40
3.2.4.2	Ragioni per l'utilizzo:	41
3.2.4.3	Riferimenti nel progetto:	41
3.2.5	Dependency Injection	41
3.2.5.1	Descrizione del pattern:	41
3.2.5.2	Ragioni per l'utilizzo:	42
3.2.5.3	Riferimenti nel progetto:	42
3.2.6	Command	43
3.2.6.1	Descrizione del pattern:	43
3.2.6.2	Ragioni per l'utilizzo:	43
3.2.6.3	Riferimenti nel progetto:	44
3.2.6.4	Command<T>	44
3.2.6.5	Command0<T>	44
3.2.6.6	Command1<T, A>	45
3.2.7	Adapter	45
3.2.7.1	Descrizione del pattern:	45
3.2.7.2	Ragioni per l'utilizzo:	45
3.2.7.3	Riferimenti nel progetto:	46
3.3	Architettura di deployment	46
3.3.1	Architettura three-tier	46
3.3.2	Architettura a microservizi	46
3.3.3	Infrastruttura	47
3.3.3.1	Infrastructure as Code^G (IaC)	48
3.4	Architettura Front End	48
3.4.1	Autenticazione	48
3.4.1.1	AuthScreen	49
3.4.1.2	AuthHeaderWidget	50
3.4.1.3	GoogleLoginButtonWidget	50
3.4.1.4	User	51
3.4.1.5	UserDTO	52
3.4.1.6	AuthViewModel	53
3.4.1.7	AuthRepository	54
3.4.1.8	AuthService	54
3.4.2	Invio SOS	55
3.4.2.1	SosBottomSheetWidget	56
3.4.2.2	SosAlertViewModel	56
3.4.2.3	SosAlertRepository	57
3.4.2.4	SosAlertService	57
3.4.3	Chatbot	58
3.4.3.1	ChatbotScreen	58



3.4.3.2	ChatWidget	59
3.4.3.3	ChatHistoryWidget	59
3.4.3.4	ChatbotModeToggleWidget	60
3.4.3.5	ChatbotSendMessageWidget	60
3.4.3.6	ChatbotCreateChatWidget	61
3.4.3.7	ChatbotViewModel	61
3.4.3.8	Chat	62
3.4.3.9	ProxyChat	62
3.4.3.10	LocalChat	62
3.4.3.11	ChatMessage	63
3.4.3.12	ChatMode	63
3.4.3.13	MessageType	64
3.4.3.14	MessageResponse	64
3.4.3.15	ChatbotRepository	65
3.4.3.16	ChatbotService	65
3.4.3.17	ChatDTO	66
3.4.4	Diario criptato e diario fittizio	66
3.4.4.1	DiaryAccessScreen	67
3.4.4.2	PasswordFormWidget	67
3.4.4.3	DiaryAccessViewModel	68
3.4.4.4	DiaryAccountRepository	68
3.4.4.5	DiaryAccountService	68
3.4.4.6	DiaryAccessResult	68
3.4.4.7	DiaryScreen	69
3.4.4.8	DiaryPasswordSettingWidget	69
3.4.4.9	NoteListWidget	69
3.4.4.10	NoteActionsWidget	70
3.4.4.11	NoteEditorWidget	70
3.4.4.12	NoteAudioPlayerWidget	70
3.4.4.13	DiaryViewModel	71
3.4.4.14	DiarySession	71
3.4.4.15	DiaryType	72
3.4.4.16	Note	72
3.4.4.17	ProxyNote	73
3.4.4.18	LocalNote	73
3.4.4.19	NoteElement	74
3.4.4.20	NoteTextElement	74
3.4.4.21	NoteImgElement	75
3.4.4.22	NoteAudioElement	75
3.4.4.23	NoteService	75



3.4.4.24	NoteRepository	76
3.4.4.25	NoteDTO	76
3.4.5	Contatti Fidati	76
3.4.5.1	TrustedContactScreen	77
3.4.5.2	TrustedContactListWidget	77
3.4.5.3	TrustedContactFormWidget	78
3.4.5.4	TrustedContactActionsWidget	78
3.4.5.5	TrustedContactViewModel	78
3.4.5.6	TrustedContact	79
3.4.5.7	TrustedContactRepository	79
3.4.5.8	TrustedContactService	80
3.4.5.9	TrustedContactDTO	80
3.4.6	Luoghi Sicuri	80
3.4.6.1	SafePlaceMapScreen	81
3.4.6.2	SafePlaceMapWidget	81
3.4.6.3	SafePlaceViewModel	82
3.4.6.4	SafePlace	82
3.4.6.5	SafePlaceRepository	83
3.4.6.6	SafePlaceService	83
3.4.6.7	SafePlaceDTO	83
3.4.7	Dead Man's Switch	84
3.4.7.1	DeadManScreen	85
3.4.7.2	DeadManFormWidget	85
3.4.7.3	DeadManViewModel	86
3.4.7.4	DeadManSettings	86
3.4.7.5	DeadManRepository	87
3.4.7.6	DeadManService	87
3.4.7.7	DeadManSettingsDTO	87
3.4.8	Area Informativa	88
3.4.8.1	InfoHubScreen	88
3.4.8.2	CommunityScreen	88
3.4.8.3	CommunityListWidget	89
3.4.8.4	CommunityViewModel	89
3.4.8.5	Community	89
3.4.8.6	CommunityRepository	90
3.4.8.7	CommunityService	90
3.4.8.8	CommunityDTO	90
3.4.8.9	LegislationScreen	91
3.4.8.10	LegislationListWidget	91
3.4.8.11	LegislationViewModel	91



3.4.8.12	Legislation	92
3.4.8.13	LegislationRepository	92
3.4.8.14	LegislationService	93
3.4.8.15	LegislationDTO	93
3.4.8.16	InfoMaterialScreen	93
3.4.8.17	InfoMaterialListWidget	93
3.4.8.18	InfoMaterialViewModel	94
3.4.8.19	InfoMaterial	94
3.4.8.20	InfoMaterialRepository	95
3.4.8.21	InfoMaterialService	95
3.4.8.22	InfoMaterialDTO	95
3.4.8.23	EmergencyContactScreen	95
3.4.8.24	EmergencyContactListWidget	96
3.4.8.25	EmergencyContactViewModel	96
3.4.8.26	EmergencyContact	96
3.4.8.27	EmergencyContactRepository	97
3.4.8.28	EmergencyContactService	97
3.4.8.29	EmergencyContactDTO	97
3.5	Architettura Back End	98
3.5.1	Autenticazione	98
3.5.1.1	Architettura e Flusso di Autenticazione: Modulo Federated Login	98
3.5.1.1.1	Iniziazione del Flusso e Autenticazione Federata	99
3.5.1.1.2	Emissione dei Token di Sessione	99
3.5.1.1.3	Validazione Perimetrale e Instradamento Sicuro	99
3.5.2	Luoghi sicuri	100
3.5.2.1	Architettura e Flusso di Elaborazione: Modulo Luoghi Sicuri	100
3.5.2.1.1	Delega della Logica Geospaziale al Livello Client (Edge Computing)	101
3.5.2.2	API associate	101
3.5.2.3	Architettura logica: Modulo Luoghi Sicuri	103
3.5.2.3.1	Marker	104
3.5.2.3.2	SafePlacesController	105
3.5.2.3.3	GetMarkerPort	105
3.5.2.3.4	SafePlacesService	106
3.5.2.3.5	SafePlacesRepositoryPort	106
3.5.2.3.6	S3SafePlacesRepository	106
3.5.2.3.7	Materiale informativo	107
3.5.2.4	Architettura e Flusso di Elaborazione: Modulo Materiale Informativo	107



3.5.2.5	API associate	107
3.5.2.6	Architettura logica: Modulo Materiale Informativo	109
3.5.2.6.1	Resource	110
3.5.2.6.2	ResourcesController	111
3.5.2.6.3	GetResourcesPort	111
3.5.2.6.4	ResourcesService	111
3.5.2.6.5	ResourcesRepositoryPort	112
3.5.2.6.6	S3ResourcesRepository	112
3.5.3	Diario criptato e fittizio	113
3.5.3.1	Architettura e Flusso di Elaborazione: diario criptato e fittizio	113
3.5.3.2	API associate	115
3.5.3.3	Architettura logica: microservizio diario	122
3.5.3.3.1	DiaryType	123
3.5.3.3.2	Note	123
3.5.3.3.3	NoteElement	124
3.5.3.3.4	NoteElementDTO	124
3.5.3.3.5	NoteDTO	125
3.5.3.3.6	GetNoteCmd	125
3.5.3.3.7	GetNotesCmd	126
3.5.3.3.8	AddElementNewNoteCmd	126
3.5.3.3.9	AddNoteCmd	127
3.5.3.3.10	AddNoteElementCmd	127
3.5.3.3.11	DeleteNoteCmd	128
3.5.3.3.12	DeleteNoteElementCmd	128
3.5.3.3.13	ValidatePasswordCmd	129
3.5.3.3.14	ValidateTokenCmd	129
3.5.3.3.15	SetPasswordCmd	130
3.5.3.3.16	CheckPasswordCmd	130
3.5.3.3.17	GetNotePort	131
3.5.3.3.18	SetNotePort	131
3.5.3.3.19	DeleteNotePort	131
3.5.3.3.20	SetNoteElementPort	132
3.5.3.3.21	DeleteNoteElementPort	132
3.5.3.3.22	SetPasswordPort	132
3.5.3.3.23	ValidationPort	133
3.5.3.3.24	CheckPasswordPort	133
3.5.3.3.25	NoteRepositoryPort	134
3.5.3.3.26	FileRepositoryPort	134
3.5.3.3.27	DiaryAuthRepositoryPort	135



3.5.3.3.28	NoteService	135
3.5.3.3.29	DiaryAuthService	136
3.5.3.3.30	DynamoNoteAdapter	136
3.5.3.3.31	S3NoteAdapter	137
3.5.3.3.32	DynamoAuthAdapter	137
3.5.3.3.33	DiaryNoteController	138
3.5.3.3.34	DiaryAccessController	139
3.5.4	Chatbot	139
3.5.4.1	Architettura e Flusso di Elaborazione: chatbot (detective delle relazioni e specchio intelligente)	140
3.5.4.2	API associate	141
3.5.4.3	Architettura logica del chatbot	146
3.5.4.3.1	Chat	147
3.5.4.3.2	ChatMessage	147
3.5.4.3.3	ChatbotLLMPort	148
3.5.4.3.4	BedrockLLMAdapter	148
3.5.4.3.5	ChatbotLLMService	148
3.5.4.3.6	ChatsRepositoryPort	149
3.5.4.3.7	DynamoChatAdapter	149
3.5.4.3.8	CreateChatPort	150
3.5.4.3.9	GetChatPort	150
3.5.4.3.10	UpdateChatPort	151
3.5.4.3.11	DeleteChatPort	151
3.5.4.3.12	ChatMessagePort	151
3.5.4.3.13	ChatbotCRUDService	152
3.5.4.3.14	CreateChatCmd	152
3.5.4.3.15	GetChatCmd	153
3.5.4.3.16	GetChatListCmd	153
3.5.4.3.17	UpdateChatCmd	153
3.5.4.3.18	DeleteChatCmd	154
3.5.4.3.19	AddChatMessageCmd	154
3.5.4.3.20	ChatDTO	155
3.5.4.3.21	ChatMessageDTO	155
3.5.4.3.22	ChatBotController	156
3.5.5	Contatti fidati, invio SOS e Dead man's switch	156
3.5.5.1	Architettura e Flusso di Elaborazione: contatti fidati, SOS e Dead man's switch	157
3.5.5.2	API associate	158
3.5.5.3	Architettura logica: Modulo contatti fidati, SOS e Dead man's switch	164

3.5.5.3.1	TrustedContact	165
3.5.5.3.2	TrustedContactDTO	165
3.5.5.3.3	AddTrustedContactCmd	166
3.5.5.3.4	GetTrustedContactCmd	166
3.5.5.3.5	DeleteTrustedContactCmd	166
3.5.5.3.6	DmsConfigurationSettings	167
3.5.5.3.7	DmsConfigurationSettingsDTO	167
3.5.5.3.8	AlertCmd	168
3.5.5.3.9	EmailMessage	168
3.5.5.3.10	TrustedContactController	169
3.5.5.3.11	SetDmsHeartBeatPort	169
3.5.5.3.12	GetDmsConfigurationSettingsPort	170
3.5.5.3.13	SetDmsConfigurationSettingsPort	170
3.5.5.3.14	UpdateDmsAlertPort	170
3.5.5.3.15	SetTrustedContactPort	171
3.5.5.3.16	GetTrustedContactPort	171
3.5.5.3.17	DeleteTrustedContactPort	171
3.5.5.3.18	SendAlertPort	172
3.5.5.3.19	DmsCRUDService	172
3.5.5.3.20	DmsAlertService	172
3.5.5.3.21	TrustedContactCRUDService	173
3.5.5.3.22	SOSAlertService	173
3.5.5.3.23	DmsRepositoryPort	174
3.5.5.3.24	TrustedContactRepositoryPort	175
3.5.5.3.25	NotificationPort	175
3.5.5.3.26	DynamoDmsAdapter	176
3.5.5.3.27	DynamoTrustedContactAdapter	176
3.5.5.3.28	SesNotificationAdapter	176

4	Stato dei requisiti funzionali	177
4.1	Requisiti Funzionali obbligatori	177
4.2	Requisiti Funzionali desiderabili	189
4.3	Requisiti Funzionali opzionali	191



Elenco delle tabelle

2	Tabella dello stato dei requisiti funzionali obbligatori	189
3	Tabella Requisiti Funzionali Desiderabili	190
4	Tabella dello stato dei requisiti funzionali opzionali	192

Elenco delle figure

1	Rappresentazione del pattern MVVM nel modulo di Autenticazione	36
2	Rappresentazione del pattern Observer applicato nel modulo dei contatti fidati	38
3	Rappresentazione del pattern Proxy applicato alla gestione delle chat . . .	39
4	Rappresentazione del pattern Singleton applicato alla sessione del diario . .	40
5	Rappresentazione del pattern Dependency Injection nel modulo di Autenti- cazione	41
6	Rappresentazione del pattern Command	43
7	Diagramma della classe Command<T>	44
8	Diagramma della classe Command0<T>	45
9	Diagramma della classe Command1<T, A>	45
10	Diagramma delle classi relativo all'autenticazione	49
11	Diagramma della classe AuthScreen	50
12	Diagramma della classe AuthHeaderWidget	50
13	Diagramma della classe GoogleLoginButtonWidget	51
14	Diagramma della classe User	52
15	Diagramma della classe UserDTO	53
16	Diagramma della classe AuthViewModel	53
17	Diagramma della classe AuthRepository	54
18	Diagramma della classe AuthenticationService	55
19	Diagramma delle classi relativo alla funzionalità dell'invio email di soccorso	56
20	Diagramma della classe SosBottomSheetWidget	56
21	Diagramma della classe SosAlertViewModel	57
22	Diagramma della classe SosAlertRepository	57
23	Diagramma della classe SosAlertService	57
24	Diagramma delle classi relativo alle funzionalità del Chatbot	58
25	Diagramma della classe ChatbotScreen	59
26	Diagramma della classe ChatWidget	59
27	Diagramma della classe ChatHistoryWidget	60
28	Diagramma della classe ChatbotModeToggleWidget	60
29	Diagramma della classe ChatbotSendMessageWidget	61
30	Diagramma della classe ChatbotCreateChatWidget	61



31	Diagramma della classe ChatbotViewModel	61
32	Diagramma della classe Chat	62
33	Diagramma della classe ProxyChat	62
34	Diagramma della classe LocalChat	63
35	Diagramma della classe ChatMessage	63
36	Diagramma della classe ChatMode	64
37	Diagramma della classe MessageType	64
38	Diagramma della classe MessageResponse	65
39	Diagramma della classe ChatbotRepository	65
40	Diagramma della classe ChatbotService	66
41	Diagramma della classe ChatDTO	66
42	Diagramma delle classi relativo alle funzionalità del diario criptato	67
43	Diagramma della classe DiaryAccessScreen	67
44	Diagramma della classe PasswordFormWidget	67
45	Diagramma della classe DiaryAccessViewModel	68
46	Diagramma della classe DiaryAccountRepository	68
47	Diagramma della classe DiaryAccountService	68
48	Diagramma della classe DiaryAccessResult	69
49	Diagramma della classe DiaryScreen	69
50	Diagramma della classe DiaryPasswordSettingWidget	69
51	Diagramma della classe NoteListWidget	70
52	Diagramma della classe NoteActionsWidget	70
53	Diagramma della classe NoteEditorWidget	70
54	Diagramma della classe NoteAudioPlayerWidget	71
55	Diagramma della classe DiaryViewModel	71
56	Diagramma della classe DiarySession	72
57	Diagramma della classe DiaryType	72
58	Diagramma della classe Note	73
59	Diagramma della classe ProxyNote	73
60	Diagramma della classe LocalNote	74
61	Diagramma della classe NoteElement	74
62	Diagramma della classe NoteTextElement	74
63	Diagramma della classe NoteImgElement	75
64	Diagramma della classe NoteAudioElement	75
65	Diagramma della classe NoteService	76
66	Diagramma della classe NoteRepository	76
67	Diagramma della classe NoteDTO	76
68	Diagramma delle classi relativo alle funzionalità dei Contatti Fidati	77
69	Diagramma della classe TrustedContactScreen	77
70	Diagramma della classe TrustedContactListWidget	78



71	Diagramma della classe TrustedContactFormWidget	78
72	Diagramma della classe TrustedContactActionsWidget	78
73	Diagramma della classe TrustedContactViewModel	79
74	Diagramma della classe TrustedContact	79
75	Diagramma della classe TrustedContactRepository	80
76	Diagramma della classe TrustedContactService	80
77	Diagramma della classe TrustedContactDTO	80
78	Diagramma delle classi complessivo della funzionalità Luoghi Sicuri	81
79	Diagramma della classe SafePlaceMapScreen	81
80	Diagramma della classe SafePlaceMapWidget	82
81	Diagramma della classe SafePlaceViewModel	82
82	Diagramma della classe SafePlace	83
83	Diagramma della classe SafePlaceRepository	83
84	Diagramma della classe SafePlaceService	83
85	Diagramma della classe SafePlaceDTO	84
86	Diagramma delle classi relativo alla funzionalità del Dead Man's Switch	85
87	Diagramma della classe DeadManScreen	85
88	Diagramma della classe DeadManFormWidget	86
89	Diagramma della classe DeadManViewModel	86
90	Diagramma della classe DeadManSettings	87
91	Diagramma della classe DeadManRepository	87
92	Diagramma della classe DeadManService	87
93	Diagramma della classe DeadManSettingsDTO	88
94	Diagramma delle classi complessivo per l'Area Informativa	88
95	Diagramma della classe InfoHubScreen	88
96	Diagramma della classe CommunityScreen	89
97	Diagramma della classe CommunityListWidget	89
98	Diagramma della classe CommunityViewModel	89
99	Diagramma della classe Community	90
100	Diagramma della classe CommunityRepository	90
101	Diagramma della classe CommunityService	90
102	Diagramma della classe CommunityDTO	91
103	Diagramma della classe LegislationScreen	91
104	Diagramma della classe LegislationListWidget	91
105	Diagramma della classe LegislationViewModel	92
106	Diagramma della classe Legislation	92
107	Diagramma della classe LegislationRepository	92
108	Diagramma della classe LegislationService	93
109	Diagramma della classe LegislationDTO	93
110	Diagramma della classe InfoMaterialScreen	93



111	Diagramma della classe InfoMaterialListWidget	94
112	Diagramma della classe InfoMaterialViewModel	94
113	Diagramma della classe InfoMaterial	94
114	Diagramma della classe InfoMaterialRepository	95
115	Diagramma della classe InfoMaterialService	95
116	Diagramma della classe InfoMaterialDTO	95
117	Diagramma della classe EmergencyContactScreen	95
118	Diagramma della classe EmergencyContactListWidget	96
119	Diagramma della classe EmergencyContactViewModel	96
120	Diagramma della classe EmergencyContact	97
121	Diagramma della classe EmergencyContactRepository	97
122	Diagramma della classe EmergencyContactService	97
123	Diagramma della classe EmergencyContactDTO	98
124	Architettura e Flusso di Autenticazione	98
125	Architettura moduli Luoghi Sicuri	100
126	Componenti del microservizio luoghi sicuri	103
127	Diagramma della classe Marker	104
128	Diagramma della classe SafePlacesController	105
129	Diagramma dell'interfaccia GetMarkerPort	105
130	Diagramma della classe SafePlacesService	106
131	Diagramma dell'interfaccia SafePlacesRepositoryPort	106
132	Diagramma della classe S3SafePlacesRepository	106
133	Architettura moduli Materiale Informativo	107
134	Componenti del microservizio materiale informativo	109
135	Diagramma della classe Resource	110
136	Diagramma della classe ResourcesController	111
137	Diagramma della classe GetResourcesPort	111
138	Diagramma della classe ResourcesService	111
139	Diagramma della classe ResourcesRepositoryPort	112
140	Diagramma della classe S3ResourcesRepository	112
141	Architettura diari	113
142	Componenti della Lambda per la gestione dei diari	122
143	Diagramma dell'enumerazione DiaryType	123
144	Diagramma della classe Note	123
145	Diagramma della classe NoteElement	124
146	Diagramma della classe NoteElementDTO	124
147	Diagramma della classe NoteDTO	125
148	Diagramma della classe GetNoteCmd	125
149	Diagramma della classe GetNotesCmd	126
150	Diagramma della classe AddElementNewNoteCmd	126



151	Diagramma della classe AddNoteCmd	127
152	Diagramma della classe AddNoteElementCmd	127
153	Diagramma della classe DeleteNoteCmd	128
154	Diagramma della classe DeleteNoteElementCmd	128
155	Diagramma della classe ValidatePasswordCmd	129
156	Diagramma della classe ValidateTokenCmd	129
157	Diagramma della classe SetPasswordCmd	130
158	Diagramma della classe CheckPasswordCmd	130
159	Diagramma dell'interfaccia GetNotePort	131
160	Diagramma dell'interfaccia SetNotePort	131
161	Diagramma dell'interfaccia DeleteNotePort	131
162	Diagramma dell'interfaccia SetNoteElementPort	132
163	Diagramma dell'interfaccia DeleteNoteElementPort	132
164	Diagramma dell'interfaccia SetPasswordPort	132
165	Diagramma dell'interfaccia ValidationPort	133
166	Diagramma dell'interfaccia CheckPasswordPort	133
167	Diagramma dell'interfaccia NoteRepositoryPort	134
168	Diagramma dell'interfaccia FileRepositoryPort	134
169	Diagramma dell'interfaccia DiaryAuthRepositoryPort	135
170	Diagramma della classe NoteService	135
171	Diagramma della classe DiaryAuthService	136
172	Diagramma della classe DynamoNoteAdapter	136
173	Diagramma della classe S3NoteAdapter	137
174	Diagramma della classe DynamoAuthAdapter	137
175	Diagramma della classe DiaryNoteController	138
176	Diagramma della classe DiaryAccessController	139
177	Architettura chatbot	139
178	Componenti del microservizio Chatbot	146
179	Diagramma della classe Chat	147
180	Diagramma della classe ChatMessage	147
181	Diagramma dell'interfaccia ChatbotLLMPort	148
182	Diagramma della classe BedrockLLMAdapter	148
183	Diagramma della classe ChatbotLLMService	148
184	Diagramma dell'interfaccia ChatsRepositoryPort	149
185	Diagramma della classe DynamoChatAdapter	149
186	Diagramma dell'interfaccia CreateChatPort	150
187	Diagramma dell'interfaccia GetChatPort	150
188	Diagramma dell'interfaccia UpdateChatPort	151
189	Diagramma dell'interfaccia DeleteChatPort	151
190	Diagramma dell'interfaccia ChatMessagePort	151



191	Diagramma della classe ChatbotCRUDService	152
192	Diagramma della classe CreateChatCmd	152
193	Diagramma della classe GetChatCmd	153
194	Diagramma della classe GetChatListCmd	153
195	Diagramma della classe UpdateChatCmd	153
196	Diagramma della classe DeleteChatCmd	154
197	Diagramma della classe AddChatMessageCmd	154
198	Diagramma della classe ChatDTO	155
199	Diagramma della classe ChatMessageDTO	155
200	Diagramma della classe LambdaHandler	156
201	Architettura del sistema di contatti fidati, SOS e Dead man's switch	156
202	Componenti del microservizio contatti fidati, SOS e Dead man's switch . .	164
203	Diagramma della classe TrustedContact	165
204	Diagramma della classe TrustedContactDTO	165
205	Diagramma della classe AddTrustedContactCmd	166
206	Diagramma della classe GetTrustedContactCmd	166
207	Diagramma della classe DeleteTrustedContactCmd	166
208	Diagramma della classe DmsConfigurationSettings	167
209	Diagramma della classe DmsConfigurationSettingsDTO	167
210	Diagramma della classe AlertCmd	168
211	Diagramma della classe EmailMessage	168
212	Diagramma della classe TrustedContactController	169
213	Diagramma della classe SetDmsHeartBeatPort	169
214	Diagramma della classe GetDmsConfigurationSettingsPort	170
215	Diagramma della classe SetDmsConfigurationSettingsPort	170
216	Diagramma della classe UpdateDmsAlertPort	170
217	Diagramma della classe SetTrustedContactPort	171
218	Diagramma della classe GetTrustedContactPort	171
219	Diagramma della classe DeleteTrustedContactPort	171
220	Diagramma della classe SendAlertPort	172
221	Diagramma della classe DmsCRUDService	172
222	Diagramma della classe DmsAlertService	172
223	Diagramma della classe TrustedContactCRUDService	173
224	Diagramma della classe SOSAlertService	173
225	Diagramma della classe DmsRepositoryPort	174
226	Diagramma della classe TrustedContactRepositoryPort	175
227	Diagramma della classe NotificationPort	175
228	Diagramma della classe DynamoDmsAdapter	176
229	Diagramma della classe DynamoTrustedContactAdapter	176
230	Diagramma della classe SesNotificationAdapter	176



1. Introduzione

1.1. Scopo del documento

Il presente documento ha lo scopo di fornire una descrizione completa, formale e strutturata del prodotto software, delineandone in modo rigoroso le caratteristiche tecniche e architetturali.

La **Specifica Tecnica^G** rappresenta il riferimento principale per tutte le attività di progettazione, sviluppo, **Verifica^G** e manutenzione del sistema, assicurando coerenza rispetto ai requisiti definiti nel documento di **Analisi dei requisiti^G** e alle scelte progettuali adottate.

In particolare, il documento si propone di:

- Definire l'**Architettura^G** complessiva del sistema, descrivendo le componenti software, le loro responsabilità e le modalità di interazione attraverso appositi diagrammi
- Specificare i moduli funzionali e le interfacce esposte, includendo i contratti di comunicazione e i formati dei dati scambiati
- Descrivere l'**Architettura^G** di **Deployment^G**, evidenziando la distribuzione delle componenti nei diversi ambienti di esecuzione e le relative dipendenze infrastrutturali
- Documentare le tecnologie, i linguaggi di programmazione, i **Framework^G** e gli strumenti adottati, motivando le scelte effettuate in relazione ai requisiti di progetto e al **Capitolato^G**
- Illustrare i principali design pattern e le soluzioni architetturali utilizzate, con particolare attenzione agli aspetti di scalabilità, manutenibilità e sicurezza
- Fornire indicazioni sui metodi di testing e di **Validazione^G**
- Supportare le attività di evoluzione e manutenzione del software, garantendo una base documentale chiara e coerente.

1.2. Riferimenti

Al fine di evitare ambiguità relative al linguaggio e ai termini utilizzati all'interno dei documenti formali, viene allegato il [Glossario](#). I termini tecnici, gli acronimi e le parole con significato specifico all'interno del progetto sono contrassegnati da una lettera 'G' in apice (es. **Agile^G**) e sono descritti in dettaglio nel documento apposito.



1.2.1 Riferimenti normativi

- **Capitolato^G d'appalto C4: *L'app che Protegge e Trasforma***
Parti consultate: documento completo
Versione: 1.0
Ultimo accesso: 2026-01-14
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C4.pdf>
- **Norme di Progetto**
Parti consultate: documento completo
Versione: v.1.0.0
Ultimo accesso: 2026-02-09
https://swe-bitbybit.github.io/SWE-docs/docs/RTB/documenti_interni/norme_di_progetto.pdf

1.2.2 Riferimenti informativi

- **Slide sulla Progettazione Software del Prof. Vardanega**
Parti consultate: documento completo
Versione: A.A.2025/2026
Ultimo accesso: 2026-03-30
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/T06.pdf>
- **Materiale relativo alla parte pratica del corso di Ingegneria del Software**
Parti consultate: documento completo
Versione: A.A.2022/2023
Ultimo accesso: 2026-03-30
<https://www.math.unipd.it/~rcardin/swea/2022/>
- **repository *Github* del Prof. Cardin**
Ultimo accesso: 2026-03-30
<https://github.com/rcardin/swe-imdb>
- **Guida Flutter sull'Architettura^G di un'applicazione**
Ultimo accesso: 2026-03-30
<https://docs.flutter.dev/app-architecture/guide>
- **Glossario**
Parti consultate: documento completo
Versione: v.1.0.0
Ultimo accesso: 2026-02-27
https://swe-bitbybit.github.io/SWE-docs/docs/RTB/documenti_interni/Glossario.pdf



2. Tecnologie

Le tecnologie utilizzate sono state scelte trovando un punto d'incontro tra la necessità di garantire la sicurezza e la tutela dei dati personali degli utenti e il bisogno di realizzare un'**Architettura**^G solida, modulare e facilmente estendibile. Di seguito sono riportate tutte le tecnologie adottate.

2.1. Framework

2.1.1 Flutter

Flutter è un **Framework**^G open-source sviluppato da Google per la creazione di applicazioni multi-piattaforma a partire da un unico codice sorgente. Consente di sviluppare interfacce per dispositivi mobili, web e desktop utilizzando il linguaggio Dart. Flutter si basa su un sistema di widget componibili, che possono rappresentare qualsiasi elemento dell'interfaccia utente. Flutter include un proprio motore grafico (come Impeller o storicamente Skia) che disegna direttamente i componenti a schermo, garantendo elevata coerenza visiva e prestazioni indipendenti dalla piattaforma sottostante. È il **Framework**^G alla base dell'App Che Protegge e Trasforma, il che consente di poter generare build per iOS e Android da un unico codice sorgente, a fronte di un maggiore utilizzo di risorse rispetto alle soluzioni native e di una minore possibilità di controllo diretto e personalizzazione delle funzionalità native della piattaforma.

- **Versione utilizzata:** 3.41.6
- **Documentazione:** <https://docs.flutter.dev/>

2.2. Linguaggi di programmazione

2.2.1 Dart

Dart è un linguaggio di programmazione sviluppato da Google, progettato per la realizzazione di applicazioni client moderne. È un linguaggio tipizzato (con supporto sia statico che dinamico) e orientato agli oggetti. Viene utilizzato principalmente in combinazione con Flutter, dove il codice Dart viene compilato ahead-of-time in codice nativo per migliorare le prestazioni, oppure eseguito tramite compilazione just-in-time durante lo sviluppo. L'utilizzo di Flutter rende di fatto quasi obbligatoria l'adozione di Dart come linguaggio.

- **Versione utilizzata:** 3.11.4
- **Documentazione:** <https://dart.dev/docs>



2.2.2 Python

Python è un linguaggio di programmazione ad alto livello, interpretato e orientato agli oggetti, noto per la sua sintassi semplice e leggibile. Il codice Python viene eseguito da un interprete che traduce le istruzioni in bytecode, poi eseguito dalla macchina virtuale Python. Nel progetto è utilizzato lato **Backend^G** per la scrittura delle funzioni AWS Lambda e per i test tramite Pytest.

- **Versione utilizzata:** 3.14.3
- **Documentazione:** <https://docs.python.org/3/>

2.3. Servizi AWS

2.3.1 Lambda

AWS Lambda è un servizio **Serverless^G** che consente di eseguire codice senza gestire direttamente server o infrastruttura. Le funzioni vengono attivate in risposta a eventi e scalano automaticamente in base al carico. Nel progetto è utilizzato per implementare la logica **Backend^G**, eseguendo funzioni scritte in Python invocate tramite API Gateway.

- **Versione utilizzata:** Runtime Python 3.14
- **Documentazione:** <https://docs.aws.amazon.com/lambda/>

2.3.2 API Gateway

AWS API Gateway è un servizio che permette di creare, pubblicare e gestire **API REST^G** e HTTP, fungendo da punto di ingresso per le richieste client. Gestisce autenticazione, routing e integrazione con altri servizi AWS. Nel progetto è utilizzato per esporre endpoint HTTP che invocano le funzioni Lambda.

- **Versione utilizzata:** HTTP API (v2)
- **Documentazione:** <https://docs.aws.amazon.com/apigateway/>

2.3.3 DynamoDB

Amazon DynamoDB è un database NoSQL completamente gestito, progettato per offrire alte prestazioni e scalabilità automatica. I dati sono memorizzati in tabelle con chiavi primarie e accessibili con bassa latenza. Nel progetto è utilizzato per memorizzare dati strutturati come le note testuali del diario criptato e le chat con il chatbot.

- **Documentazione:** <https://docs.aws.amazon.com/dynamodb/>



2.3.4 S3

Amazon S3 (Simple Storage Service) è un servizio di storage a oggetti che consente di archiviare e recuperare dati in modo scalabile e sicuro. I file sono organizzati in bucket e accessibili tramite API. Nel progetto è utilizzato per memorizzare contenuti multimediali come immagini e file audio.

- **Documentazione:** <https://docs.aws.amazon.com/s3/>

2.3.5 Cognito

Amazon Cognito è un servizio di gestione delle identità che consente autenticazione, autorizzazione e gestione degli utenti. Supporta integrazione con provider esterni come Google. Nel progetto è utilizzato per gestire l'autenticazione degli utenti tramite account Google e per il controllo degli accessi.

- **Documentazione:** <https://docs.aws.amazon.com/cognito/>

2.3.6 Bedrock

Amazon Bedrock è un servizio che consente di utilizzare modelli di intelligenza artificiale generativa tramite API, senza gestire direttamente l'infrastruttura sottostante. Permette l'accesso a diversi modelli foundation per task come generazione di testo. Nel progetto è utilizzato per implementare la funzionalità di chatbot.

- **Documentazione:** <https://docs.aws.amazon.com/bedrock/>

2.3.7 SES

Amazon SES (Simple Email Service) è un servizio per l'invio e la ricezione di email in modo scalabile e affidabile. Supporta integrazione via API e SMTP. Nel progetto è utilizzato per inviare email ai contatti fidati degli utenti.

- **Documentazione:** <https://docs.aws.amazon.com/ses/>

2.3.8 CloudWatch

Amazon CloudWatch è un servizio di monitoraggio e osservabilità che raccoglie metriche, log ed eventi dai servizi AWS. Consente analisi, visualizzazione e configurazione di allarmi. Nel progetto è utilizzato per la raccolta e l'analisi dei log generati dalle funzioni Lambda.

- **Documentazione:** <https://docs.aws.amazon.com/cloudwatch/>



2.3.9 EventBridge

Amazon EventBridge è un servizio di event bus **Serverless^G** che consente di gestire e instradare eventi tra diversi servizi AWS. Esso permette di definire regole per reagire automaticamente a eventi generati da servizi AWS o da fonti esterne. Nel progetto è utilizzato per orchestrare i flussi di controllo di utilizzo dell'app da parte degli utenti, e l'invio delle email secondo le tempistiche stabilite dal Dead man's switch.

- **Documentazione:** <https://docs.aws.amazon.com/eventbridge/>

2.3.10 SAM

AWS **Serverless^G** Application Model (SAM) è un **Framework^G** open-source che semplifica la definizione e il **Deployment^G** di applicazioni **Serverless^G** su AWS. Estende AWS CloudFormation introducendo una sintassi più compatta per risorse come Lambda, API Gateway e DynamoDB e include strumenti per build, testing locale e deploy delle applicazioni. Nel progetto è utilizzato per definire e distribuire l'infrastruttura **Serverless^G** in modo semplificato.

- **Documentazione:** <https://docs.aws.amazon.com/serverless-application-model/>

2.3.11 CloudFormation

AWS CloudFormation è un servizio di **Infrastructure as Code^G** che consente di modellare e gestire risorse AWS tramite template dichiarativi in formato **YAML^G** o JSON. Nel progetto è utilizzato come base per il provisioning dell'infrastruttura, anche tramite template generati da SAM.

- **Documentazione:** <https://docs.aws.amazon.com/cloudformation/>
- **Versioni:** CloudFormation non prevede versioni di servizio esplicite; eventuali aggiornamenti sono gestiti direttamente da AWS. I template possono però utilizzare versioni di specifiche risorse e trasformazioni (es. `AWS::ServerlessG`).

2.4. Librerie

2.4.1 http

`http` è una libreria per il linguaggio Dart che consente di effettuare richieste HTTP in modo semplice e asincrono. Fornisce metodi per eseguire operazioni come GET, POST, PUT e DELETE, gestendo automaticamente la serializzazione delle richieste e la ricezione delle risposte. È progettata per essere leggera e facilmente integrabile in applicazioni client.



- **Versione utilizzata:** 1.6.0
- **Documentazione:** <https://pub.dev/packages/http>

2.4.2 image_picker

`image_picker` è una libreria Flutter che permette di accedere alla fotocamera e alla galleria del dispositivo per selezionare o acquisire immagini e video. Fornisce un'interfaccia unificata tra piattaforme diverse, gestendo le differenze tra iOS e Android.

- **Versione utilizzata:** 1.2.1
- **Documentazione:** https://pub.dev/packages/image_picker

2.4.3 path_provider

`path_provider` è una libreria Flutter che consente di ottenere i percorsi delle directory del file system del dispositivo, come directory temporanee o documenti applicativi. Facilita l'accesso a percorsi corretti e compatibili tra piattaforme per la gestione dei file.

- **Versione utilizzata:** 2.1.5
- **Documentazione:** https://pub.dev/packages/path_provider

2.4.4 provider

`provider` è una libreria per Flutter che offre un meccanismo di gestione dello stato e di propagazione delle dipendenze lungo l'albero dei widget. Si basa sul sistema nativo di `InheritedWidget` di Flutter, semplificandone notevolmente l'utilizzo attraverso un'API dichiarativa. Consente di rendere disponibili oggetti (come i ViewModel) a interi sottoalberi dell'interfaccia, facilitando la comunicazione reattiva tra i layer dell'applicazione.

- **Versione utilizzata:** 6.1.5+1
- **Documentazione:** <https://pub.dev/packages/provider>

2.4.5 url_launcher

`url_launcher` è una libreria Flutter che consente di aprire URL nel browser predefinito del dispositivo, avviare applicazioni di posta elettronica, effettuare chiamate telefoniche o aprire altri tipi di risorse tramite i rispettivi handler nativi. Fornisce un'interfaccia unificata e compatibile tra le diverse piattaforme supportate da Flutter.

- **Versione utilizzata:** 6.3.2
- **Documentazione:** https://pub.dev/packages/url_launcher



2.4.6 flutter_dotenv

`flutter_dotenv` è una libreria che permette di caricare variabili di configurazione da un file `.env` all'interno di un'applicazione Flutter. Consente di separare le configurazioni sensibili o dipendenti dall'ambiente (come chiavi API o endpoint) dal codice sorgente, rendendo più agevole la gestione di ambienti diversi senza modificare il codice dell'applicazione.

- **Versione utilizzata:** 6.0.0
- **Documentazione:** https://pub.dev/packages/flutter_dotenv

2.4.7 flutter_web_auth_2

`flutter_web_auth_2` è una libreria Flutter progettata per gestire flussi di autenticazione basati su protocolli web standard come OAuth 2.0 e OpenID Connect. Apre una sessione browser controllata, intercetta il redirect dell'URL di callback e restituisce il codice di autorizzazione all'applicazione, abilitando l'integrazione con provider di identità esterni senza esporre le credenziali nel codice nativo.

- **Versione utilizzata:** 5.0.2
- **Documentazione:** https://pub.dev/packages/flutter_web_auth_2

2.4.8 flutter_map

`flutter_map` è una libreria Flutter open-source per la visualizzazione di mappe interattive. Fornisce funzionalità per il rendering di layer multipli, gestione di marker, polilinee e poligoni, oltre al supporto per gesture come zoom e pan. È altamente configurabile e non dipende da servizi proprietari esterni come Google Maps.

- **Versione utilizzata:** 8.3.0
- **Documentazione:** https://pub.dev/packages/flutter_map

2.4.9 geolocator

`geolocator` è una libreria Flutter che consente di accedere ai servizi di geolocalizzazione del dispositivo. Permette di ottenere la posizione corrente, monitorare aggiornamenti di posizione in tempo reale e gestire i permessi richiesti dalle diverse piattaforme.

- **Versione utilizzata:** 14.0.2
- **Documentazione:** <https://pub.dev/packages/geolocator>



2.4.10 get_it

`get_it` è un service locator per Dart e Flutter utilizzato per la gestione delle dipendenze all'interno dell'applicazione. Consente di registrare e recuperare istanze di oggetti in modo centralizzato, facilitando l'implementazione di architetture modulari e testabili.

- **Versione utilizzata:** 9.2.1
- **Documentazione:** https://pub.dev/packages/get_it

2.4.11 latlong2

`latlong2` è una libreria Dart che fornisce strutture dati e utilities per la gestione di coordinate geografiche (latitudine e longitudine). Include funzionalità per il calcolo di distanze tra punti e altre operazioni geospaziali di base.

- **Versione utilizzata:** 0.10.1
- **Documentazione:** <https://pub.dev/packages/latlong2>

2.4.12 path

`path` è una libreria Dart che fornisce strumenti per la gestione e manipolazione dei percorsi file, indipendenti dalla piattaforma di esecuzione.

- **Versione utilizzata:** 1.9.1
- **Documentazione** <https://pub.dev/packages/path>

2.4.13 flutter_secure_storage

`flutter_secure_storage` è una libreria Flutter che consente il salvataggio di dati sensibili in modo sicuro e specifico per la piattaforma di esecuzione. I dati vengono salvati come coppie chiave-valore e possono essere criptati prima del salvataggio.

- **Versione utilizzata:** 10.0.0
- **Documentazione** https://pub.dev/packages/flutter_secure_storage

2.4.14 intl

`intl` è una libreria Dart per l'internazionalizzazione e localizzazione, e fornisce funzionalità che includono la formattazione e il parsing di date e testo.

- **Versione utilizzata:** 0.20.2
- **Documentazione** <https://pub.dev/packages/intl>



2.4.15 just_audio

`just_audio` è una libreria Flutter che fornisce strumenti per la creazione e utilizzo di player audio, e supporta molteplici formati di tracce audio.

- **Versione utilizzata:** 0.10.5
- **Documentazione** https://pub.dev/packages/just_audio

2.4.16 plugin_platform_interface

`plugin_platform_interface` è una libreria Dart che mette a disposizione una classe base per platform interface con funzionalità per garantire che le loro implementazioni usino la keyword `extend`, cosicchè le sottoclassi abbiano accesso alle implementazioni dei metodi della classe base.

- **Versione utilizzata:** 2.1.8
- **Documentazione** https://pub.dev/packages/plugin_platform_interface/versions

2.4.17 file_picker

`file_picker` è una libreria Flutter che permette l'utilizzo del file picker nativo per la selezione di uno o più file. La libreria permette anche di filtrare i file selezionabili dall'`UtenteG` in base al formato.

- **Versione utilizzata:** 11.0.2
- **Documentazione** https://pub.dev/packages/file_picker

2.4.18 url_launcher_platform_interface

`url_launcher_platform_interface` è una libreria flutter che permette estensioni platform-specific del plugin `url_launcher`.

- **Versione utilizzata:** 2.3.2
- **Documentazione** https://pub.dev/packages/url_launcher_platform_interface

2.4.19 command_it

`command_it` è una libreria Flutter che permette l'implementazione del pattern *Command*, tramite la creazione di wrap functions con gestione automatica dello stato.

- **Versione utilizzata:** 9.5.1
- **Documentazione** https://pub.dev/packages/command_it



2.4.20 listen_it

`listen_it` è una libreria Flutter che fornisce primitive reattive per la creazione di oggetti che notificano automaticamente eventuali ascoltatori in seguito a mutamenti del proprio stato.

- **Versione utilizzata:** 5.3.5
- **Documentazione** https://pub.dev/packages/listen_it

2.4.21 mocktail

`mocktail` è una libreria Dart che fornisce una API per la creazione di Mock senza doverli creare da zero o usare la generazione di codice.

- **Versione utilizzata:** 1.0.5
- **Documentazione** <https://pub.dev/packages/mocktail>

2.4.22 shared_preferences

`shared_preferences` è una libreria Flutter che permette l'utilizzo della memorizzazione persistente platform-specific per il salvataggio di dati semplici.

- **Versione utilizzata:** 2.5.5
- **Documentazione** https://pub.dev/packages/shared_preferences

2.4.23 connectivity_plus

`connectivity_plus` è una libreria Flutter che permette all'applicazione di conoscere i tipi di connessione disponibili in un dato momento, in modo da reagire di conseguenza.

- **Versione utilizzata:** 7.1.1
- **Documentazione** https://pub.dev/packages/connectivity_plus

2.4.24 audio_session

`audio_session` è una libreria Flutter per la gestione delle sessioni audio dell'applicazione. Permette una configurazione molto approfondita delle sessioni audio, permettendo di specificare caratteristiche quali il comportamento in presenza di audio appartenenti ad altre applicazioni o quali dispositivi possono riprodurre l'audio.

- **Versione utilizzata:** 0.2.3
- **Documentazione** https://pub.dev/packages/audio_session



2.4.25 python-ulid

`python-ulid` è una libreria Python per la generazione di identificatori univoci ULID (Universally Unique Lexicographically Sortable Identifier). Gli ULID combinano unicità e ordinamento temporale, risultando particolarmente utili in sistemi distribuiti dove è necessario generare ID ordinabili senza coordinamento centrale.

- **Versione utilizzata:** 3.1.0
- **Documentazione:** <https://pypi.org/project/python-ulid/>

2.4.26 rank-bm25

`rank-bm25` è una libreria Python che implementa l'algoritmo di ranking BM25 (Best Matching 25), utilizzato nel campo dell'information retrieval per valutare la rilevanza dei documenti rispetto a una query testuale. Fornisce diverse varianti dell'algoritmo BM25 e consente di costruire rapidamente sistemi di ricerca basati su punteggi statistici senza necessità di motori di ricerca completi.

- **Versione utilizzata:** 0.2.2
- **Documentazione:** <https://pypi.org/project/rank-bm25/>

2.4.27 boto3

`boto3` è una libreria Python che fornisce strumenti per la creazione, configurazione e gestione dei servizi Amazon Web Services.

- **Versione utilizzata:** 1.43.5
- **Documentazione:** <https://docs.aws.amazon.com/boto3/latest/>

2.5. Tecnologie di analisi statica

2.5.1 SonarQube

SonarQube è una piattaforma open-source per l'analisi continua della qualità del codice. Esegue **Analisi Statica**^G su diversi linguaggi di programmazione per individuare bug, vulnerabilità, code smells e problemi di copertura dei test. Il funzionamento si basa su scanner che analizzano il codice sorgente e inviano i risultati a un server centrale, dove vengono aggregati e visualizzati tramite una dashboard dedicata.

- **Versione utilizzata:** 26.3.0
- **Documentazione:** <https://docs.sonarsource.com/sonarqube/>



2.6. Tecnologie di analisi dinamica

2.6.1 Pytest

Pytest è un **Framework**^G di testing per il linguaggio Python che consente di scrivere test in modo semplice e scalabile. Supporta test funzionali, unitari e di integrazione, offrendo funzionalità come fixture, parametrizzazione dei test e scoperta automatica dei test.

- **Versione utilizzata:** 9.0.2
- **Documentazione:** <https://docs.pytest.org/>

2.6.2 Moto

Moto è una libreria Python che consente di mockare i servizi AWS durante i test. Simula il comportamento delle API AWS, permettendo di testare codice che interagisce con servizi **Cloud**^G senza effettuare chiamate reali. Funziona intercettando le richieste e restituendo risposte simulate coerenti con quelle dei servizi AWS.

- **Versione utilizzata:** 5.1.23
- **Documentazione:** <https://docs.getmoto.org/>

2.7. Altro

2.7.1 Ollama

Ollama è una piattaforma che consente di eseguire e gestire LLM in locale tramite una semplice interfaccia. Permette il download, l'esecuzione e l'interazione con modelli di intelligenza artificiale direttamente in ambiente locale, senza necessità di servizi **Cloud**^G esterni. Il funzionamento si basa su un runtime che gestisce i modelli e espone endpoint per inferenza.

- **Versione utilizzata:** 0.18.3
- **Documentazione:** <https://ollama.com/docs>

2.7.2 Docker

Docker è una piattaforma di containerizzazione che consente di creare, distribuire ed eseguire applicazioni all'interno di ambienti isolati (container). I container includono tutto il necessario per eseguire un'applicazione (codice, runtime, librerie e dipendenze), garantendo portabilità e consistenza tra ambienti diversi. Nel progetto è utilizzato a supporto delle pipeline di **CI/CD**^G e per testare in locale servizi come DynamoDB e S3.

- **Documentazione:** <https://docs.docker.com/>



3. Architettura

3.1. Architettura logica

Il sistema software è progettato secondo i principi di un'**Architettura a Strati^G** (o *Layered Architecture*), che impone una chiara separazione delle responsabilità tra i diversi livelli applicativi del sistema. L'obiettivo cardine di questa scelta è massimizzare l'indipendenza formale della logica di business dalle interfacce grafiche **Utente^G** e dai meccanismi fisici di gestione del dato, favorendo così la facilità di test, la manutenzione nel tempo e il forte disaccoppiamento (*Separation of Concerns^G*).

All'ingresso superiore dell'applicazione si posiziona l'ecosistema logico del dispositivo, mentre le fondamenta sono governate e calcolate dall'infrastruttura di servizio. L'interfaccia tra questi due mondi è ben definita tramite API e totalmente slegata dai linguaggi di programmazione o dai sistemi operativi utilizzati.

3.1.1 Frontend Mobile

Il client mobile è sviluppato incapsulando il codice secondo i dettami moderni di una **Clean Architecture^G** (fortemente affine al noto pattern architetturale *Model-View-ViewModel* - MVVM). Il codice dell'applicazione è suddiviso in almeno tre strati logici indipendenti e gerarchici:

- **Presentation Layer (View):** Costituisce il livello più visibile e frontale dell'applicazione. Riceve gli input da parte dell'**Utente^G** e si occupa unicamente di aggregare e renderizzare il layout grafico a schermo tramite widget componenti, senza mai eseguire logica complessa.
- **State Management^G Layer (ViewModel):** Funge da mediatore tra View e dominio, gestendo lo stato della view. Riceve input dalla view, muta lo stato dell'UI e notifica la view che si aggiornerà sulla base del nuovo stato incapsulato (es. passando da "In caricamento" a "Successo"). Inoltre, gestisce le validazioni elementari dei dati inseriti, comandando ai servizi sottostanti le operazioni formali da compiere.
- **Data Layer^G:** È lo strato più basso dell'ambiente Client, incaricato unicamente dell'acquisizione e gestione dei dati, ed è logicamente suddiviso in *Service* e *repository*. I Service sono gli unici componenti ad interfacciarsi con l'esterno, conoscendo gli endpoint del **Backend^G** per eseguire le chiamate asincrone HTTP e gestendo l'accesso a dati memorizzati localmente o in remoto. I repository fungono da intermediari: raccolgono i dati grezzi provenienti dai Service e li formattano in modelli di dominio strutturati (es. un oggetto *Note*), fornendoli al ViewModel puliti e pronti all'uso.



3.1.2 Backend Serverless

Ogni Lambda Function costituisce un microservizio indipendente, implementato seguendo il pattern dell'**Architettura^G** esagonale (conosciuto anche come *Ports and Adapters pattern*). Questo approccio garantisce un basso accoppiamento tra le componenti del sistema, isolando la domain logic dalle tecnologie esterne e facilitando la scrittura dei test. In questo pattern il sistema comunica con le componenti esterne attraverso interfacce, le port, mentre gli adapter sono responsabili dell'implementazioni concrete di queste.

Il nucleo del sistema è rappresentato dalla domain logic, che racchiude sia la logica applicativa che quella di business. È implementata nelle classi **Service**, che non dipendono da alcuna tecnologia specifica e interagiscono con le risorse esterne esclusivamente attraverso interfacce. Le operazioni esposte dai **Service** accettano oggetti di dominio o Command come input, oggetti che incapsulano i dati necessari per eseguire un'azione specifica.

Le port si dividono in due categorie:

- Inbound port: definiscono le interfacce attraverso cui le risorse esterne comunicano con la domain logic. Il loro utilizzo isola il nucleo del sistema da qualsiasi dipendenza tecnologica esterna, rendendo intercambiabili le implementazioni che le realizzano.
- Outbound port: definiscono le interfacce attraverso cui la domain logic interagisce con i servizi esterni, come le risorse di persistenza. In particolare, le **Repository^G** port specificano i metodi che gli adapter devono implementare per le operazioni di lettura e scrittura sui dati.

Gli adapter si dividono anche essi in 2 categorie:

- Inbound adapter: sono rappresentati dalle classi **Controller**, responsabili della ricezione degli eventi AWS Lambda e della loro traduzione in oggetti di dominio comprensibili alla domain logic. Il **Controller** determina la route dall'evento in ingresso, esegue il parsing del body e dei parametri, costruisce il body della risposta da restituire al client e delega l'esecuzione alla domain logic attraverso le inbound port.
- Outbound adapter: costituiscono l'implementazione concreta delle outbound port per i servizi AWS utilizzati da ciascun microservizio. Ogni adapter incapsula il client del servizio AWS corrispondente, ad esempio DynamoDB per la persistenza dei dati e S3 per la gestione dei file multimediali, esponendo i metodi definiti dalla porta. Questo approccio permette di sostituire il servizio sottostante modificando esclusivamente l'adapter, senza impatto sulla *domain logic*.



3.2. Design pattern utilizzati

3.2.1 Model-view-viewmodel

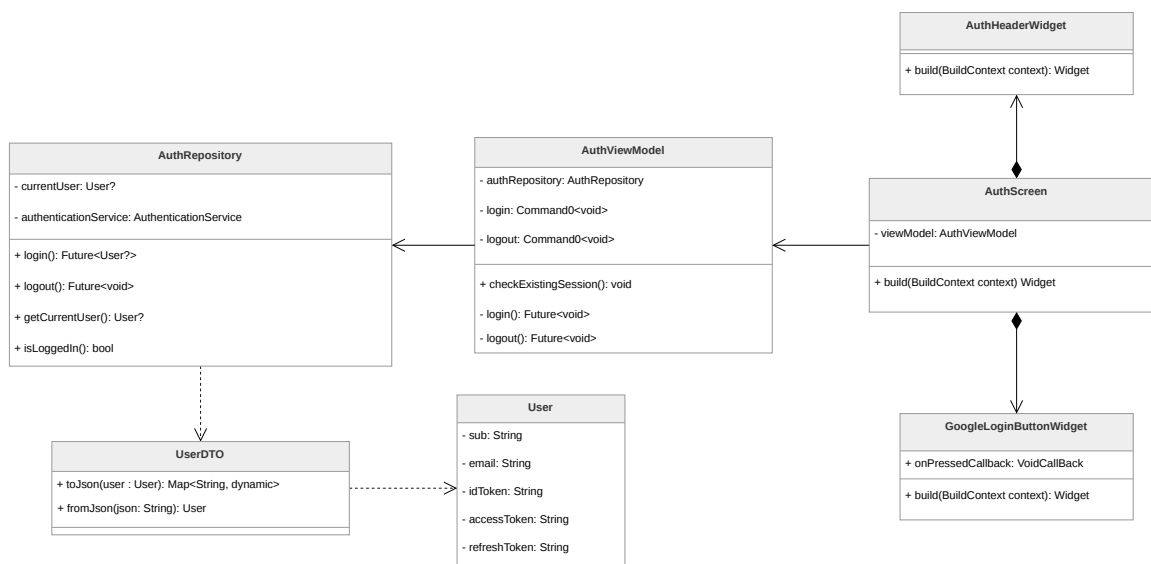


Figura 1: Rappresentazione del pattern MVVM nel modulo di Autenticazione

3.2.1.1 Descrizione del pattern:

Il pattern MVVM prevede la separazione tra la resa grafica, la **Presentation Logic^G** e la **Business Logic^G** in tre componenti principali:

- **Model:** contiene il codice responsabile per la **Business Logic^G** dell'applicazione.
- **View:** contiene la parte responsabile della resa grafica del software e della cattura delle interazioni dell'**Utente^G**, a cui reagisce invocando i metodi o i comandi messi a disposizione dal *ViewModel*.
- **ViewModel:** mantiene e gestisce lo stato della *View* e coordina le interazioni tra quest'ultima e il *Model*. La *View* può modificare il proprio stato esclusivamente attraverso l'interazione con i metodi e i **Command** esposti dal *ViewModel*.

Nel contesto del **Framework^G** Flutter, l'implementazione di questo pattern viene riadattata per assecondare la natura dichiarativa dell'interfaccia. Nello specifico, il *ViewModel* è tipicamente implementato sfruttando una classe o un mixin proprietario chiamato **ChangeNotifier**, il quale consente di notificare esplicitamente la *View* dei cambiamenti di stato interno. La *View*, composta a sua volta da *Widget*, ascolta reattivamente tali notifiche potendo ricostruire e aggiornare solo le specifiche porzioni grafiche necessarie.

3.2.1.2 Ragioni per l'utilizzo:

L'utilizzo del pattern garantisce notevoli vantaggi poiché permette una più semplice ed



efficace gestione delle varie parti dell'applicazione, limitando il forte accoppiamento (*coupling*) tra l'interfaccia utente visiva e la logica di business o di presentazione sottostante. Questo netto disaccoppiamento migliora la manutenibilità del codice dell'applicativo e agevola considerevolmente lo sviluppo e l'esecuzione degli unit test, in quanto permette di testare i ViewModel e i Model isolandoli dal ciclo di vita visivo tipico del sistema operativo e del **Framework**^G.

3.2.1.3 Riferimenti nel progetto:

Il pattern MVVM è ampiamente utilizzato per strutturare l'interfaccia utente dell'applicazione e la logica ad essa associata, separando la resa grafica dalla gestione dello stato e dei dati. Di seguito sono elencati i punti principali dove viene impiegato; all'interno di tali sezioni sono fornite spiegazioni dettagliate su come il pattern interagisce con il sistema circostante:

- Autenticazione → **AuthViewModel**
- Chatbot → **ChatbotViewModel**
- Diario → **DiaryViewModel**
- Contatti Fidati → **TrustedContactViewModel**



3.2.2 Observer

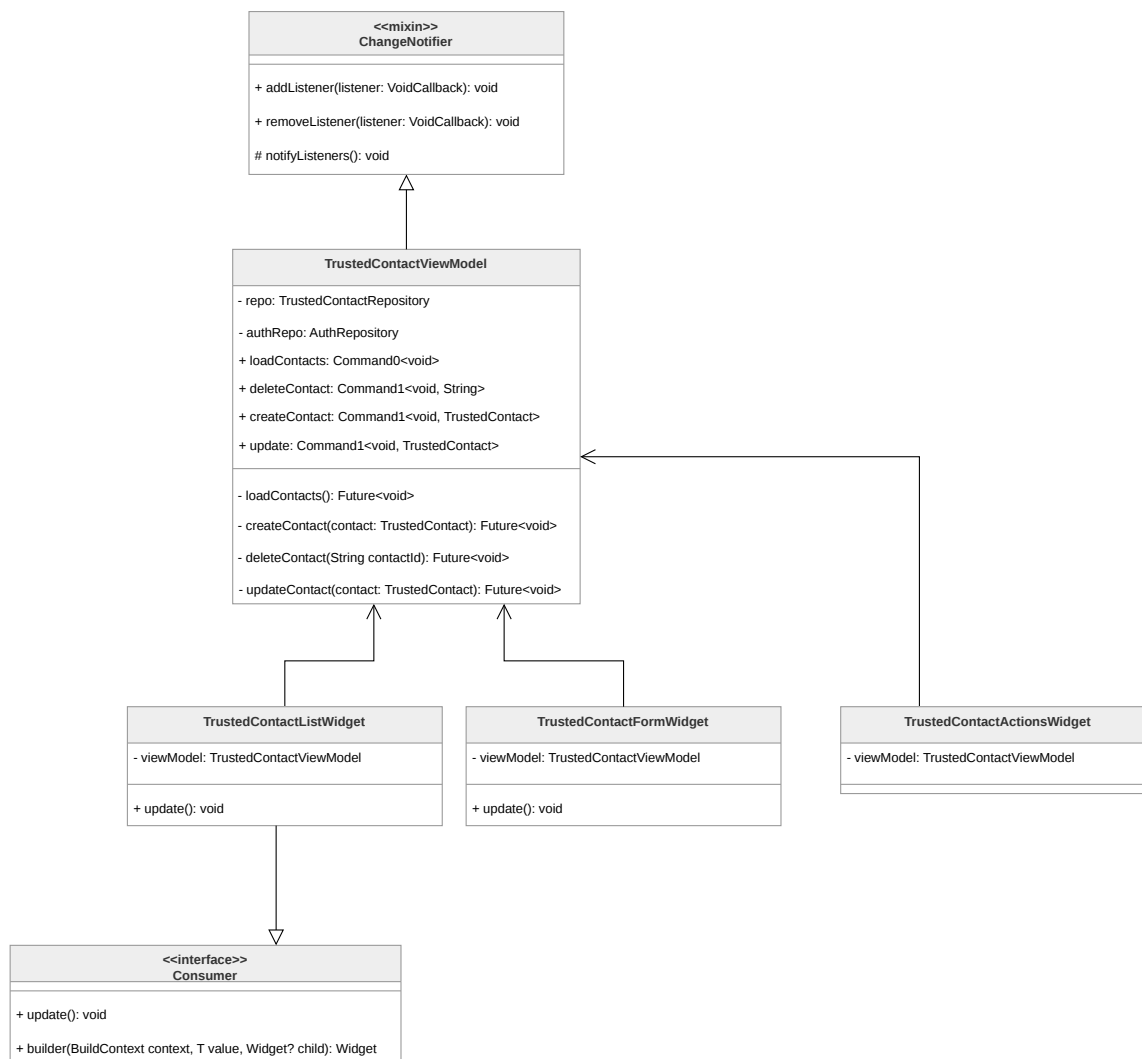


Figura 2: Rappresentazione del pattern Observer applicato nel modulo dei contatti fidati

3.2.2.1 Descrizione del pattern:

Il pattern Observer definisce la capacità per un oggetto *Publisher* di definire un meccanismo di "iscrizione" per cui un insieme di oggetti *Subscriber* può "osservarlo" ed essere notificato solo in seguito ad un suo cambiamento di stato.

Nel contesto di Flutter, l'integrazione di questo pattern avviene tramite l'utilizzo del mixin **ChangeNotifier** e dal widget specializzato **Consumer**. Il *ViewModel* agisce da *Publisher*: ogni qualvolta rileva un cambiamento rilevante nei dati o nello stato, invoca il metodo `notifyListeners()` per segnalare la necessità di un aggiornamento. Il *Consumer* funge da *Subscriber*: esso è un widget che osserva e utilizza i dati forniti dal *ViewModel* e, ricevuta la notifica di cambiamento, innesca la ricostruzione automatica della propria interfaccia. Questo meccanismo è fondamentale nel nostro caso, perché assicura che solo la porzione di interfaccia racchiusa nel **Consumer** (ovvero quella che effettivamente dipende



dai dati mutati) venga aggiornata, ottimizzando le prestazioni ed evitando ricostruzioni costose e inutili dell'intera schermata.

3.2.2.2 Ragioni per l'utilizzo:

Il pattern Observer è utilizzato per permettere ai ViewModel di notificare solo i widget che richiedono modifiche dopo un cambiamento di stato, ottimizzando la ricostruzione dell'UI in modo da non dover ricostruire l'intera schermata dopo una qualsiasi interazione. Il meccanismo di "iscrizione" rimuove la necessità di verifiche frequenti sullo stato del *Publisher* da parte dei *Subscriber* e rende il codice più facilmente estendibile e mantenibile, dato che la modifica di una componente non va a influire sulle altre. Ad esempio, l'introduzione di un nuovo widget nella *View* non implica la necessità di modificare il *ViewModel*.

3.2.2.3 Riferimenti nel progetto:

Essendo strettamente interconnesso al MVVM, il pattern Observer è utilizzato per la gestione dell'intera UI. Di seguito i punti principali dove i ViewModel agiscono da *Publisher* per le rispettive schermate:

- Autenticazione → **AuthViewModel**
- Chatbot → **ChatbotViewModel**
- Diario → **DiaryViewModel**
- Contatti Fidati → **TrustedContactViewModel**

3.2.3 Proxy

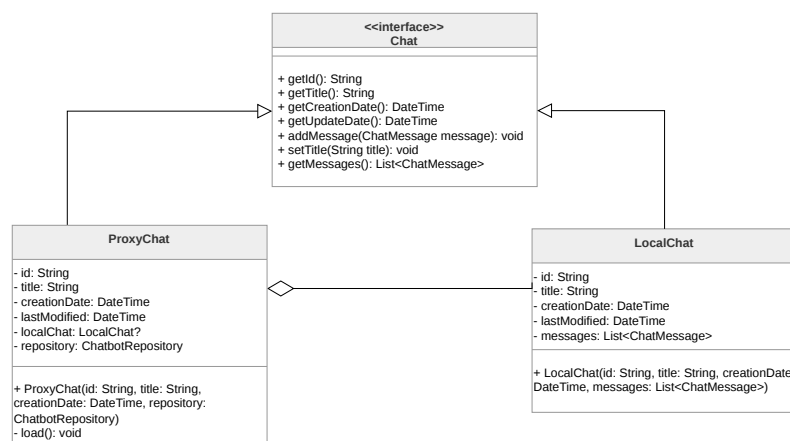


Figura 3: Rappresentazione del pattern Proxy applicato alla gestione delle chat



3.2.3.1 Descrizione del pattern:

Il pattern Proxy permette la creazione di un oggetto *Proxy* che controlla l'accesso all'oggetto reale. L'oggetto *Proxy* implementa la medesima interfaccia dell'oggetto reale, rendendolo perfettamente intercambiabile agli occhi del codice che lo utilizza, e ha il compito di delegare le richieste a un'istanza dell'oggetto reale solo quando necessario.

3.2.3.2 Ragioni per l'utilizzo:

L'utilizzo di questo pattern permette di implementare il caricamento ritardato di strutture dati potenzialmente voluminose; nello specifico, le classi che fungono da Proxy operano come segnaposti leggeri contenenti solo metadati essenziali, rimandando il caricamento completo dei contenuti (come i messaggi di una singola chat) al momento del loro effettivo accesso. Questo approccio permette di ridurre sensibilmente l'utilizzo di banda, limitando la quantità di dati recuperati dal **Backend^G** a quelli strettamente necessari, garantendo al contempo una maggiore fluidità nel caricamento degli elenchi e un consumo di risorse sul dispositivo più efficiente.

3.2.3.3 Riferimenti nel progetto:

Il pattern Proxy è utilizzato principalmente per la gestione dei modelli di dati persistenti:

- Chatbot → **ProxyChat**
- Diari Personali → **ProxyNote**

3.2.4 Singleton

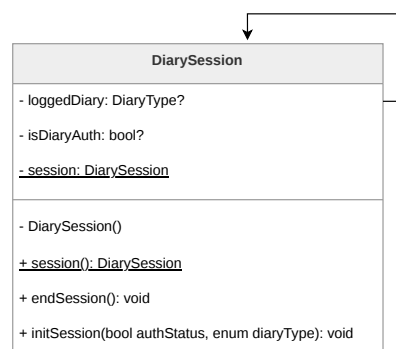


Figura 4: Rappresentazione del pattern Singleton applicato alla sessione del diario

3.2.4.1 Descrizione del pattern:

Il pattern Singleton garantisce che una classe abbia una sola istanza attiva in ogni momento e fornisce un punto di accesso globale a tale istanza. Questa proprietà si ottiene



rendendo privato il costruttore della classe, impedendone l'istanziatura diretta dall'esterno, e mettendo a disposizione un metodo statico o una proprietà (*getter*) che restituisce l'unica istanza disponibile.

Nell'applicazione, questo pattern viene implementato seguendo gli standard del linguaggio Dart, utilizzando un costruttore privato e una variabile statica per mantenere il riferimento all'unica istanza creata al primo accesso.

3.2.4.2 Ragioni per l'utilizzo:

L'impiego del Singleton è fondamentale per gestire componenti che devono mantenere uno stato unico e coerente in tutto il sistema. Nello specifico, garantisce l'esistenza di una sola sessione attiva per il diario, impedendo che l'**Utente^G** possa accedere contemporaneamente alla versione criptata e a quella fittizia. Questo approccio previene la condivisione accidentale di dati sensibili e assicura che ogni operazione di lettura o scrittura avvenga sempre nel contesto dell'archivio corretto, eliminando alla radice potenziali conflitti di sincronizzazione.

3.2.4.3 Riferimenti nel progetto:

Il pattern Singleton è utilizzato per il coordinamento dei componenti di sessione:

- Diari Personali → **DiarySession**

3.2.5 Dependency Injection

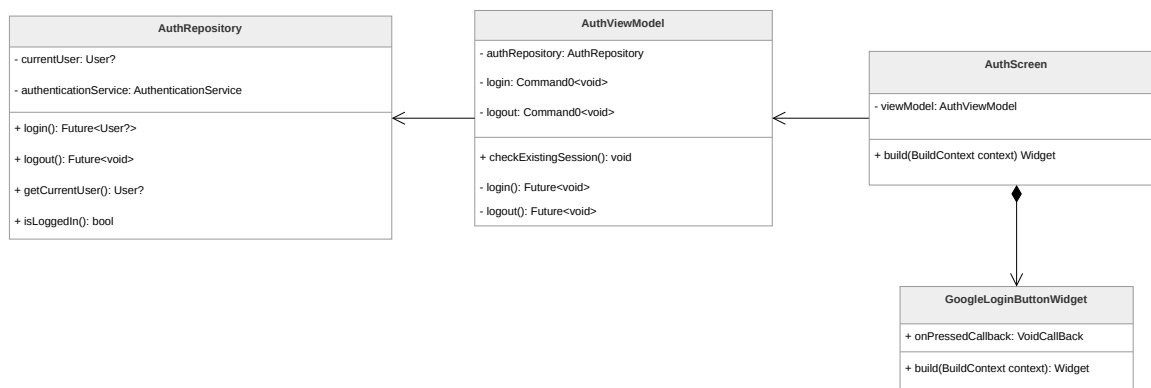


Figura 5: Rappresentazione del pattern Dependency Injection nel modulo di Autenticazione

3.2.5.1 Descrizione del pattern:

Il pattern Dependency Injection (DI) è una tecnica in cui un oggetto riceve dall'esterno le istanze degli oggetti da cui dipende, anziché doverle creare internamente. Nel contesto dell'applicazione viene utilizzata in forma di *constructor injection*: le dipendenze vengono passate ai componenti direttamente come parametri del costruttore durante la fase di istanziamento.



Questo approccio permette di slegare i componenti logici dalla creazione delle proprie risorse. I *ViewModel*, ad esempio, non istanziano mai i propri *repository*, ma li ricevono già configurati dal punto di ingresso del modulo o dell'applicazione, garantendo che ogni componente operi esclusivamente con le interfacce necessarie.

3.2.5.2 Ragioni per l'utilizzo:

L'utilizzo della DI favorisce un'**Architettura^G** modulare e flessibile, riducendo l'accoppiamento tra le classi. Il vantaggio principale risiede nella testabilità: l'iniezione tramite costruttore permette di sostituire facilmente le dipendenze reali con oggetti fittizi (*mock*), consentendo l'esecuzione di unit test in isolamento. Inoltre, centralizzare la logica di creazione degli oggetti facilita la manutenzione e l'eventuale sostituzione di implementazioni specifiche senza dover modificare i client che le utilizzano.

3.2.5.3 Riferimenti nel progetto:

Il pattern è applicato diffusamente in tutta l'applicazione. Di seguito alcune classi che integrano attivamente questo approccio:

- Autenticazione → `AuthViewModel`
- Chatbot → `ChatbotViewModel`
- Contatti Fidati → `TrustedContactViewModel`



3.2.6 Command

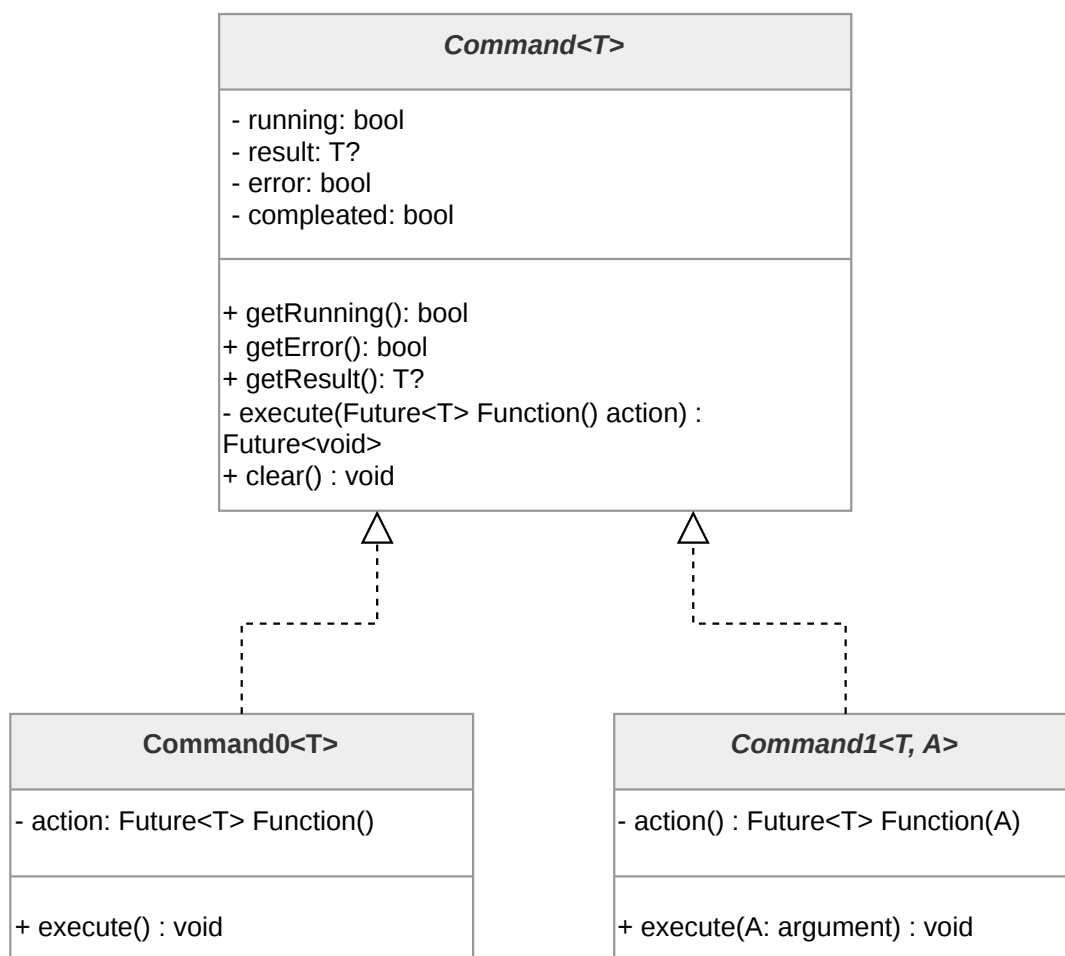


Figura 6: Rappresentazione del pattern Command

3.2.6.1 Descrizione del pattern:

Il pattern Command è un design pattern comportamentale che incapsula una richiesta come un oggetto autonomo, consentendo di parametrizzare i client con diverse richieste, accodare o registrare le esecuzioni e supportare operazioni annullabili. Nel contesto di un'applicazione Flutter governata dal pattern MVVM, il pattern Command è utilizzato per incapsulare e standardizzare le azioni invocabili dall'interfaccia utente, esponendo comandi robusti e isolati dal layer di View.

3.2.6.2 Ragioni per l'utilizzo:

Il pattern command permette di ottenere un forte disaccoppiamento tra la componente visiva che invoca l'azione (la View) e il modulo deputato all'esecuzione logica (il ViewModel o strati inferiori). Isolando e reificando l'interazione in un oggetto *Command*, lo sviluppo



beneficia in standardizzazione, accentrando il controllo della gestione di esecuzione asincrona e dei relativi stati interni dell'azione (attesa di elaborazione, conferme di successo o eccezioni previste). Questo elimina quasi integralmente le ridondanze nel codice e mitiga il verificarsi di interazioni non volute, come l'avvio accidentale e simultaneo di richieste doppie a seguito di pressioni ripetute sull'interfaccia.

3.2.6.3 Riferimenti nel progetto:

Il pattern viene implementato sistematicamente in tutti i ViewModel dell'applicazione per la gestione delle interazioni **Utente^G** e delle operazioni asincrone. Di seguito sono riportate le principali classi che costituiscono l'infrastruttura del pattern Command nell'applicazione:

3.2.6.4 Command<T>

È la classe base astratta del pattern. Mantiene internamente lo stato dell'azione tramite i campi privati `_running`, `_result` e `_error`. Espone queste informazioni tramite getter pubblici che la View utilizza per aggiornare l'interfaccia. Il metodo privato `_execute()` contiene la logica comune a tutte le implementazioni concrete: impedisce esecuzioni multiple simultanee, aggiorna lo stato prima e dopo l'esecuzione dell'azione e gestisce le eccezioni.

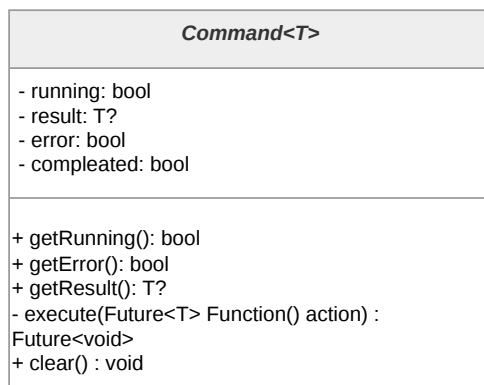


Figura 7: Diagramma della classe **Command<T>**

3.2.6.5 Command0<T>

È l'implementazione concreta di **Command<T>** per azioni senza argomenti. Riceve nel costruttore una funzione di tipo **Future<T> Function()** che rappresenta l'azione da eseguire. Espone il metodo pubblico `execute()`, che richiama il metodo `_execute()` della classe base che esegue la funzione memorizzata come stato della specifica istanza della classe **Command**.

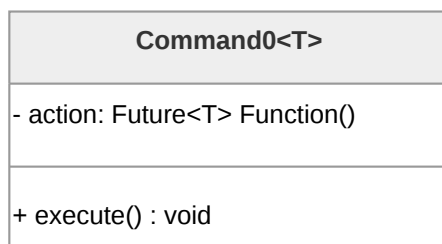


Figura 8: Diagramma della classe **Command0<T>**

3.2.6.6 Command1<T, A>

È l'implementazione concreta di **Command<T>** per azioni che richiedono un argomento di tipo A. Riceve nel costruttore una funzione di tipo **Future<T> Function(A)** e il metodo pubblico **execute()** accetta l'argomento che viene passato all'azione al momento dell'esecuzione.

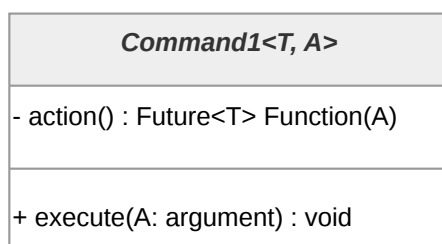


Figura 9: Diagramma della classe **Command1<T, A>**

3.2.7 Adapter

3.2.7.1 Descrizione del pattern:

Il pattern Adapter è un design pattern strutturale che permette l'interazione tra interfacce incompatibili, agendo come un mediatore che converte l'interfaccia di una classe in un'altra attesa dal client. Nell'applicazione, questo pattern viene impiegato per interfacciare il nucleo logico del sistema con servizi esterni, **Driver^G** o librerie di terze parti.

3.2.7.2 Ragioni per l'utilizzo:

L'integrazione di questo pattern è fondamentale per preservare l'integrità della **Business Logic^G**. Attraverso l'uso degli Adapter, i moduli centrali possono interagire con database o API esterne senza dipendere dai loro protocolli specifici o dai formati di dati grezzi. Questo isolamento garantisce che il cuore del sistema rimanga agnostico rispetto alle scelte tecnologiche infrastrutturali, facilitando sostituzioni o aggiornamenti di componenti esterni con un impatto minimo o nullo sul resto del codice.



3.2.7.3 Riferimenti nel progetto:

Il pattern è impiegato nel layer di accesso ai dati. Viene utilizzato per mappare le risposte provenienti dai servizi esterni in oggetti di dominio validi per l'applicazione e, specularmente, per tradurre le richieste del sistema nel formato richiesto dai destinatari esterni, garantendo una comunicazione fluida tra layer con responsabilità differenti.

3.3. Architettura di deployment

3.3.1 Architettura three-tier

Il sistema adotta un'**Architettura^G three-tier** composta da tre livelli distinti con responsabilità separate.

- **presentation tier**, corrisponde al client mobile sviluppato in Flutter, che interagisce con il **Backend^G** esclusivamente tramite richieste HTTPS verso gli endpoint esposti da API Gateway.
- **logic tier**, è costituito da AWS API Gateway e AWS Lambda. API Gateway espone le Lambda come endpoint HTTPS e funge da proxy HTTP: riceve la richiesta del client, la inoltra alla Lambda corrispondente sotto forma di oggetto evento, e restituisce al client la risposta HTTP prodotta dalla funzione. Questo livello non richiede gestione manuale del carico in quanto entrambi i servizi scalano automaticamente in risposta alla domanda, garantendo disponibilità e sicurezza senza provisioning di infrastruttura dedicata.
- **data tier**, è composto dalle istanze di Amazon DynamoDB per la persistenza dei dati strutturati e dai bucket Amazon S3 per la memorizzazione dei contenuti multimediali, quali immagini e file audio associati alle note del diario.

3.3.2 Architettura a microservizi

Per il logic tier viene adottato il pattern architetturale a **microservizi**. In questo modello l'applicazione è realizzata da componenti indipendenti, ciascuno responsabile di un sottoinsieme ben definito di funzionalità. Ogni componente è deployabile, aggiornabile e scalabile in modo autonomo rispetto agli altri.

Nel progetto ogni microservizio corrisponde a una singola funzione AWS Lambda, deployata tramite AWS SAM. Questa corrispondenza elimina la necessità dell'implementazione delle API per la comunicazione tra Lambda diverse che si introdurrebbe invocando Lambda distinte per ogni operazione, e riduce i costi di esecuzione in quanto AWS deve andare ad inizializzare una sola Lambda per microservizio. Ogni microservizio gestisce inoltre un proprio layer di persistenza dedicato, senza condivisione di istanze di database tra servizi



distinti.

Per evitare l'accoppiamento tra microservizi, ogni Lambda interagisce esclusivamente con le proprie istanze dedicate di DynamoDB o S3, senza condivisione di risorse di persistenza tra servizi distinti. Questa scelta è visibile nei diagrammi dell'**Architettura^G** di ciascun microservizio (si veda Sezione 3.5).

Il ridimensionamento è gestito in modo automatico da AWS Lambda in risposta al volume di richieste, senza intervento manuale e senza costi per capacità inutilizzata.

3.3.3 Infrastruttura

L'infrastruttura del sistema è ospitata interamente su **Amazon Web Services (AWS)**. In questo modello il provisioning, il bilanciamento del carico, la scalabilità e la gestione dell'hardware sottostante sono demandati al cloud provider, riducendo la necessità di gestione da parte del team di sviluppo.

Questa scelta risulta coerente con l'**Architettura^G** three-tier descritta in precedenza e con l'adozione del pattern a microservizi, in quanto consente di distribuire componenti indipendenti che scalano automaticamente in funzione del carico. L'infrastruttura serverless introduce inoltre i seguenti vantaggi:

- **Scalabilità dinamica^G**: le risorse computazionali vengono allocate automaticamente da AWS in base al numero di richieste ricevute riducendo i costi dovuti a capacità inutilizzata;
- **Isolamento dei servizi**: ogni microservizio viene eseguito in modo indipendente tramite una funzione AWS Lambda dedicata, con risorse di persistenza e permessi separati rispetto agli altri servizi;
- **Sicurezza**: l'accesso alle risorse AWS è regolato tramite policy IAM, limitando ogni Lambda esclusivamente alle operazioni necessarie al proprio dominio applicativo;
- **Riduzione dell'overhead operativo (Zero Ops^G)**: la gestione dell'infrastruttura fisica, degli aggiornamenti di sicurezza, della disponibilità dei servizi e della replica dei dati è demandata ad AWS.

L'infrastruttura applicativa è organizzata attraverso i seguenti componenti principali:

- **Amazon API Gateway**: costituisce il punto di ingresso del sistema lato backend ed espone gli endpoint HTTPS consumati dal client Flutter. Il servizio si occupa di ricevere le richieste HTTP, validarle e instradarle verso la funzione AWS Lambda associata;



- **Amazon Cognito:** gestisce l'autenticazione e l'autorizzazione degli utenti attraverso token JWT validati da API Gateway prima dell'invocazione delle Lambda protette;
- **AWS Lambda:** implementa il logic tier dell'applicazione secondo il pattern a microservizi. Ogni Lambda contiene la business logic relativa a uno specifico dominio funzionale ed esegue il codice in ambienti di esecuzione creati dinamicamente da AWS;
- **Amazon DynamoDB:** fornisce la persistenza dei dati strutturati tramite database NoSQL;
- **Amazon S3:** viene utilizzato per l'archiviazione dei contenuti multimediali, quali immagini e file audio.

3.3.3.1 Infrastructure as Code^G (IaC)

L'infrastruttura cloud viene gestita tramite il paradigma dell'**Infrastructure as Code^G** (IaC). Tutte le risorse AWS necessarie al funzionamento del sistema vengono descritte tramite file **YAML^G** versionati all'interno del repository del progetto.

Per la gestione dell'infrastruttura viene utilizzato **AWS SAM (Serverless Application Model)**, framework sviluppato da AWS e basato su **AWS CloudFormation**. AWS SAM introduce una sintassi semplificata specifica per applicazioni serverless, consentendo di definire in modo compatto risorse quali:

- funzioni AWS Lambda;
- endpoint API Gateway;
- bucket Amazon S3;
- tabelle Amazon DynamoDB;
- variabili d'ambiente e configurazioni di deployment.

3.4. Architettura Front End

3.4.1 Autenticazione

La funzionalità di autenticazione è stata implementata seguendo il pattern architetturale MVVM raccomandato dalle linee guida di Flutter, adattato al flusso OAuth 2.0 con OpenID Connect descritto nelle specifiche. La struttura separa nettamente le responsabilità tra i layer: il Service gestisce la comunicazione con gli endpoint e il flusso OAuth, il repository mantiene lo stato dell'**Utente^G** autenticato e orchestra la traduzione dei dati grezzi in oggetti di dominio tramite i DTO, il ViewModel espone lo stato e le azioni alla



View tramite i Command del pattern MVVM. Per la modellazione di un **Utente^G** viene utilizzato il domain model **User**.

L'istanza unica dell'**Utente^G** autenticato viene gestita tramite il Provider di Flutter a livello di app per mantenere una sola istanza attiva per tutta la durata della sessione.

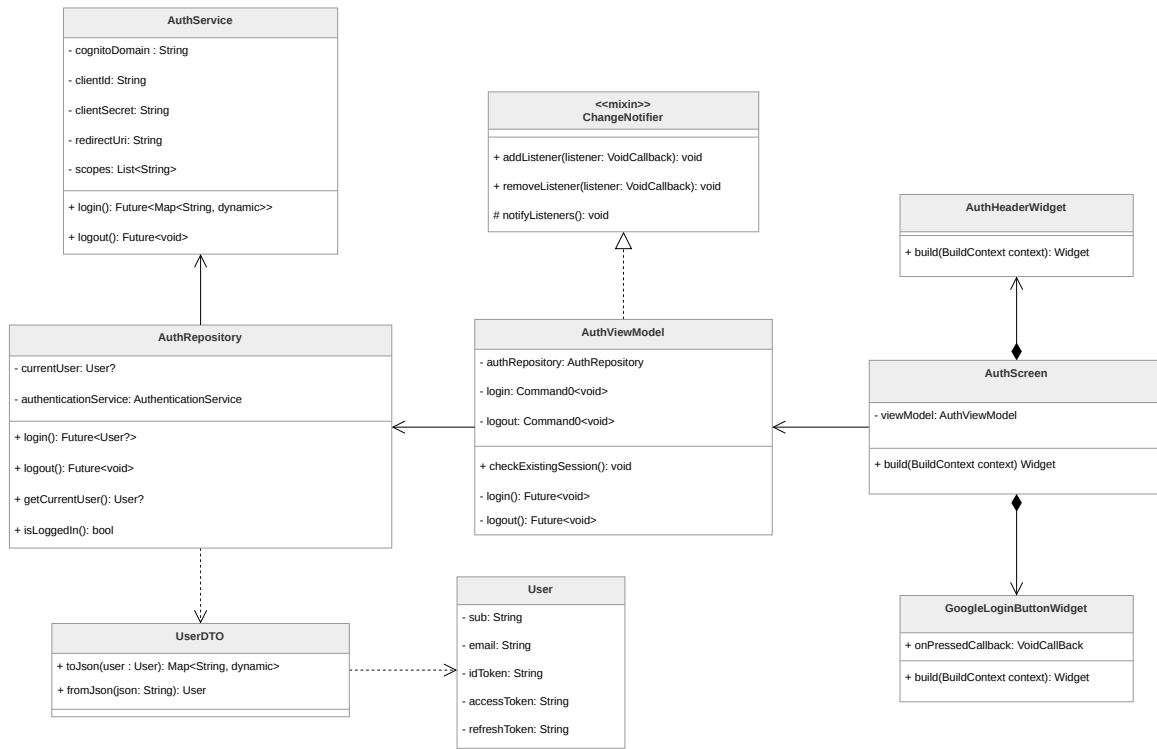


Figura 10: Diagramma delle classi relativo all'autenticazione

3.4.1.1 AuthScreen

Widget principale che rappresenta la schermata di autenticazione dell'applicazione. Nel contesto del pattern MVVM, svolge il ruolo di View, occupandosi esclusivamente della costruzione dell'interfaccia Utente tramite il metodo **build(BuildContext context)**. Mantiene un riferimento al **AuthViewModel**, dal quale osserva lo stato dell'autenticazione e a cui delega le azioni dell'**Utente^G**. La schermata è composta da widget più piccoli come **AuthHeaderWidget** e **GoogleLoginButtonWidget**, favorendo modularità e riusabilità.

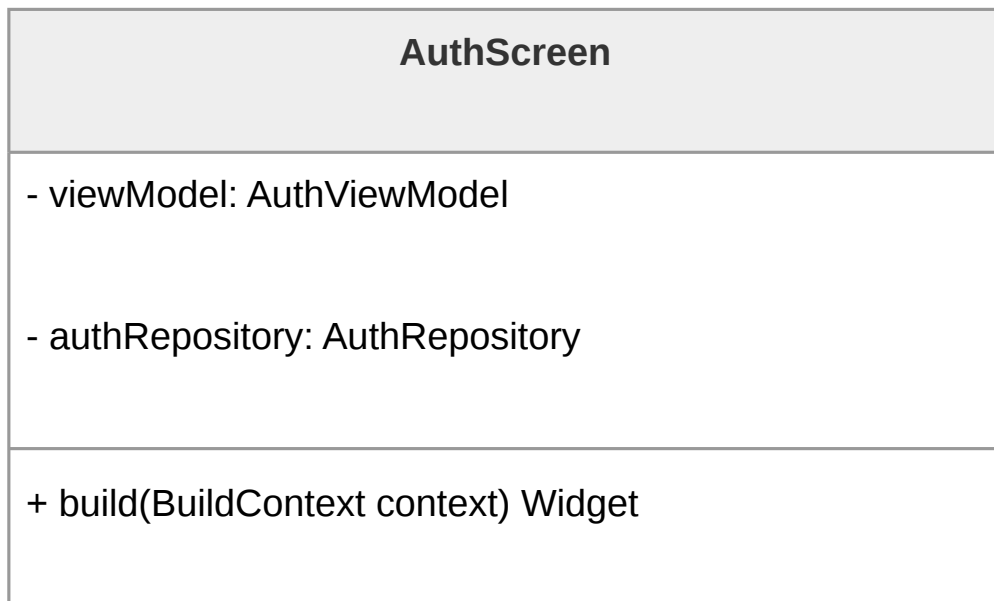


Figura 11: Diagramma della classe **AuthScreen**

3.4.1.2 AuthHeaderWidget

La classe AuthHeaderWidget rientra nel livello View del pattern MVVM e ha il solo compito di costruire la porzione superiore dell'interfaccia, contenente titolo e altri elementi grafici identificativi. Non contiene alcuna logica applicativa e contribuisce alla separazione delle responsabilità tra UI e gestione dello stato.

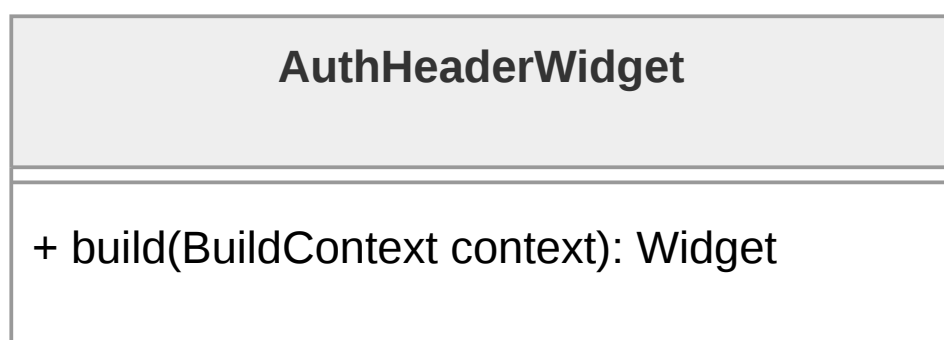


Figura 12: Diagramma della classe **AuthHeaderWidget**

3.4.1.3 GoogleLoginButtonWidget

Rappresenta il pulsante che consente all'**Utente^G** di avviare il Processo di autenticazione. È incluso nella **AuthScreen** tramite composizione e riceve una callback **onPressedCallback** che viene eseguita alla pressione del pulsante.

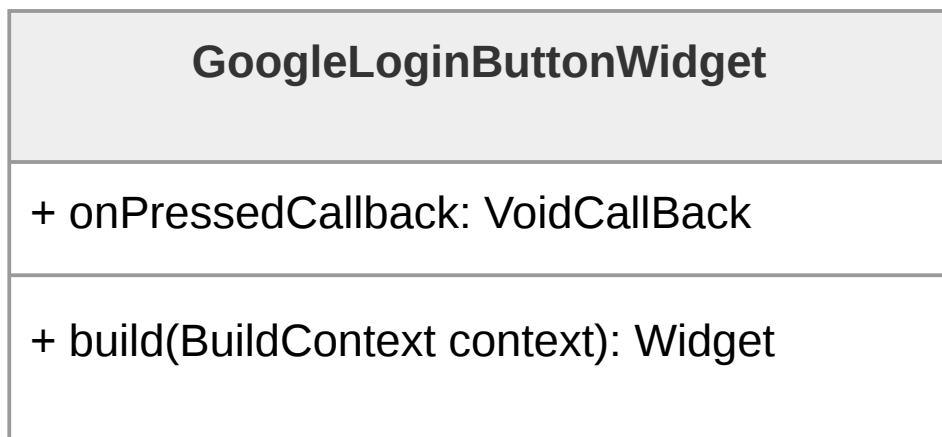


Figura 13: Diagramma della classe `GoogleLoginButtonWidget`

3.4.1.4 User

Rappresenta il modello di dominio dell'**Utente^G** autenticato all'interno dell'applicazione. Contiene le informazioni essenziali legate all'identità e alla sessione, come l'identificativo univoco e i token di accesso, ed è utilizzata dal resto del sistema per gestire lo stato dell'autenticazione in modo indipendente dal formato dei dati provenienti dall'esterno.

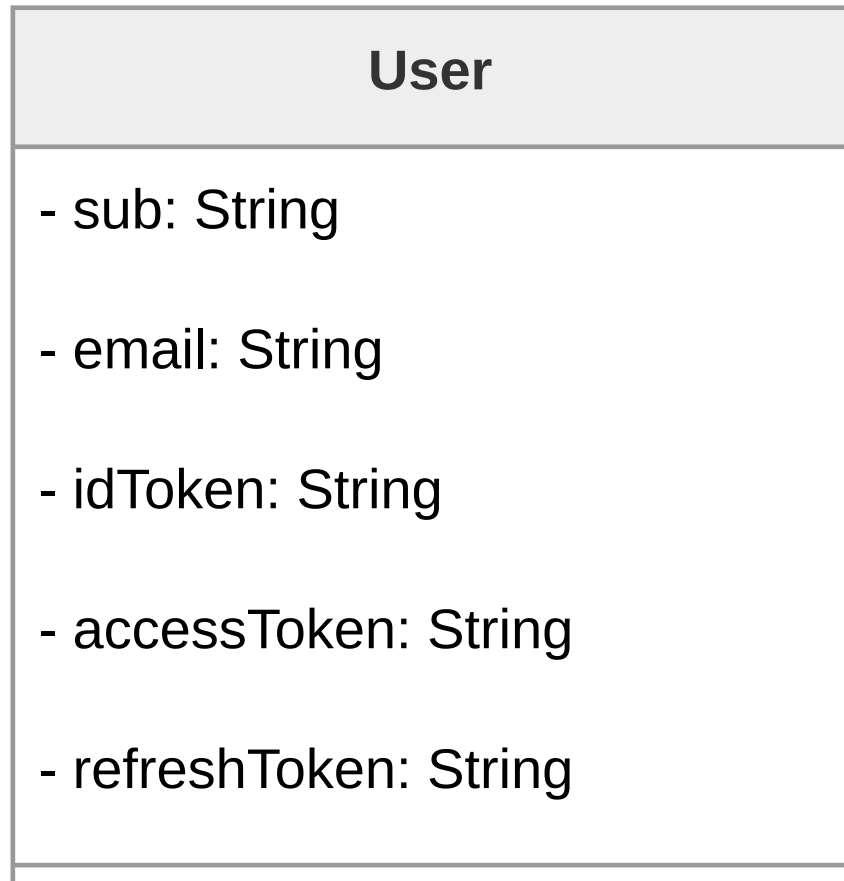
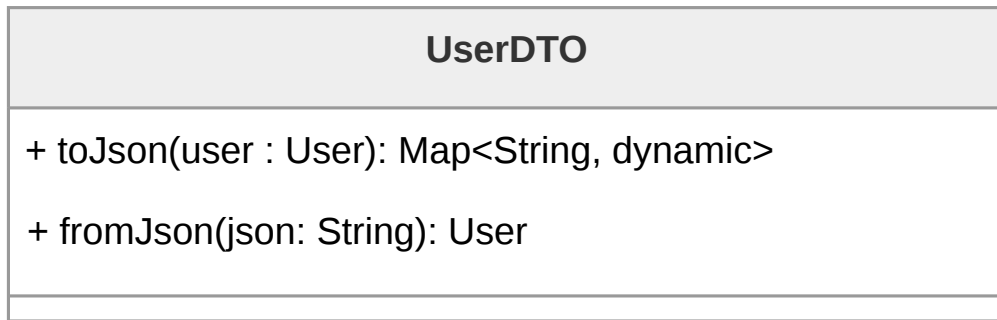


Figura 14: Diagramma della classe **User**

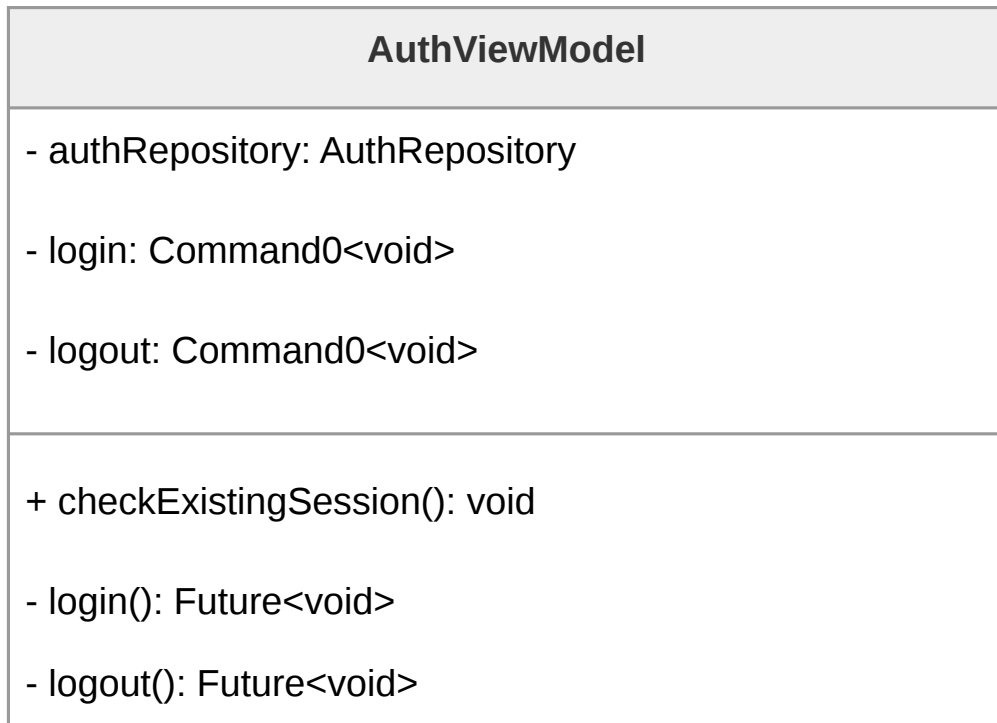
3.4.1.5 UserDTO

Ha il compito di gestire la conversione tra il modello di dominio **User** e la rappresentazione serializzata dei dati. Attraverso i metodi **toJson** e **fromJson**, consente rispettivamente di trasformare un oggetto **User** in formato JSON e di ricostruirlo a partire da dati serializzati.

Figura 15: Diagramma della classe **UserDTO**

3.4.1.6 AuthViewModel

Gestisce lo stato dell'autenticazione e coordina le operazioni richieste dalla View interagendo con il **AuthRepository**. Le funzionalità principali, come il login e il logout, sono esposte tramite il Command pattern. La logica concreta è incapsulata nei metodi **login()** e **logout()**, che vengono tipicamente passati come azioni ai rispettivi Command, mentre il metodo **checkExistingSession()** consente di verificare la presenza di una sessione già attiva al momento dell'avvio.

Figura 16: Diagramma della classe **AuthViewModel**



3.4.1.7 AuthRepository

Funge da intermediario tra il **AuthViewModel** e il livello di accesso ai dati, incapsulando la logica necessaria per gestire l'autenticazione. Mantiene lo stato dell'**Utente^G** corrente tramite **currentUser** e delega le operazioni di comunicazione remota a **AuthenticationService**. Il repository espone metodi per effettuare il login, il logout, recuperare l'**Utente^G** corrente e verificare se esiste una sessione attiva.

AuthRepository
- currentUser: User?
- authenticationService: AuthenticationService
+ login(): Future<User?>
+ logout(): Future<void>
+ getCurrentUser(): User?
+ isLoggedIn(): bool

Figura 17: Diagramma della classe **AuthRepository**

3.4.1.8 AuthService

È responsabile della comunicazione con il sistema di autenticazione esterno, ad esempio un provider basato su OAuth o AWS Cognito. Contiene le informazioni di configurazione necessarie, tra cui dominio, credenziali del client, URI di redirect e scope richiesti. Si occupa esclusivamente di eseguire le chiamate di rete per il login e il logout

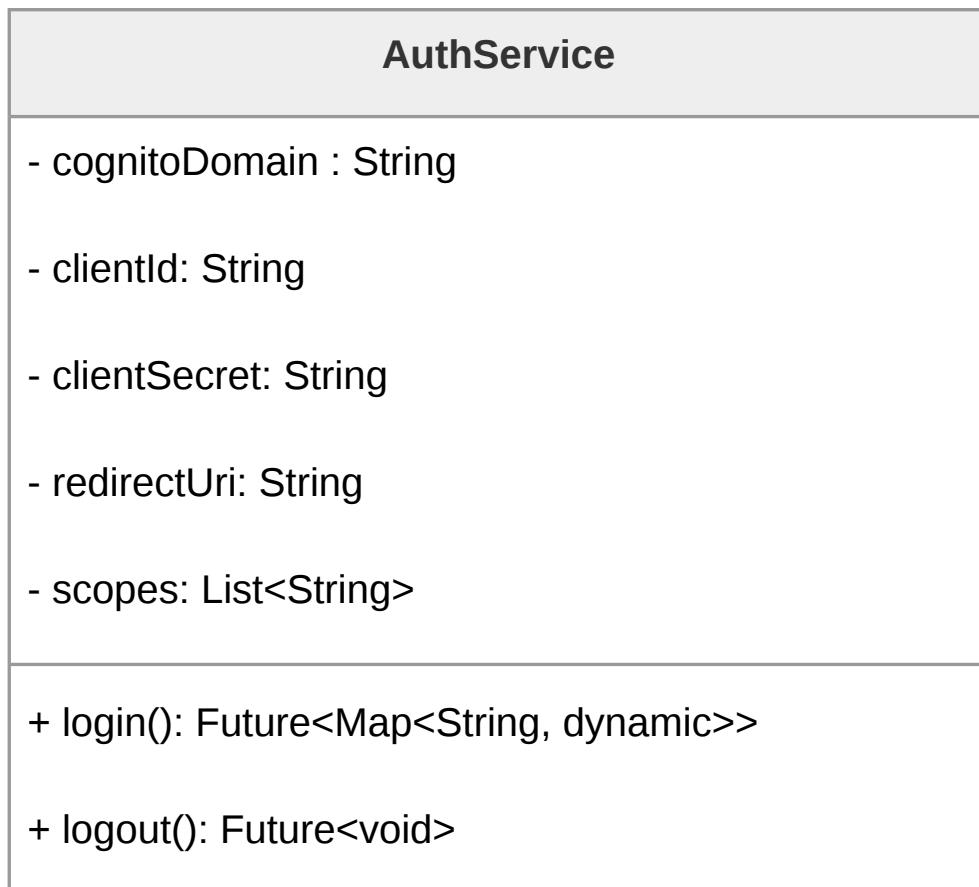


Figura 18: Diagramma della classe `AuthenticationService`

3.4.2 Invio SOS

La funzionalità di invio dell>alert SOS è strutturata seguendo il pattern MVVM raccomandato dalla guida ufficiale di Flutter, ed utilizza anch'essa il pattern Command. Il diagramma descrive l'insieme delle classi che collaborano per permettere all'**Utente^G** di inviare un messaggio di soccorso ai propri contatti fidati tramite la pressione di un pulsante nel `SosBottomSheetWidget`, visibile in tutte le schermate dell'applicazione. La classe `SosAlertViewModel` coordina le precondizioni necessarie all'invio, verificando la presenza di contatti fidati tramite le informazioni contenute nel `TrustedContactRepository`, prima di inoltrare l'operazione al `SosAlertRepository`.

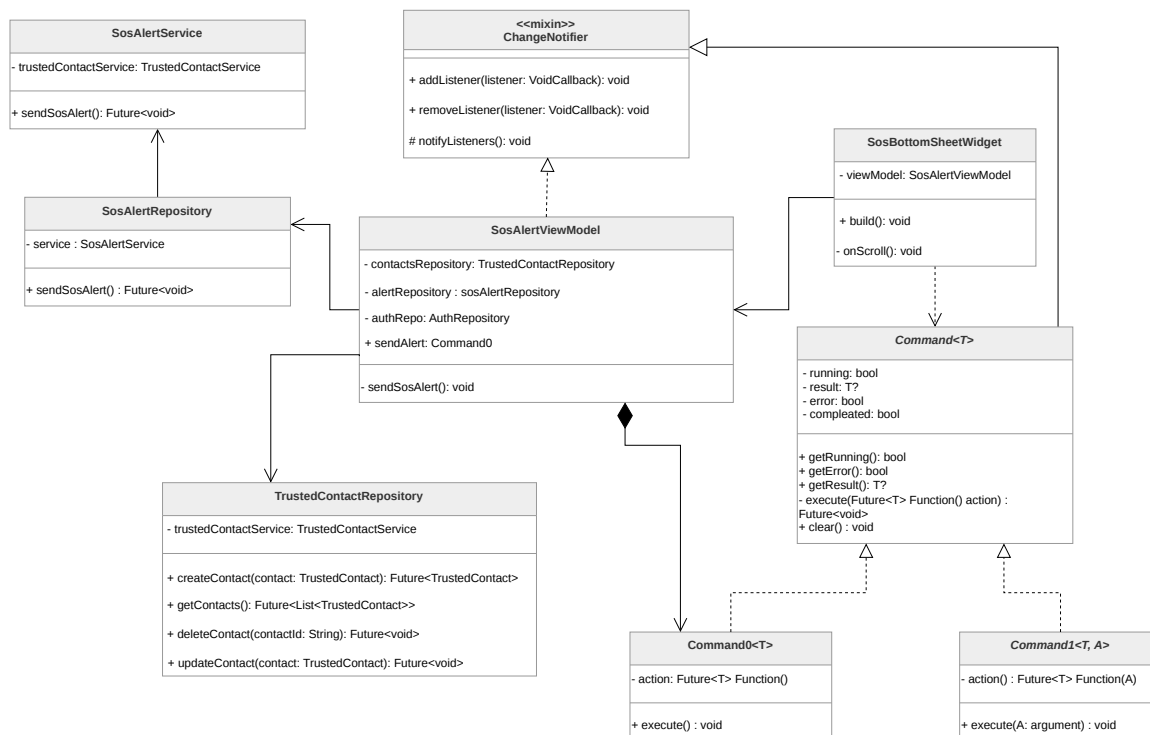


Figura 19: Diagramma delle classi relativo alla funzionalità dell'invio email di soccorso

3.4.2.1 SosBottomSheetWidget

Widget che funge da punto di ingresso visivo per la funzionalità. Viene mostrato e nascosto in risposta allo scroll dell'**Utente^G** tramite `_onScroll()` ed è presente in tutte le schermate dell'applicazione. Mantiene un riferimento al **SosAlertViewModel** da cui riceve lo stato corrente dell'operazione e a cui delega l'esecuzione dell'invio quando l'**Utente^G** preme il pulsante.

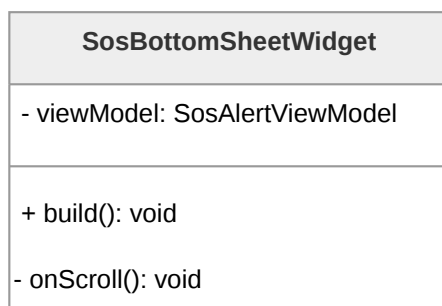


Figura 20: Diagramma della classe **SosBottomSheetWidget**

3.4.2.2 SosAlertViewModel

Layer di presentazione della funzionalità. Coordina le dipendenze necessarie all'invio dei segnali d'allarme:



- **TrustedContactRepository** per la verifica della presenza di contatti;
- **SosAlertRepository** per l'invio effettivo;
- **AuthRepository** per recuperare l'identità dell'**Utente^G** autenticato.

Espone **sendAlert** come **Command0** che la View può eseguire direttamente. La logica di orchestrazione delle precondizioni è contenuta nel metodo privato **_sendSosAlert()**, che viene passato come action al Command nel costruttore.

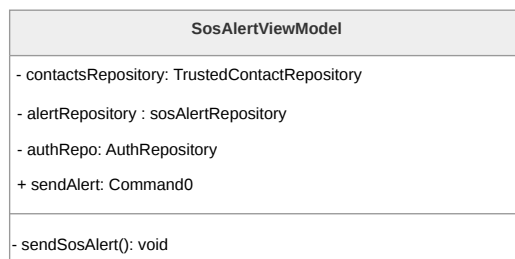


Figura 21: Diagramma della classe **SosAlertViewModel**

3.4.2.3 SosAlertRepository

Ha la singola responsabilità di orchestrare l'invio dell'alert SOS. Delega la chiamata HTTP al **Backend^G** tramite **SosAlertService** e rappresenta il punto di separazione tra il layer di presentazione e il layer di accesso ai dati per questa funzionalità specifica.

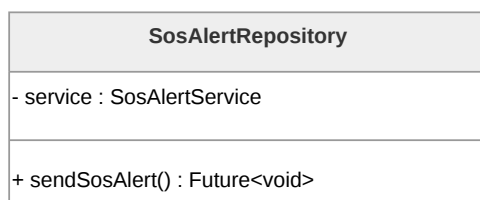


Figura 22: Diagramma della classe **SosAlertRepository**

3.4.2.4 SosAlertService

Gestisce la comunicazione con l'endpoint dedicato all'invio dell'alert. Riceve da **SosAlertRepository** la richiesta di invio e si occupa esclusivamente della chiamata di rete, restituendo i dati grezzi della risposta.

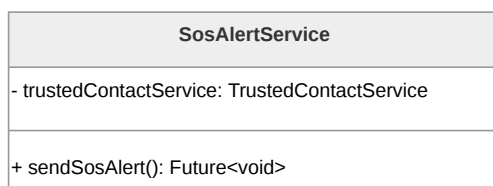


Figura 23: Diagramma della classe **SosAlertService**

3.4.3 Chatbot

MessageResponse.

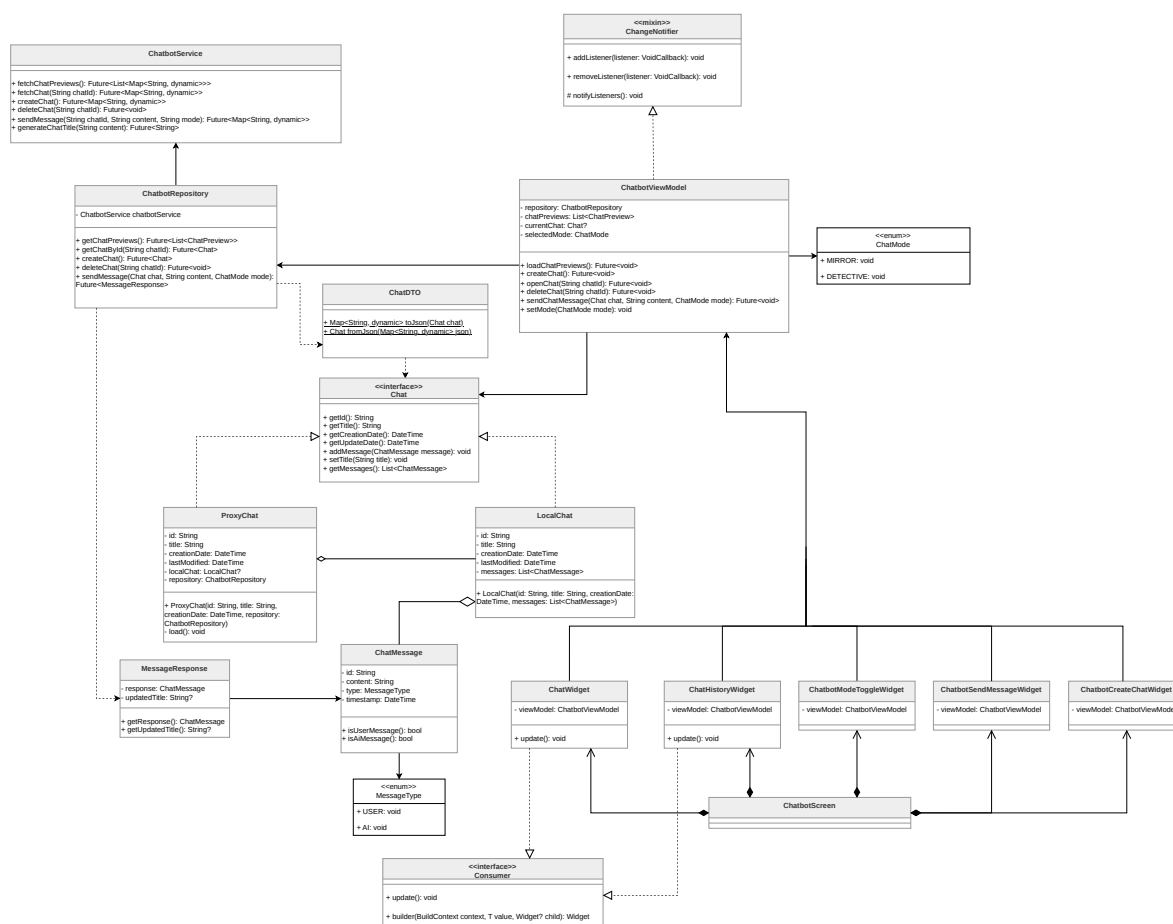


Figura 24: Diagramma delle classi relativo alle funzionalità del Chatbot

3.4.3.1 ChatbotScreen

La classe `ChatbotScreen` rappresenta la schermata dell'applicazione dedicata al Chatbot all'interno del *Presentation Layer*. La classe è composta a sua volta da numerosi widget e



contiene la minima logica necessaria al rendering di tali componenti visivi, permettendo quindi una netta separazione tra presentazione e logica secondo il pattern *MVVM*. Tra i numerosi widget che la costituiscono, i principali sono **ChatWidget**, **ChatHistoryWidget**, **ChatbotModeToggleWidget** e **ChatbotSendMessageWidget**.

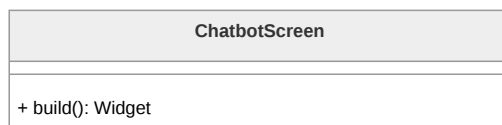


Figura 25: Diagramma della classe **ChatbotScreen**

3.4.3.2 ChatWidget

La classe **ChatWidget** è responsabile della visualizzazione della conversazione attiva. In quanto **Consumer**, essa osserva il **ChatbotViewModel** per ottenere i dati aggiornati e reagisce ai cambiamenti dello stato applicativo, garantendo la costruzione automatica dei widget rappresentanti i messaggi scambiati in chat.

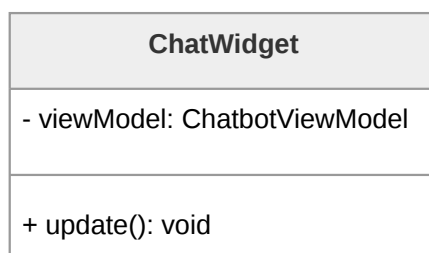


Figura 26: Diagramma della classe **ChatWidget**

3.4.3.3 ChatHistoryWidget

ChatHistoryWidget si occupa di mostrare l'elenco delle conversazioni passate sotto forma di anteprime. Tale widget è inserito all'interno di un menu a tendina dedicato, e, in quanto **Consumer**, osserva il **ChatbotViewModel** per recuperare i dati aggiornati relativi alle chat passate. Questo widget trae grande beneficio dall'utilizzo del pattern Proxy per il fetch delle chat. Tale pattern consente infatti di generare la lista di anteprime delle conversazioni passate senza dover recuperare l'intero contenuto di ogni singola chat.

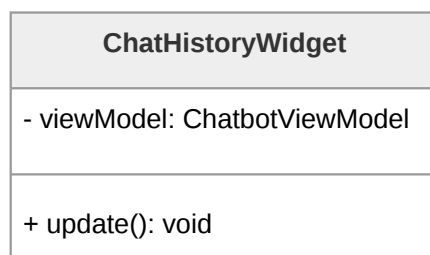


Figura 27: Diagramma della classe **ChatHistoryWidget**

3.4.3.4 ChatbotModeToggleWidget

Questo componente dell'interfaccia consente all'**Utente^G** di selezionare la modalità operativa del chatbot. Le due modalità disponibili sono il Detective delle Relazioni e lo Specchio Intelligente. La modalità in uso viene specificata nell'invocazione dei metodi relativi all'invio di un messaggio in chat e determinano come il chatbot risponde ai messaggi dell'**Utente^G**. La generazione della risposta è a completo carico del **Backend^G**.

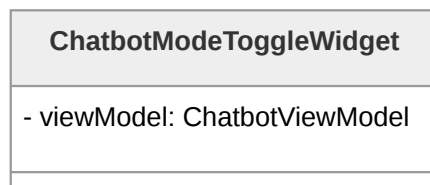


Figura 28: Diagramma della classe **ChatbotModeToggleWidget**

3.4.3.5 ChatbotSendMessageWidget

ChatbotSendMessageWidget è un pulsante che gestisce l'invio dei messaggi da parte dell'**Utente^G**, delegando la gestione dei nuovi messaggi al ViewModel. Quando **ChatbotSendMessageWidget** viene premuto, il messaggio contenuto nel campo d'inserimento testuale viene inoltrato a **ChatbotViewModel**, che a sua volta comunica con **ChatbotRepository** in modo da memorizzare il messaggio nella chat corrente e di ricevere la risposta dal LLM.

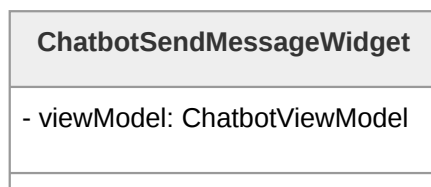


Figura 29: Diagramma della classe **ChatbotSendMessageWidget**

3.4.3.6 ChatbotCreateChatWidget

Questo widget fornisce all'**Utente^G** la possibilità di avviare una nuova conversazione. Anche in questo caso, la logica viene delegata al ViewModel, preservando il disaccoppiamento tra interfaccia e dominio applicativo.

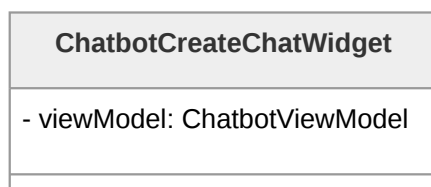


Figura 30: Diagramma della classe **ChatbotCreateChatWidget**

3.4.3.7 ChatbotViewModel

ChatbotViewModel gestisce lo stato dell'interfaccia chatbot: mantiene la lista delle conversazioni passate, la chat correntemente aperta e la modalità operativa selezionata. In quanto **ChangeNotifier**, notifica automaticamente le classi View (**ChatWidget**, **ChatHistoryWidget** e i widget di azione) ogni volta che lo stato cambia. Per accedere ai dati si affida a **ChatbotRepository**, al quale delega il recupero e la persistenza delle conversazioni.

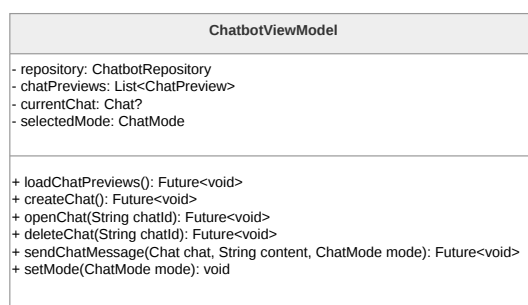


Figura 31: Diagramma della classe **ChatbotViewModel**



3.4.3.8 Chat

L'interfaccia **Chat** definisce il modello astratto di una conversazione tra **Utente^G** e il chatbot. Insieme a **ProxyChat** e **LocalChat**, è utilizzata nell'implementazione del pattern Proxy.

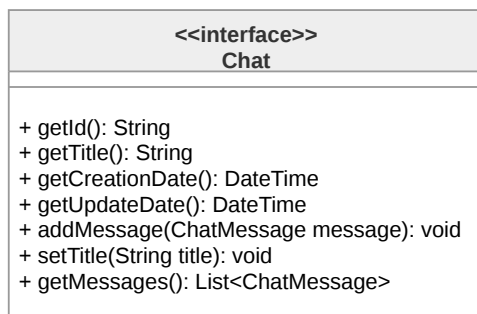


Figura 32: Diagramma della classe **Chat**

3.4.3.9 ProxyChat

ProxyChat implementa l'interfaccia **Chat** e viene utilizzata da **ChatbotViewModel** per rappresentare le conversazioni storiche. Ritarda il caricamento completo dei messaggi, delegandolo a **ChatbotRepository** solo quando necessario, e mantiene internamente un riferimento a **LocalChat**, che istanzia e popola al primo accesso effettivo ai dati.

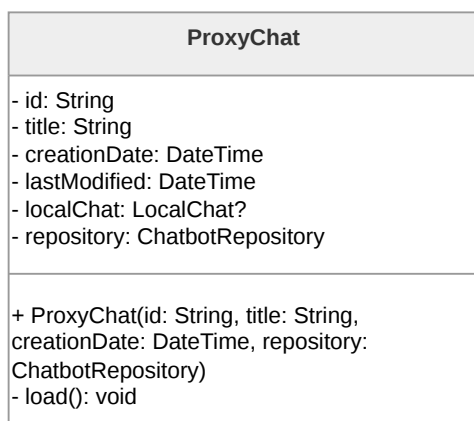


Figura 33: Diagramma della classe **ProxyChat**

3.4.3.10 LocalChat



LocalChat è l'implementazione del modello di conversazione che mantiene in memoria tutte le informazioni e i messaggi associati alla chat. Essa rappresenta l'oggetto reale utilizzato da **ProxyChat** quando c'è effettivamente bisogno di caricare i messaggi della chat, ad esempio per visualizzarli a schermo.

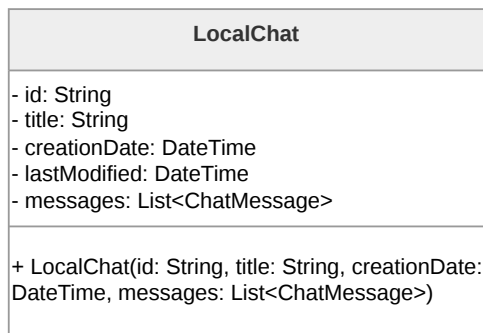


Figura 34: Diagramma della classe **LocalChat**

3.4.3.11 ChatMessage

ChatMessage modella un singolo messaggio all'interno di una conversazione, includendo contenuto, tipologia e data di creazione e di aggiornamento. Fornisce inoltre un modo per distinguere tra messaggi generati dall'**Utente^G** e quelli prodotti dal LLM.

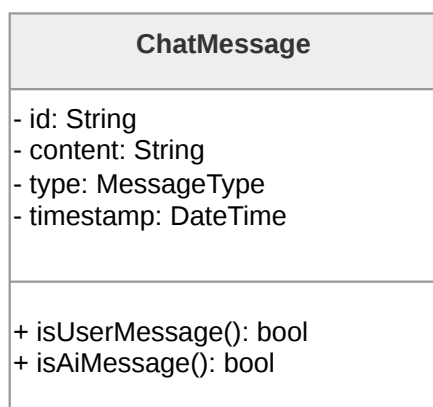


Figura 35: Diagramma della classe **ChatMessage**

3.4.3.12 ChatMode

ChatMode è un'enumerazione che rappresenta le due diverse modalità operative del chatbot. Viene utilizzato da **ChatbotViewModel**, che lo inoltra a **ChatbotRepository**, il quale



lo utilizza per invocare i metodi del **ChatbotService** relativi all'invio dei messaggi in chat. Sarà poi il **Backend^G** a interpretarlo e utilizzarlo per generare la risposta al messaggio inviato dall'**Utente^G**.

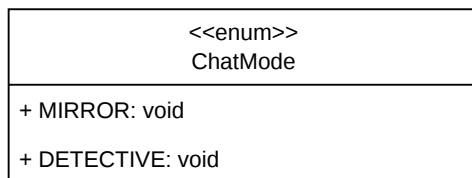


Figura 36: Diagramma della classe **ChatMode**

3.4.3.13 MessageType

MessageType è un'enumerazione che consente di classificare i messaggi in base alla loro origine, distinguendo tra input dell'**Utente^G** e output del LLM. È utilizzata nella classe **ChatMessage** e necessaria per stabilire l'impaginazione dei messaggi all'interno di **ChatWidget**.

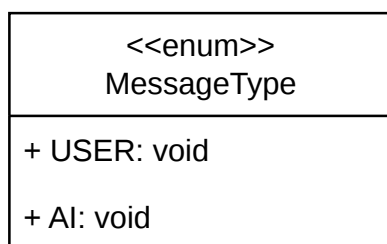


Figura 37: Diagramma della classe **MessageType**

3.4.3.14 MessageResponse

MessageResponse incapsula il risultato di un'interazione con il LLM, includendo il messaggio generato e, eventualmente, il titolo generato a partire dal messaggio originale. È utilizzato da **ChatbotRepository**, che a sua volta lo ritorna in risposta a **ChatbotViewModel** per l'aggiornamento dello stato della View.

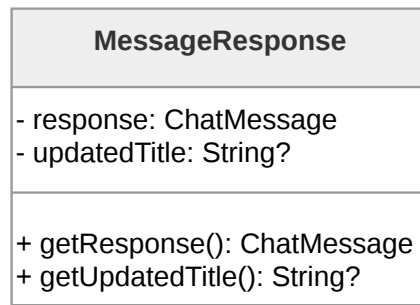


Figura 38: Diagramma della classe **MessageResponse**

3.4.3.15 ChatbotRepository

ChatbotRepository è il tramite tra **ChatbotViewModel** e la comunicazione di rete: riceve le richieste dal ViewModel, le inoltra a **ChatbotService** e trasforma le risposte grezze in oggetti di dominio (**LocalChat**, **ChatMessage**) tramite **ChatDTO**, restituendoli al ViewModel in un formato pronto all'uso.

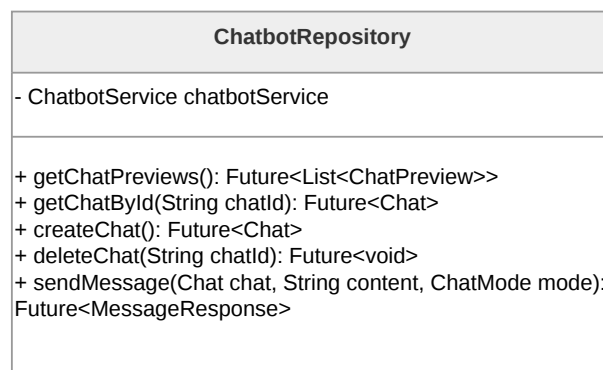
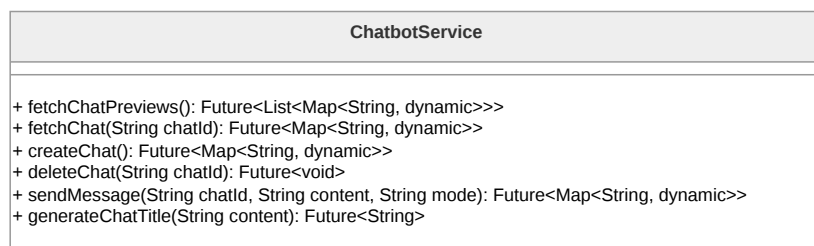


Figura 39: Diagramma della classe **ChatbotRepository**

3.4.3.16 ChatbotService

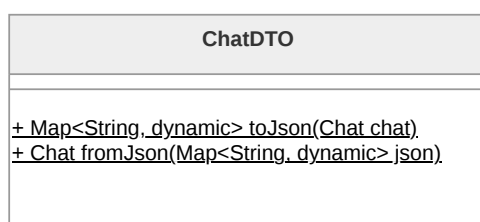
ChatbotService rappresenta il livello infrastrutturale responsabile della comunicazione con le API remote. Si occupa della gestione delle richieste e delle risposte, operando su rappresentazioni dei dati adatte allo scambio su rete. Restituisce i dati grezzi a **ChatbotRepository**, il quale si occupa poi di trasformarli in modelli di dominio.

Figura 40: Diagramma della classe **ChatbotService**

3.4.3.17 ChatDTO

ChatDTO è un oggetto di trasferimento dati utilizzato per convertire le informazioni tra il formato utilizzato dal sistema esterno e quello del dominio applicativo. Ha due ruoli principali:

- trasformare i risultati delle chiamate ad API da JSON a oggetti di tipo **Chat**
- serializzare oggetti di tipo **Chat**, generando la loro rappresentazione JSON per l'invio alle API.

Figura 41: Diagramma della classe **ChatDTO**

3.4.4 Diario criptato e diario fittizio

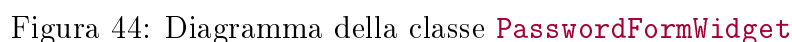
La funzionalità di diario criptato e diario fittizio permette all'**Utente^G** di utilizzare due archivi separati in cui memorizzare note, contenenti testo, immagini e/o contenuti audio, protetti da password. L'implementazione della funzionalità segue il pattern MV-VM e le linee guida del **Framework^G** Flutter, risultanti nella suddivisione tra un **Data Layer^G**, composto da classi Service e repository, responsabili per la **Business Logic^G** e per la **Persistence Logic^G** rispettivamente, e un **UI Layer^G**, composto da classi View e ViewModel. Gli oggetti principali della funzionalità sono le classi Note (**ProxyNote** e **LocalNote**), **NoteElement** e **DiarySession**.



La classe `DiaryAccessScreen` si occupa della rappresentazione visiva della schermata per l'accesso alla funzionalità dei diari. Contiene solamente la logica necessaria per il rendering dei widget che la compongono, tra cui `PasswordFormWidget`, per garantire la separazione tra presentazione e logica, come previsto dal pattern MVVM. Per questo delega la gestione dello stato a `DiaryAccessViewModel`.



La classe `PasswordFormWidget` è responsabile della visualizzazione dei campi utilizzati per l'accesso ai diari. `PasswordFormWidget` è un `Consumer` che osserva `DiaryAccessViewModel` per ottenere i dati aggiornati e reagire di conseguenza.





3.4.4.3 DiaryAccessViewModel

La classe **DiaryAccessViewModel** gestisce lo stato dell'applicazione e coordina le operazioni tra i Layer, implementando il pattern Observer. La classe è un **ChangeNotifier** e mantiene una lista di *listeners* che vengono notificati quando avvengono cambiamenti che necessitano di un loro aggiornamento, in questo caso componenti che devono reagire a tentativi di accesso ai diari da parte dell'**Utente^G**. Per accedere ai dati e verificare le credenziali si affida alla classe **DiaryAccountRepository**.

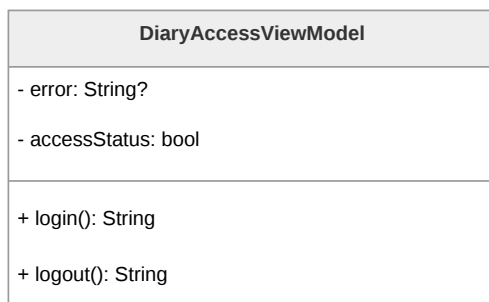


Figura 45: Diagramma della classe **DiaryAccessViewModel**

3.4.4.4 DiaryAccountRepository

DiaryAccountRepository si occupa della trasformazione delle risposte ricevute da **DiaryAccountService** in oggetti di dominio e nascondendone la natura al resto dell'applicazione, fungendo da strato di astrazione tra il *service* ed il ViewModel.

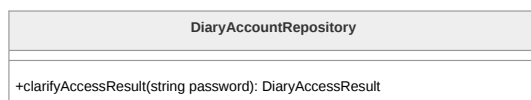


Figura 46: Diagramma della classe **DiaryAccountRepository**

3.4.4.5 DiaryAccountService

La classe **DiaryAccountService** rappresenta lo strato infrastrutturale responsabile della comunicazione con le API remote, occupandosi della gestione delle richieste e delle risposte. Restituisce i dati grezzi a **DiaryAccountRepository**, il quale si occupa poi di trasformarli in oggetti di dominio.

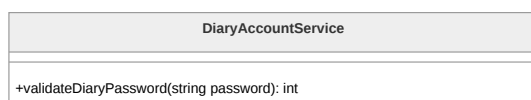


Figura 47: Diagramma della classe **DiaryAccountService**

3.4.4.6 DiaryAccessResult

L'enumerazione **DiaryAccessResult** permette la classificazione della risposta ad un ten-



tativo di accesso ai diari da parte dell'**Utente^G**, distinguendo tra accesso avvenuto con successo ad uno dei due tipi di diario, e accesso fallito con errore o superamento del limite di tentativi di accesso.

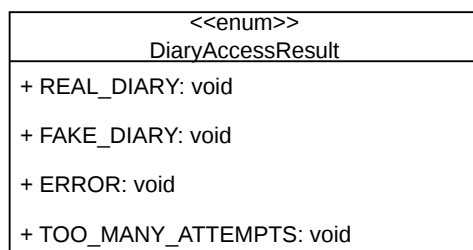


Figura 48: Diagramma della classe **DiaryAccessResult**

3.4.4.7 DiaryScreen

La classe **DiaryScreen** gestisce la rappresentazione visiva della schermata principale dei diari nella *presentation layer*, contenendo solamente la logica necessaria al rendering dei suoi componenti widget, tra cui **NoteListWidget** e **NoteActionsWidget**. La gestione dello stato è delegata alla classe **DiaryViewModel**.

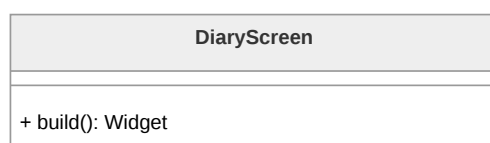


Figura 49: Diagramma della classe **DiaryScreen**

3.4.4.8 DiaryPasswordSettingWidget

La classe **DiaryPasswordSettingWidget** gestisce la presentazione grafica dell'interfaccia per l'impostazione della password per l'accesso al diario fittizio.

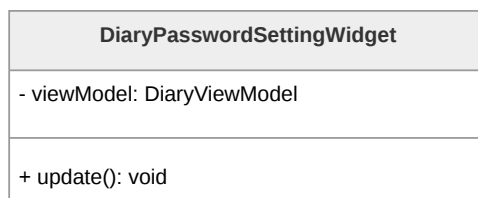


Figura 50: Diagramma della classe **DiaryPasswordSettingWidget**

3.4.4.9 NoteListWidget

La classe **NoteListWidget** si occupa della visualizzazione della lista delle note contenute nel diario in cui l'**Utente^G** ha effettuato l'accesso e l'eventuale visualizzazione dei



contenuti di una nota selezionata. Trattandosi di un **Consumer**, la classe osserva gli aggiornamenti dei dati in **DiaryViewModel** in modo da reagire di conseguenza alla selezione, modifica o eliminazione delle note.

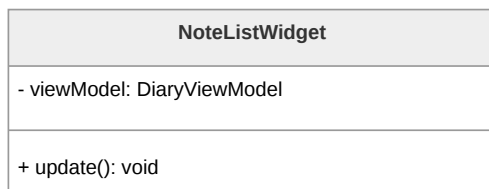


Figura 51: Diagramma della classe **NoteListWidget**

3.4.4.10 NoteActionsWidget

La classe **NoteActionsWidget** si occupa della visualizzazione delle componenti di interazione della funzionalità di diario. La logica applicativa viene delegata a **DiaryViewModel**, preservando il disaccoppiamento tra interfaccia e dominio applicativo.

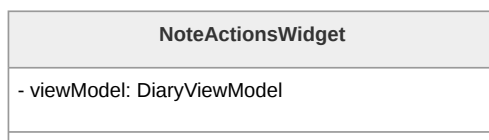


Figura 52: Diagramma della classe **NoteActionsWidget**

3.4.4.11 NoteEditorWidget

La classe **NoteEditorWidget** gestisce la visualizzazione di una nota del diario selezionata dall'**Utente^G**, offrendo la possibilità di visualizzarne gli elementi che la compongono e permettendone l'aggiunta e la modifica.

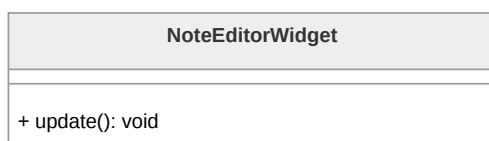


Figura 53: Diagramma della classe **NoteEditorWidget**

3.4.4.12 NoteAudioPlayerWidget

La classe **NoteAudioPlayerWidget** è responsabile della visualizzazione di un audio player che consente all'**Utente^G** di visualizzare e riprodurre le tracce audio inserite nella nota.

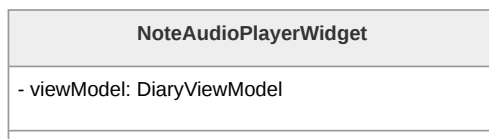


Figura 54: Diagramma della classe **NoteAudioPlayerWidget**

3.4.4.13 DiaryViewModel

DiaryViewModel si occupa della gestione dello stato dell'applicazione e del coordinamento delle operazioni tra UI e **Data Layer^G**. La classe tiene conto della lista di note presenti all'interno del diario attualmente aperto e della nota attualmente aperta dall'**Utente^G**. Inoltre delega alla classe **NoteRepository** l'accesso e la gestione dei dati delle note per le operazioni di memoria. La classe implementa il Mixin **ChangeNotifier**, e quindi il pattern Observer, mantenendo una lista di *listeners* che vengono notificati a seguito di operazioni a loro rilevanti, in questo caso **NoteListWidget** e **NoteActionsWidget**.



Figura 55: Diagramma della classe **DiaryViewModel**

3.4.4.14 DiarySession

La classe **DiarySession** modella l'istanza effettiva del diario, tenendo traccia del tipo di diario a cui l'**Utente^G** ha effettuato l'accesso e gestendo le informazioni di sessione. La



classe implementa il pattern Singleton per fare in modo che in ogni dato momento di utilizzo dell'applicazione possa esistere al più una singola istanza di diario.

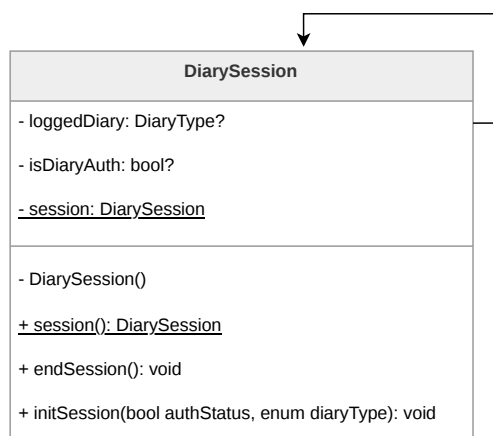


Figura 56: Diagramma della classe **DiarySession**

3.4.4.15 DiaryType

L'enumerazione **DiaryType** rappresenta le due tipologie di diario disponibili e viene utilizzata da **DiarySession** per comunicare a **DiaryViewModel** quale archivio di note visualizzare.

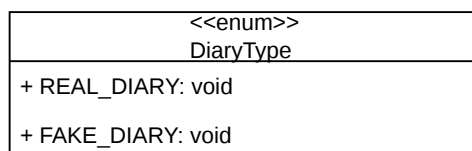


Figura 57: Diagramma della classe **DiaryType**

3.4.4.16 Note

L'interfaccia **Note** definisce i metodi necessari per gli oggetti rappresentanti le note all'interno dei diari. Implementa il pattern Proxy insieme alle classi **ProxyNote** e **LocalNote**.

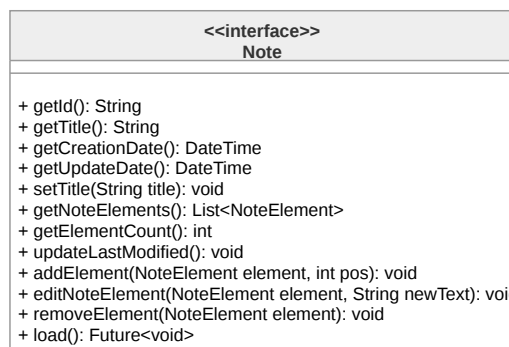


Figura 58: Diagramma della classe **Note**

3.4.4.17 ProxyNote

La classe **ProxyNote** realizza il pattern Proxy per rendere più efficiente il caricamento da repository delle note, permettendo di rimandare il caricamento dei dati interni di una nota fino a quando non è effettivamente necessario, e così risparmiando risorse di sistema. La classe contiene un riferimento ad un oggetto **LocalNote** a cui delega le operazioni una volta effettuato il caricamento della nota.

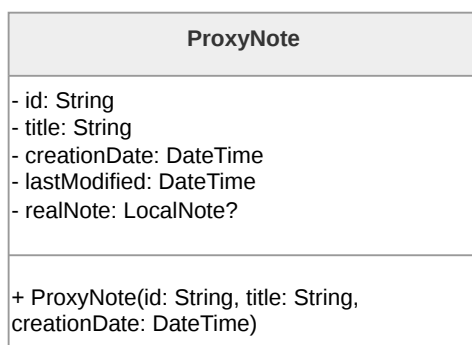


Figura 59: Diagramma della classe **ProxyNote**

3.4.4.18 LocalNote

LocalNote è l'implementazione dell'interfaccia **Note** che rappresenta l'oggetto reale, utilizzato dall'implementazione proxy per le operazioni sulla nota una volta effettuato il suo caricamento.

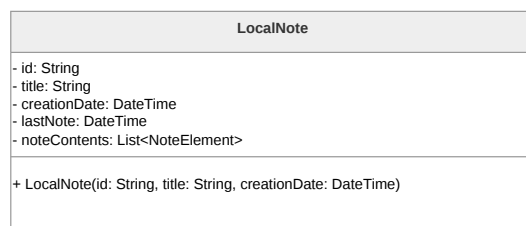


Figura 60: Diagramma della classe **LocalNote**

3.4.4.19 NoteElement

La classe astratta **NoteElement** rappresenta un elemento presente all'interno di una nota e ha delle specializzazioni in base al tipo di contenuto dell'oggetto, che può essere testuale, un'immagine oppure del contenuto audio.

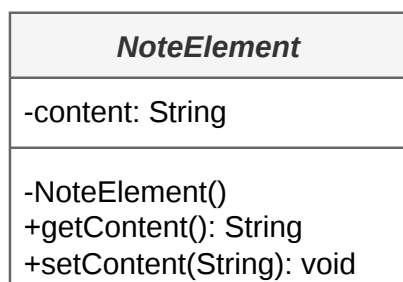


Figura 61: Diagramma della classe **NoteElement**

3.4.4.20 NoteTextElement

La classe concreta **NoteTextElement** è la specializzazione di **NoteElement** per contenuti testuali.

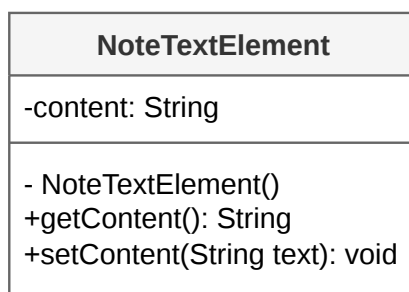


Figura 62: Diagramma della classe **NoteTextElement**



3.4.4.21 NoteImgElement

La classe concreta **NoteImgElement** è la specializzazione di **NoteElement** per contenuti di tipo grafico.

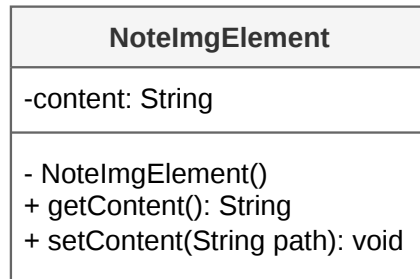


Figura 63: Diagramma della classe **NoteImgElement**

3.4.4.22 NoteAudioElement

La classe concreta **NoteAudioElement** è la specializzazione di **NoteElement** per contenuti audio.

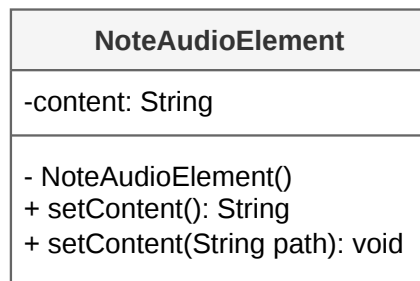
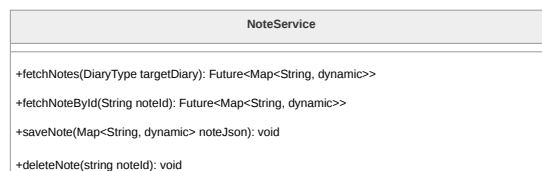


Figura 64: Diagramma della classe **NoteAudioElement**

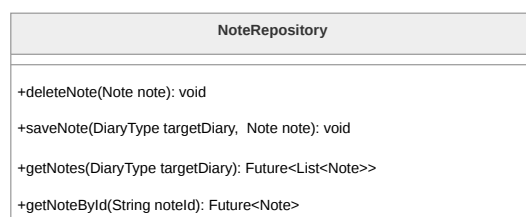
3.4.4.23 NoteService

La classe **NoteService** rappresenta il livello infrastrutturale dedicato alla comunicazione con le API remote, gestendo richieste e risposte durante le operazioni sulle note. Restituisce i dati grezzi a **NoteRepository**, il quale si occupa poi di trasformarli in modelli di dominio tramite **NoteDTO**.

Figura 65: Diagramma della classe **NoteService**

3.4.4.24 NoteRepository

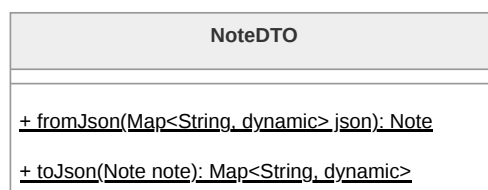
La classe **NoteRepository** funge da strato di astrazione tra **DiaryViewModel** e la **Persistence Logic^G** di **NoteService**, trasformando i dati grezzi in oggetti di dominio tramite **NoteDTO** e nascondendone la natura al resto dell'applicazione.

Figura 66: Diagramma della classe **NoteRepository**

3.4.4.25 NoteDTO

NoteDTO è un Data Transfer Object con due ruoli principali:

- deserializzare i risultati delle chiamate API, in formato JSON, trasformandoli in oggetti di tipo **Note**.
- serializzare oggetti di tipo **Note**, generando la loro rappresentazione JSON per operazioni che richiedono l'invio dei dati alle API, come il salvataggio di una nuova nota.

Figura 67: Diagramma della classe **NoteDTO**

3.4.5 Contatti Fidati

La funzionalità dei Contatti Fidati permette all'**Utente^G** di gestire una lista di persone di fiducia da avvisare in caso di inattività tramite il sistema di *Dead Man's Switch*.



Analogamente al resto dell'applicazione, la funzionalità è implementata secondo il pattern MVVM e le linee guida architetturali di Flutter, distinguendo un **Data Layer^G**, composto da Service e repository, e un **UI Layer^G**, composto da View e ViewModel.

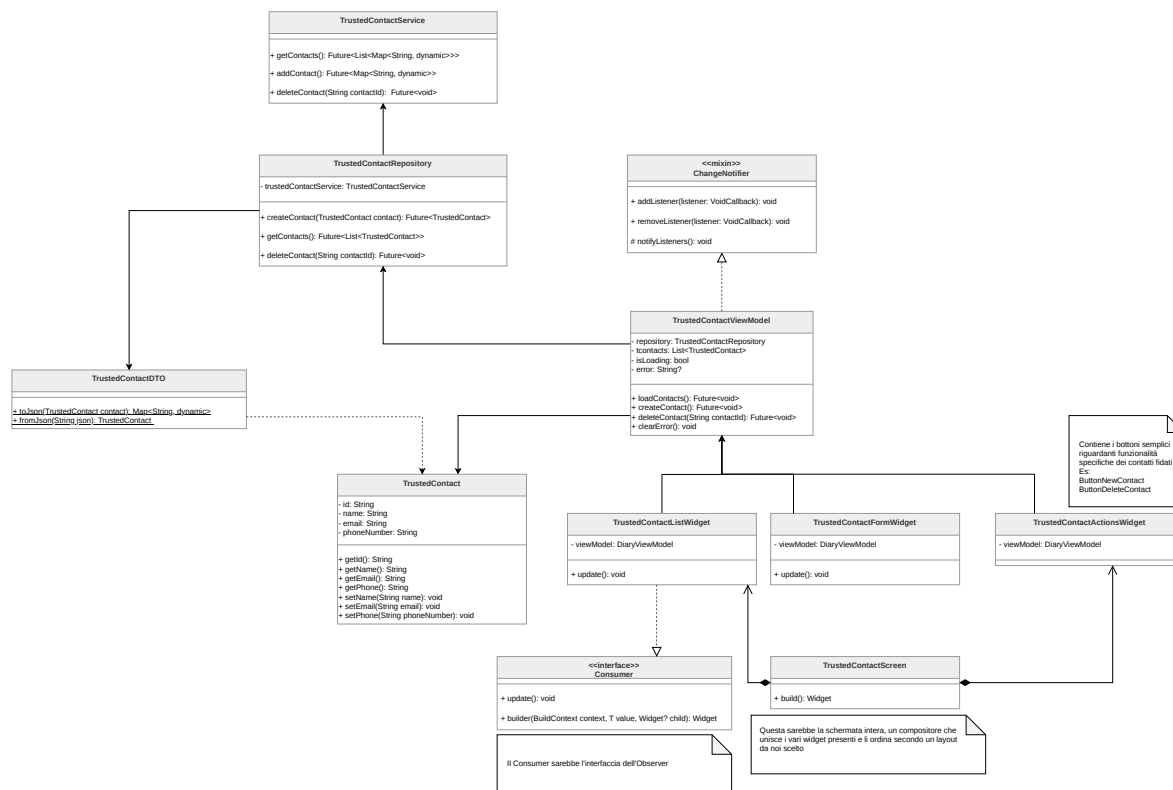


Figura 68: Diagramma delle classi relativo alle funzionalità dei Contatti Fidati

3.4.5.1 TrustedContactScreen

La classe **TrustedContactScreen** rappresenta la schermata principale dedicata alla gestione dei contatti fidati all'interno del *Presentation Layer*. Essa funge da compositore, unendo i vari widget presenti e ordinandoli secondo il layout stabilito, delegando la logica di stato ai componenti sottostanti.

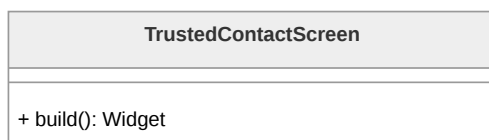


Figura 69: Diagramma della classe **TrustedContactScreen**

3.4.5.2 TrustedContactListWidget



La classe **TrustedContactListWidget** è responsabile della visualizzazione dell'elenco dei contatti attualmente salvati. Implementando l'interfaccia **Consumer**, osserva il ViewModel per reagire ai cambiamenti di stato e aggiornare dinamicamente l'interfaccia grafica.

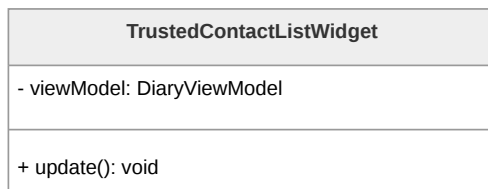


Figura 70: Diagramma della classe **TrustedContactListWidget**

3.4.5.3 TrustedContactFormWidget

TrustedContactFormWidget gestisce i campi di input necessari per l'inserimento o la modifica dei dati di un contatto fidato. In quanto **Consumer**, si aggiorna in base allo stato del ViewModel.

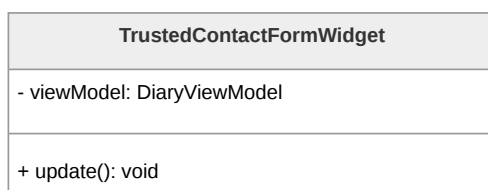


Figura 71: Diagramma della classe **TrustedContactFormWidget**

3.4.5.4 TrustedContactActionsWidget

Questo widget raggruppa i bottoni e le azioni specifiche per la gestione dei contatti fidati (ad esempio i pulsanti per l'aggiunta di un nuovo contatto o l'eliminazione di uno esistente). Comunica le intenzioni dell'**Utente^G** direttamente al ViewModel.

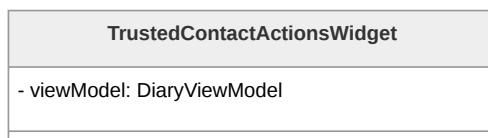


Figura 72: Diagramma della classe **TrustedContactActionsWidget**

3.4.5.5 TrustedContactViewModel

TrustedContactViewModel è il nucleo della gestione dello stato per questa funzionalità. Mantiene la lista dei contatti (**tcontacts**), lo stato di caricamento (**isLoading**) e la



gestione degli errori (**error**). Espone i metodi per caricare, creare e cancellare i contatti, facendo da ponte tra la View e il repository. Utilizza il mixin **ChangeNotifier** per notificare i widget in ascolto.

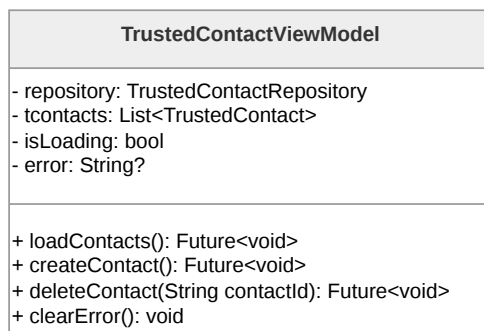


Figura 73: Diagramma della classe **TrustedContactViewModel**

3.4.5.6 TrustedContact

TrustedContact è la classe di dominio che modella il singolo contatto fidato. Contiene le informazioni anagrafiche e di recapito essenziali, quali l'identificativo unico, il nome, l'indirizzo email e il numero di telefono, fornendo i relativi metodi di accesso e modifica.

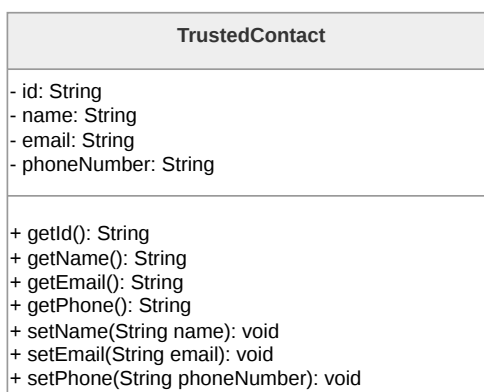


Figura 74: Diagramma della classe **TrustedContact**

3.4.5.7 TrustedContactRepository

TrustedContactRepository astrae la sorgente dei dati per il ViewModel. Utilizza **TrustedContactService** per eseguire le operazioni CRUD (creazione, lettura, eliminazione), trasformando i dati ricevuti in oggetti di dominio **TrustedContact** e isolando la logica di business dai dettagli di implementazione del network.

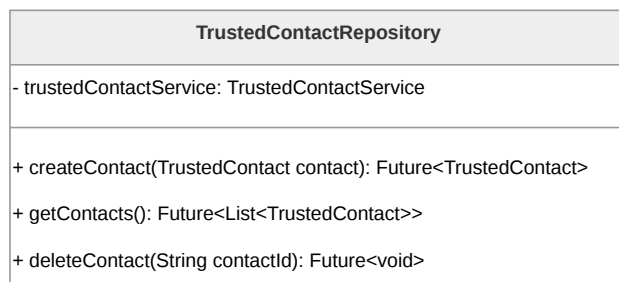


Figura 75: Diagramma della classe **TrustedContactRepository**

3.4.5.8 TrustedContactService

TrustedContactService appartiene al livello infrastrutturale e si occupa di effettuare fisicamente le chiamate API verso il **Backend^G**. Gestisce le richieste HTTP per ottenere l'elenco dei contatti, aggiungerne di nuovi o eliminarli, restituendo le risposte grezze al repository.

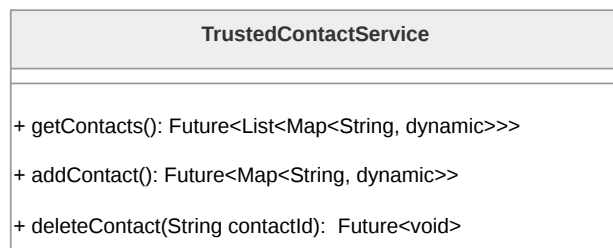


Figura 76: Diagramma della classe **TrustedContactService**

3.4.5.9 TrustedContactDTO

TrustedContactDTO (*Data Transfer Object*) gestisce la serializzazione e deserializzazione dei dati. Mappa i dati JSON provenienti dall'esterno in oggetti **TrustedContact** e viceversa, garantendo la compatibilità del formato scambiato con le API.

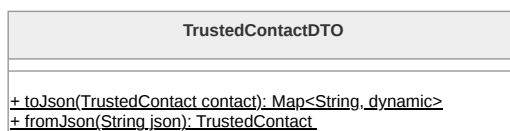


Figura 77: Diagramma della classe **TrustedContactDTO**

3.4.6 Luoghi Sicuri

La funzionalità dei Luoghi Sicuri permette all'**Utente^G** di visualizzare geograficamente, tramite un'interfaccia cartografica, le strutture di supporto, i centri anti-violenza e

altri punti di interesse rilevanti. Analogamente al resto dell'applicazione, la funzionalità è implementata secondo il pattern MVVM e le linee guida architetturali di Flutter, distinguendo un **Data Layer^G**, composto da Service e repository, e un **UI Layer^G**, composto da View e ViewModel, integrando la libreria ufficiale `google_maps_flutter` per il rendering della mappa.

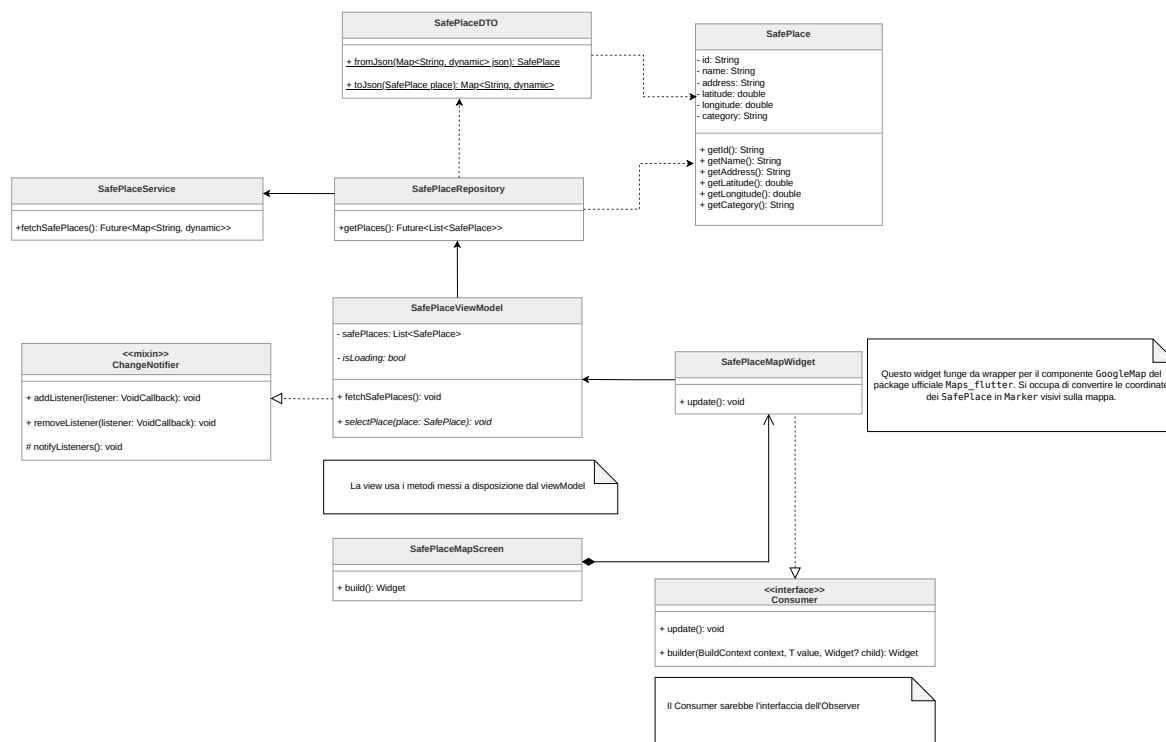


Figura 78: Diagramma delle classi complessivo della funzionalità Luoghi Sicuri

3.4.6.1 SafePlaceMapScreen

La classe `SafePlaceMapScreen` rappresenta la schermata principale dedicata alla visualizzazione dei luoghi sicuri all'interno del *Presentation Layer*. Essa funge da compositore, organizzando il layout della pagina e delegando la logica di visualizzazione cartografica al widget figlio specializzato.

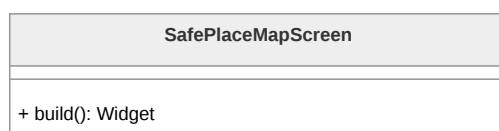


Figura 79: Diagramma della classe `SafePlaceMapScreen`

3.4.6.2 SafePlaceMapWidget

Il componente `SafePlaceMapWidget` incapsula la logica necessaria all'interazione con la cartografia digitale. Implementando l'interfaccia `Consumer`, osserva il ViewModel per reagire ai cambiamenti dei dati. Al suo interno, utilizza il widget `GoogleMap` fornito dal **Framework^G** per visualizzare la mappa e si occupa



di mappare gli oggetti di dominio **SafePlace** in entità **Marker** (i "pin" grafici) posizionati in base alle coordinate geografiche caricate.

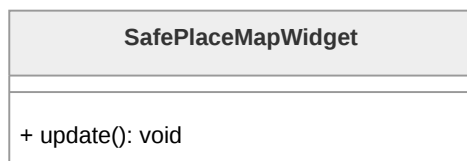


Figura 80: Diagramma della classe **SafePlaceMapWidget**

3.4.6.3 SafePlaceViewModel **SafePlaceViewModel** rappresenta il nucleo della gestione dello stato per questa funzionalità. Mantiene la lista dei luoghi sicuri (**safePlaces**), gestisce lo stato di caricamento (**isLoading**) e la gestione degli errori. Espone i metodi per il caricamento dei dati dal repository e utilizza il mixin **ChangeNotifier** per notificare i widget della View non appena i dati geografici sono pronti per essere visualizzati sulla mappa.

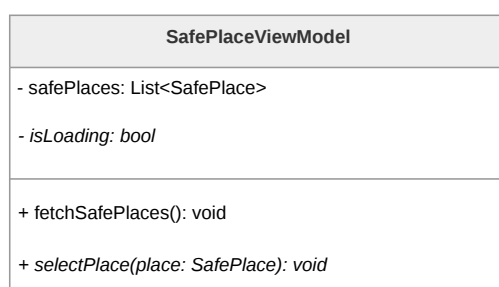


Figura 81: Diagramma della classe **SafePlaceViewModel**

3.4.6.4 SafePlace **SafePlace** è la classe di dominio che modella il singolo luogo fisico di interesse. Oltre agli attributi identificativi e descrittivi (nome, indirizzo, categoria), include le proprietà fondamentali di latitudine (**latitude**) e longitudine (**longitude**), necessarie per il corretto posizionamento del marker sulla superficie cartografica.

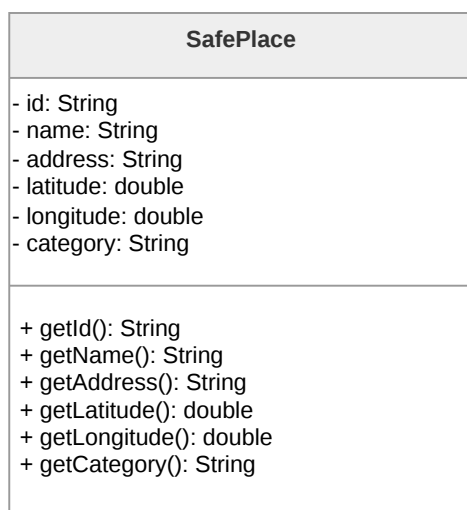


Figura 82: Diagramma della classe **SafePlace**

3.4.6.5 SafePlaceRepository Il **SafePlaceRepository** astrae la sorgente dei dati geografici per il ViewModel. Utilizza **SafePlaceService** per prelevare le informazioni grezze e le converte in oggetti di dominio **SafePlace** tramite il relativo DTO, garantendo che la logica di business riceva dati tipizzati e pronti all'uso, indipendentemente dalla specifica sorgente dati.

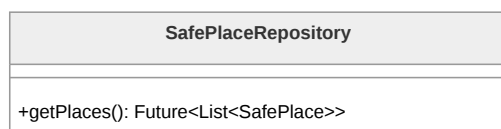


Figura 83: Diagramma della classe **SafePlaceRepository**

3.4.6.6 SafePlaceService Appartenente al livello infrastrutturale, **SafePlaceService** si occupa di effettuare le chiamate verso il **Backend^G** o sorgenti dati esterne per ottenere l'elenco dei luoghi sicuri in formato grezzo, restituendo le risposte al repository.

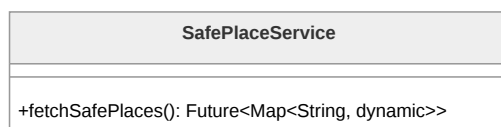


Figura 84: Diagramma della classe **SafePlaceService**

3.4.6.7 SafePlaceDTO La classe **SafePlaceDTO** (*Data Transfer Object*) gestisce la serializzazione e deserializzazione dei dati geografici. Mappa le chiavi dei campi JSON



(incluse le coordinate) in istanze della classe **SafePlace**, assicurando che il formato dei dati scambiati con l'esterno sia compatibile con il modello interno dell'applicazione.

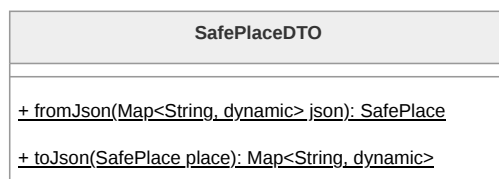


Figura 85: Diagramma della classe **SafePlaceDTO**

3.4.7 Dead Man's Switch

La funzionalità del *Dead Man's Switch* rappresenta un sistema di sicurezza critico che invia avvisi automatici in caso di inattività prolungata dell'**Utente^G**. L'**Architettura^G** frontend è focalizzata sulla gestione sicura della configurazione di tale sistema. Il frontend non interagisce direttamente con servizi di posta elettronica, ma demanda la gestione dei timer e l'invio delle email tramite AWS SES al **Backend^G**, garantendo così la sicurezza e l'affidabilità del servizio anche ad applicazione chiusa. Analogamente al resto dell'applicazione, la funzionalità è implementata secondo il pattern MVVM e le linee guida architetturali di Flutter, distinguendo un **Data Layer^G**, composto da Service e repository, e un **UI Layer^G**, composto da View e ViewModel.

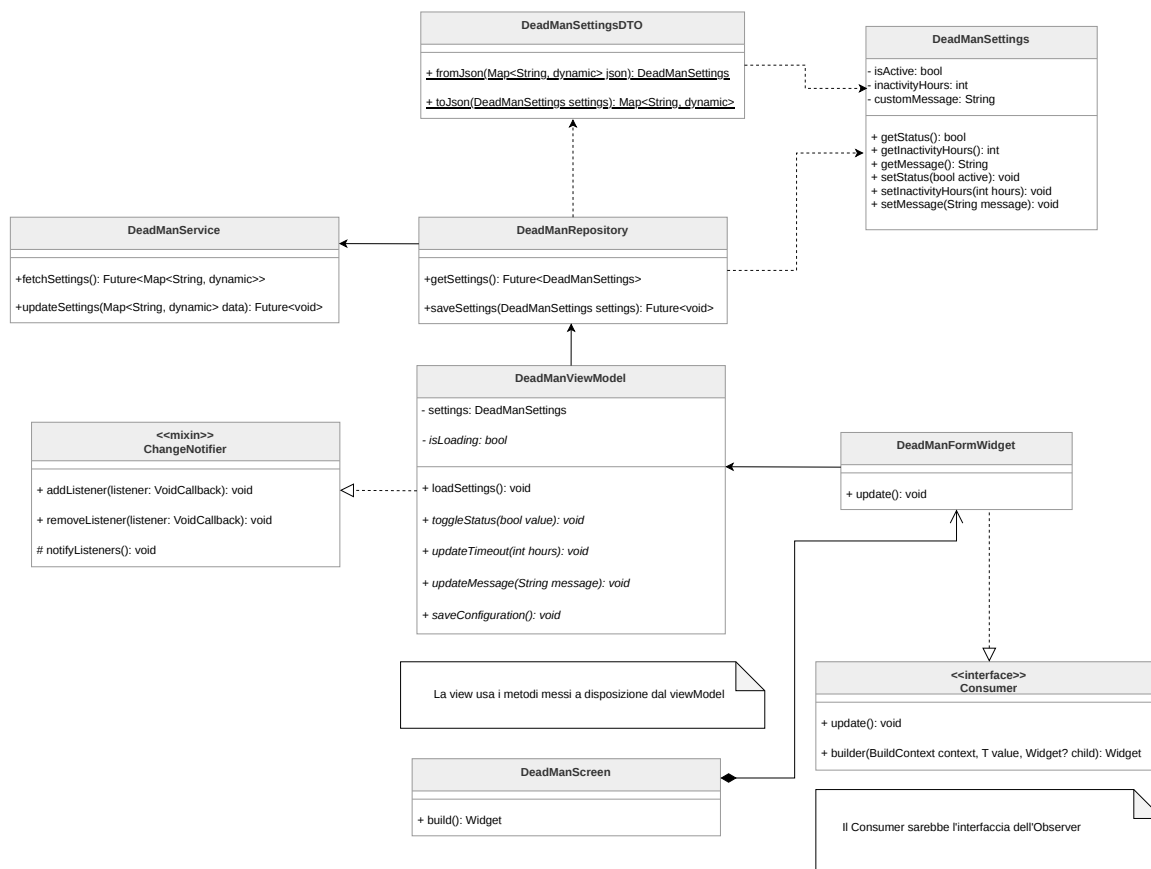


Figura 86: Diagramma delle classi relativo alla funzionalità del Dead Man's Switch

3.4.7.1 DeadManScreen La classe **DeadManScreen** rappresenta la schermata principale del pannello di controllo per la sicurezza. Funge da compositore, strutturando il layout della pagina e ospitando i form interattivi, delegando al ViewModel la reattività e la **Persistence Logic**^G.

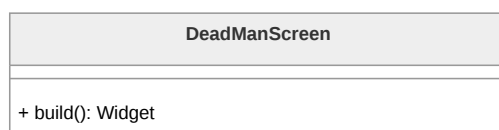


Figura 87: Diagramma della classe **DeadManScreen**

3.4.7.2 DeadManFormWidget Questo componente incapsula gli elementi interattivi dell'interfaccia, come i selettori (switch) per l'attivazione del servizio, i campi di input per la soglia di inattività (es. ore o giorni) e le aree di testo per il messaggio di emergenza personalizzato. Implementando **Consumer**, aggiorna i propri campi riflettendo fedelmente lo stato mantenuto nel ViewModel.



Figura 88: Diagramma della classe **DeadManFormWidget**

3.4.7.3 DeadManViewModel **DeadManViewModel** orchestra la logica di business di questa sezione. Mantiene in memoria le configurazioni attuali dell'**Utente^G** (**settings**) e lo stato di sincronizzazione (**isLoading**). Implementa il mixin **ChangeNotifier** per notificare la UI a ogni interazione. Oltre a recuperare i dati, espone metodi fondamentali come **saveConfiguration()**, che permette di confermare e trasmettere al repository le nuove impostazioni scelte dall'**Utente^G**.

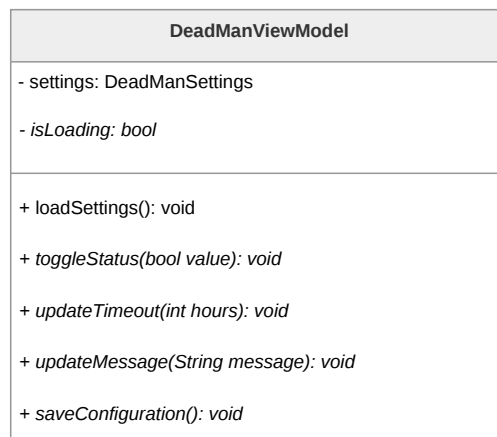
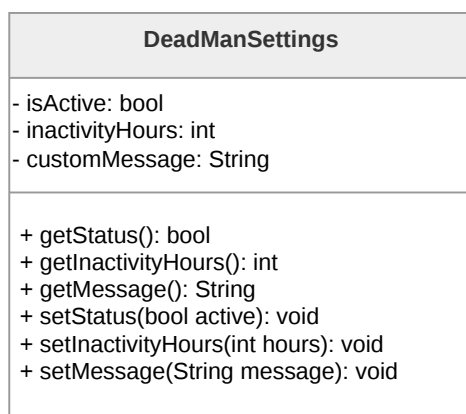
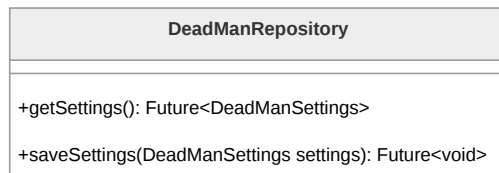


Figura 89: Diagramma della classe **DeadManViewModel**

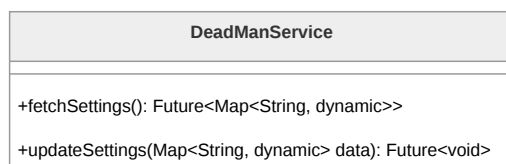
3.4.7.4 DeadManSettings **DeadManSettings** è l'entità di dominio che modella le preferenze di sicurezza. Contiene gli attributi necessari a configurare l'allarme: uno stato booleano di attivazione (**isActive**), un valore temporale che definisce la soglia di tolleranza (**inactivityHours**) e il corpo del messaggio testuale (**customMessage**) che il **Backend^G** inoltrerà ai contatti fidati tramite AWS SES.

Figura 90: Diagramma della classe **DeadManSettings**

3.4.7.5 DeadManRepository Il **DeadManRepository** gestisce la persistenza e il recupero delle configurazioni. Agisce da ponte tra la **Presentation Logic^G** e il livello infrastrutturale, utilizzando **DeadManService** per inviare le preferenze aggiornate e convertendo le risposte di rete in oggetti di dominio **DeadManSettings** sicuri e validati tramite il DTO.

Figura 91: Diagramma della classe **DeadManRepository**

3.4.7.6 DeadManService Appartenente al **Data Layer^G**, **DeadManService** si occupa della comunicazione HTTP/REST con le API del **Backend^G**. Dispone dei metodi necessari per effettuare le operazioni di *GET* (lettura parametri) e *PUT/POST* (aggiornamento configurazioni) sul database remoto.

Figura 92: Diagramma della classe **DeadManService**

3.4.7.7 DeadManSettingsDTO Il **DeadManSettingsDTO** è l'oggetto di trasferimento dati incaricato della serializzazione. Consente di mappare i campi JSON inviati e



ricevuti dal server, assicurando che la comunicazione tra l'app Flutter e il **Backend**^G mantenga un contratto dati rigoroso.

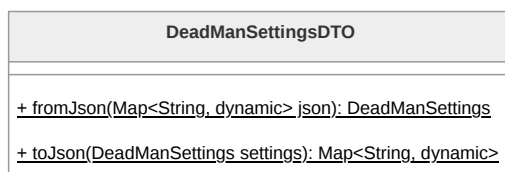


Figura 93: Diagramma della classe `DeadManSettingsDTO`

3.4.8 Area Informativa

L'Area Informativa costituisce un centro di risorse in sola lettura per l'**Utente^G**, aggregando informazioni su community di supporto, riferimenti normativi e materiale divulgativo. La funzionalità è implementata secondo il pattern MVVM e le linee guida architetturali di Flutter, distinguendo un **Data Layer^G**, composto da Service e repository, e un **UI Layer^G**, composto da View e ViewModel.

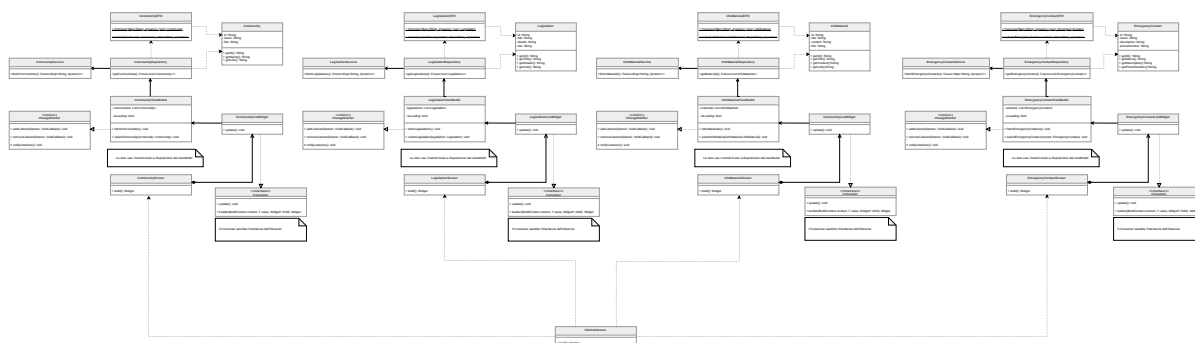


Figura 94: Diagramma delle classi complessivo per l'Area Informativa

3.4.8.1 InfoHubScreen

La classe `InfoHubScreen` funge da punto di ingresso per l'intera area informativa. Come componente del *Presentation Layer*, organizza il layout principale e gestisce la navigazione verso le tre sottosezioni specifiche: Community, Legislazioni e Materiale Informativo. Essendo una classe View e quindi solamente adibita alla visualizzazione di widget, non contiene particolare logica interna.

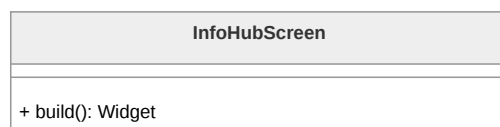


Figura 95: Diagramma della classe `InfoHubScreen`

3.4.8.2 CommunityScreen

CommunityScreen rappresenta la vista principale per la funzionalità di consultazione delle community. Organizza la struttura e l'ordinamento dei



widget per la lista e il dettaglio, delegando la **Presentation Logic^G** e il recupero dei dati al relativo ViewModel.

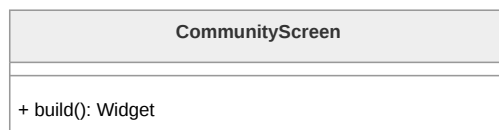


Figura 96: Diagramma della classe **CommunityScreen**

3.4.8.3 CommunityListWidget Responsabile della renderizzazione dell'elenco delle community, **CommunityListWidget** implementa l'interfaccia **Consumer**. Questo gli permette di osservare il **CommunityViewModel** e aggiornare la visualizzazione in seguito ad una notifica dal ViewModel.

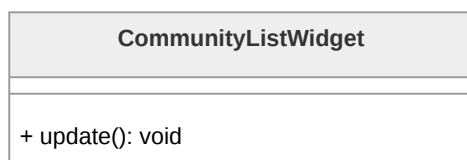


Figura 97: Diagramma della classe **CommunityListWidget**

3.4.8.4 CommunityViewModel **CommunityViewModel** è il nucleo logico della sezione. Gestisce la lista di oggetti **Community**, lo stato di caricamento (**isLoading**) per la gestione dei feedback visivi (spinner) e i messaggi di errore. Utilizza il mixin **ChangeNotifier** per notificare la View in seguito ad un cambiamento di stato dei dati.

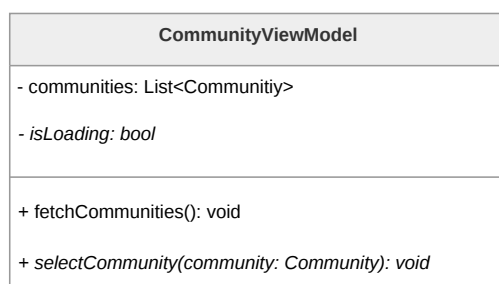


Figura 98: Diagramma della classe **CommunityViewModel**

3.4.8.5 Community Classe di dominio che modella le informazioni di una community di supporto. Include attributi fondamentali quali l'identificativo, il nome, la descrizione e l'URL di riferimento, fornendo i metodi necessari per l'accesso ai dati in sola lettura.

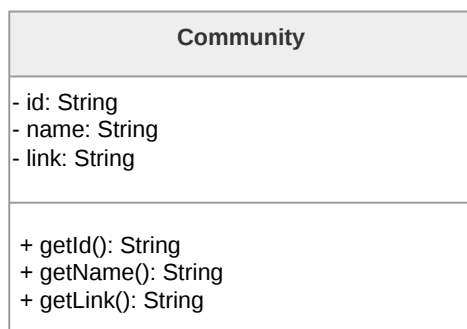


Figura 99: Diagramma della classe **Community**

3.4.8.6 CommunityRepository La classe **CommunityRepository** funge da intermediario tra il ViewModel e il Service. Si occupa di richiedere i dati grezzi, gestirne la conversione tramite il DTO e restituire oggetti di dominio **Community** "puliti", isolando la *Business Logic*^G dai dettagli di implementazione della rete.

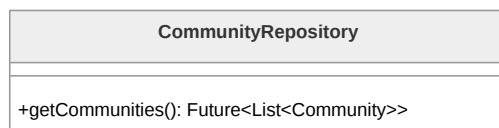


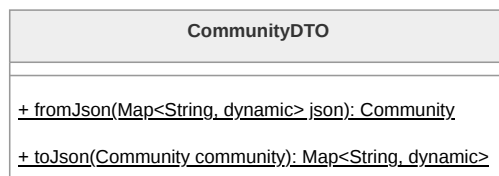
Figura 100: Diagramma della classe **CommunityRepository**

3.4.8.7 CommunityService Appartenente al **Data Layer**^G, **CommunityService** esegue le richieste HTTP verso le API del **Backend**^G per ottenere l'elenco delle community salvate nel database.

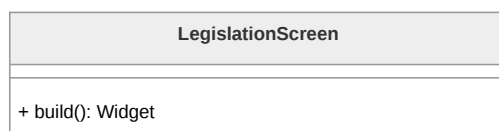


Figura 101: Diagramma della classe **CommunityService**

3.4.8.8 CommunityDTO La classe **CommunityDTO** gestisce la logica di mappatura dei dati. Implementa i metodi **fromJson** e **toJson** per trasformare le risposte JSON provenienti dal network in istanze della classe **Community**.

Figura 102: Diagramma della classe **CommunityDTO**

3.4.8.9 LegislationScreen **LegislationScreen** rappresenta la vista principale per la funzionalità di consultazione delle leggi. Organizza la struttura e l'ordinamento dei widget per la lista e il dettaglio, delegando la **Presentation Logic**^G e il recupero dei dati al relativo ViewModel.

Figura 103: Diagramma della classe **LegislationScreen**

3.4.8.10 LegislationListWidget Responsabile della renderizzazione dell'elenco delle leggi, **LegislationListWidget** implementa l'interfaccia **Consumer**. Questo gli permette di osservare il **LegislationViewModel** e aggiornare la visualizzazione in seguito ad una notifica dal ViewModel.

Figura 104: Diagramma della classe **LegislationListWidget**

3.4.8.11 LegislationViewModel **LegislationViewModel** è il nucleo logico della sezione. Gestisce la lista di oggetti **Legislation**, lo stato di caricamento (**isLoading**) per la gestione dei feedback visivi (spinner) e i messaggi di errore. Utilizza il mixin **ChangeNotifier** per notificare la View in seguito ad un cambiamento di stato dei dati.

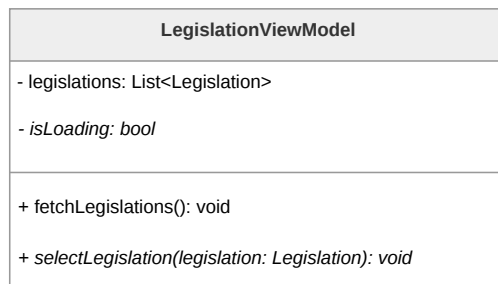


Figura 105: Diagramma della classe **LegislationViewModel**

3.4.8.12 Legislation Classe di dominio che modella le informazioni di una legge o di un riferimento normativo. Include attributi fondamentali quali l'identificativo, il titolo, il testo o la descrizione dell'articolo e l'URL di riferimento alla fonte ufficiale, fornendo i metodi necessari per l'accesso ai dati in sola lettura.

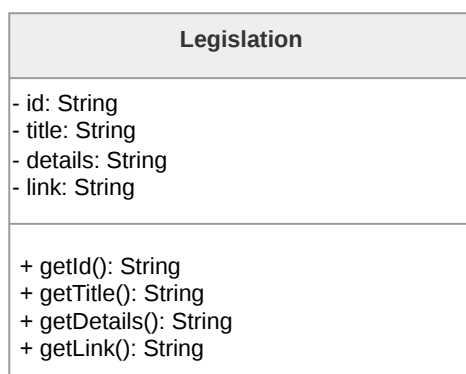


Figura 106: Diagramma della classe **Legislation**

3.4.8.13 LegislationRepository La classe **LegislationRepository** funge da intermediario tra il ViewModel e il Service. Si occupa di richiedere i dati grezzi, gestirne la conversione tramite il DTO e restituire oggetti di dominio **Legislation** "puliti", isolando la *Business Logic*^G dai dettagli di implementazione della rete.



Figura 107: Diagramma della classe **LegislationRepository**



3.4.8.14 LegislationService Appartenente al **Data Layer^G**, **LegislationService** esegue le richieste HTTP verso le API del **Backend^G** per ottenere l'elenco delle leggi salvate nel database.



Figura 108: Diagramma della classe **LegislationService**

3.4.8.15 LegislationDTO La classe **LegislationDTO** gestisce la logica di mappatura dei dati. Implementa i metodi **fromJson** e **toJson** per trasformare le risposte JSON provenienti dal network in istanze della classe **Legislation**.

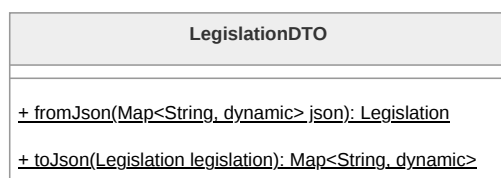


Figura 109: Diagramma della classe **LegislationDTO**

3.4.8.16 InfoMaterialScreen **InfoMaterialScreen** rappresenta la vista principale per la funzionalità di consultazione di materiale informativo e divulgativo. Organizza la struttura e l'ordinamento dei widget per la lista e il dettaglio, delegando la **Presentation Logic^G** e il recupero dei dati al relativo ViewModel.

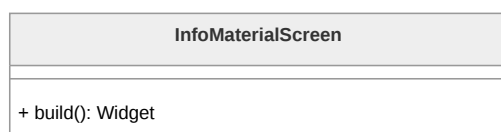


Figura 110: Diagramma della classe **InfoMaterialScreen**

3.4.8.17 InfoMaterialListWidget Responsabile della renderizzazione dell'elenco del materiale informativo, **InfoMaterialListWidget** implementa l'interfaccia **Consumer**. Questo gli permette di osservare il **InfoMaterialViewModel** e aggiornare la visualizzazione in seguito ad una notifica dal ViewModel.

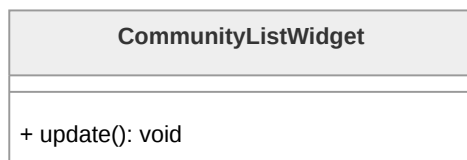


Figura 111: Diagramma della classe **InfoMaterialListWidget**

3.4.8.18 InfoMaterialViewModel **InfoMaterialViewModel** è il nucleo logico della sezione. Gestisce la lista di oggetti **InfoMaterial**, lo stato di caricamento (**isLoading**) per la gestione dei feedback visivi (spinner) e i messaggi di errore. Utilizza il mixin **ChangeNotifier** per notificare la View in seguito ad un cambiamento di stato dei dati.

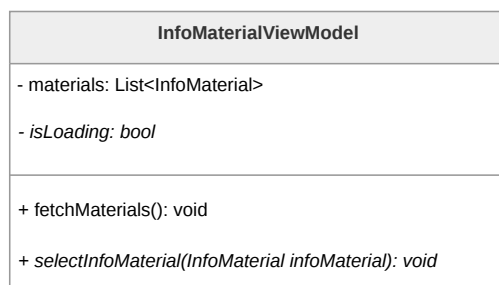


Figura 112: Diagramma della classe **InfoMaterialViewModel**

3.4.8.19 InfoMaterial Classe di dominio che modella le informazioni di un materiale informativo. Include attributi fondamentali quali l'identificativo, il titolo, il contenuto e eventuale URL di riferimento, fornendo i metodi necessari per l'accesso ai dati in sola lettura.

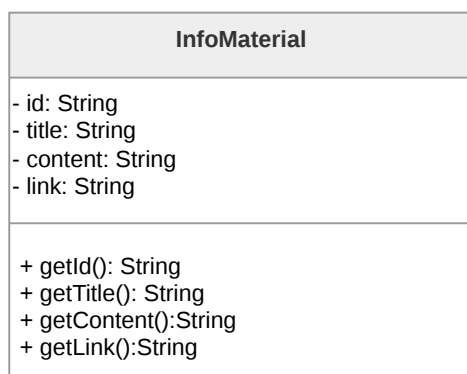


Figura 113: Diagramma della classe **InfoMaterial**



3.4.8.20 InfoMaterialRepository La classe **InfoMaterialRepository** funge da intermediario tra il ViewModel e il Service. Si occupa di richiedere i dati grezzi, gestirne la conversione tramite il DTO e restituire oggetti di dominio **InfoMaterial** "puliti", isolando la **Business Logic^G** dai dettagli di implementazione della rete.

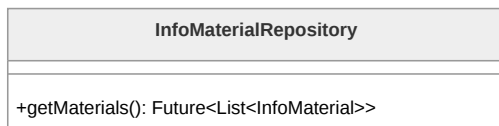


Figura 114: Diagramma della classe **InfoMaterialRepository**

3.4.8.21 InfoMaterialService Appartenente al **Data Layer^G**, **InfoMaterialService** esegue le richieste HTTP verso le API del **Backend^G** per ottenere l'elenco dei materiali informativi salvati nel database.

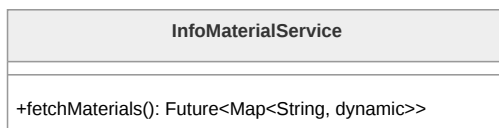


Figura 115: Diagramma della classe **InfoMaterialService**

3.4.8.22 InfoMaterialDTO La classe **InfoMaterialDTO** gestisce la logica di mappatura dei dati. Implementa i metodi **fromJson** e **toJson** per trasformare le risposte JSON provenienti dal network in istanze della classe **InfoMaterial**.

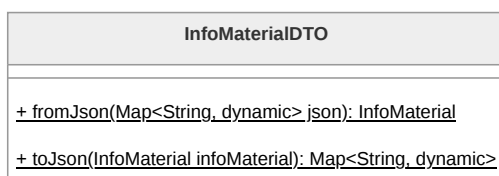


Figura 116: Diagramma della classe **InfoMaterialDTO**

3.4.8.23 EmergencyContactScreen **EmergencyContactScreen** rappresenta la vista principale per la consultazione rapida dei contatti di emergenza o degli enti di supporto preposti. Organizza la struttura e l'ordinamento dei widget, delegando la **Presentation Logic^G** e il recupero dei dati al relativo ViewModel.

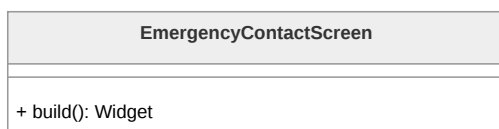


Figura 117: Diagramma della classe **EmergencyContactScreen**



3.4.8.24 EmergencyContactListWidget Responsabile della renderizzazione dell'elenco dei contatti, **EmergencyContactListWidget** implementa l'interfaccia **Consumer**. Questo gli permette di osservare il **EmergencyContactViewModel** e aggiornare la visualizzazione in seguito ad una notifica dal ViewModel. Inoltre, gestisce l'interazione dell'**Utente^G**: alla selezione di una voce, intercetta l'evento e segnala l'intenzione di avviare una chiamata telefonica.

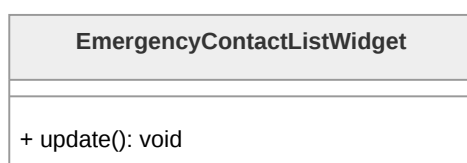


Figura 118: Diagramma della classe **EmergencyContactListWidget**

3.4.8.25 EmergencyContactViewModel **EmergencyContactViewModel** è il nucleo logico della sezione. Gestisce la lista di oggetti **EmergencyContact** e lo stato di caricamento (**isLoading**). Utilizza il mixin **ChangeNotifier** per notificare la View in seguito ad un cambiamento di stato dei dati. Inoltre, espone metodi per gestire l'avvio della telefonata (ad esempio interfacciandosi con servizi di sistema tramite *intent* o URL launcher) quando un contatto viene selezionato.

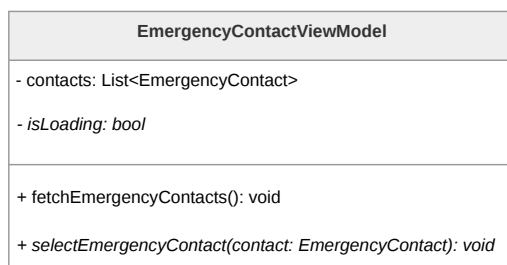


Figura 119: Diagramma della classe **EmergencyContactViewModel**

3.4.8.26 EmergencyContact Classe di dominio che modella le informazioni di un ente o contatto di emergenza. Include attributi fondamentali e immediati quali l'identificativo, il nome dell'ente e il numero di telefono (**phoneNumber**), fornendo i metodi necessari per l'accesso ai dati in sola lettura.

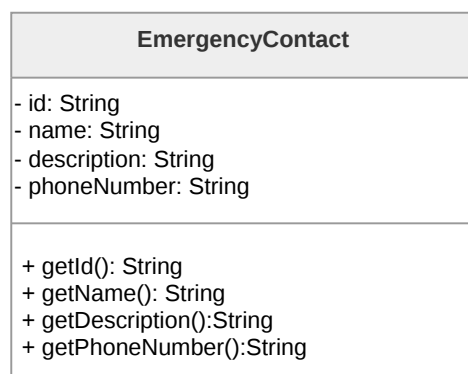


Figura 120: Diagramma della classe **EmergencyContact**

3.4.8.27 EmergencyContactRepository La classe **EmergencyContactRepository** funge da intermediario tra il ViewModel e il Service. Si occupa di richiedere i dati grezzi, gestirne la conversione tramite il DTO e restituire oggetti di dominio **EmergencyContact** "puliti", isolando la **Business Logic**^G dai dettagli di implementazione della rete.



Figura 121: Diagramma della classe **EmergencyContactRepository**

3.4.8.28 EmergencyContactService Appartenente al **Data Layer**^G, **EmergencyContactService** esegue le richieste verso il **Backend**^G per ottenere l'elenco aggiornato dei contatti di emergenza salvati nel database.



Figura 122: Diagramma della classe **EmergencyContactService**

3.4.8.29 EmergencyContactDTO La classe **EmergencyContactDTO** gestisce la logica di mappatura dei dati. Implementa i metodi di conversione JSON per trasformare i **Payload**^G di rete in istanze della classe **EmergencyContact**.

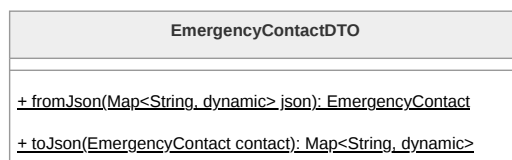


Figura 123: Diagramma della classe **EmergencyContactDTO**

3.5. Architettura Back End

3.5.1 Autenticazione

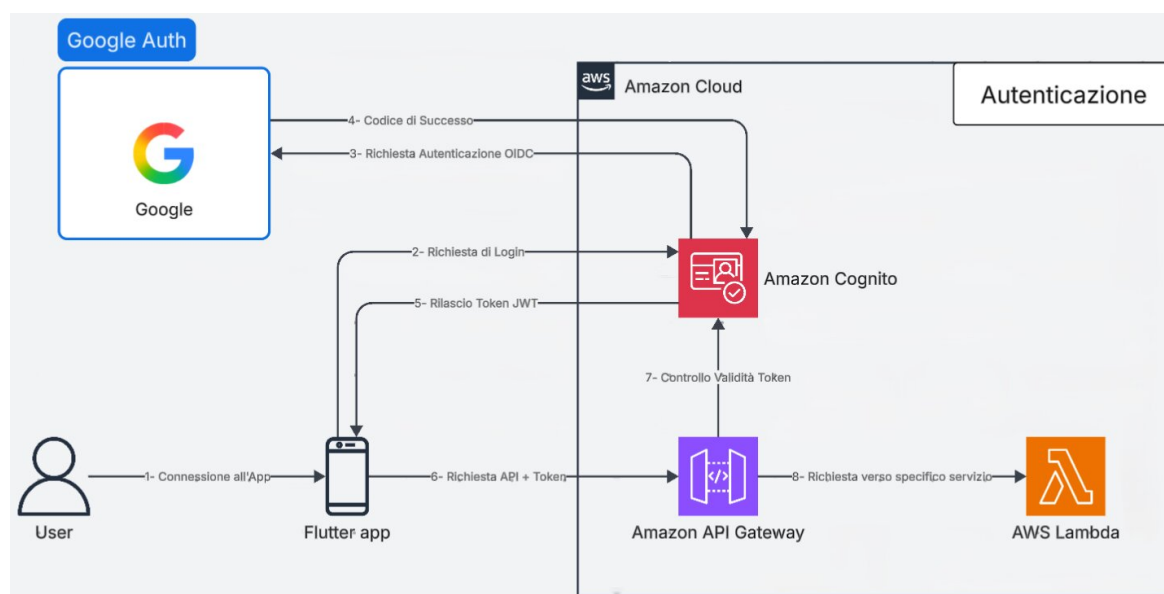


Figura 124: **Architettura^G** e Flusso di Autenticazione

3.5.1.1 Architettura e Flusso di Autenticazione: Modulo Federated Login

La presente sezione descrive l'**Architettura^G** logica e il flusso dei dati relativi al modulo di autenticazione e autorizzazione dell'applicazione. Il sistema adotta un modello di federated login basato su Amazon Cognito integrato con Google come Identity Provider (IdP), progettato per garantire un accesso sicuro e standardizzato alle risorse di **Backend^G** senza demandare la gestione crittografica delle credenziali all'applicazione client.



3.5.1.1.1 Iniziazione del Flusso e Autenticazione Federata

Il processo di autenticazione ha inizio quando l'**Utente^G** interagisce con l'applicazione client (Flutter) richiedendo l'accesso al sistema. L'applicazione delega interamente la gestione dell'identità inoltrando una richiesta di login ad Amazon Cognito. Cognito, fungendo da Service Provider, intercetta la richiesta e avvia una procedura di autenticazione basata sullo standard OpenID Connect (OIDC) delegandola ai server di Google. L'**Utente^G** completa la fase di riconoscimento interfacciandosi direttamente con l'infrastruttura del provider esterno. A seguito della corretta verifica delle credenziali, Google restituisce ad Amazon Cognito un codice di autorizzazione attestante il successo dell'operazione e l'identità dell'**Utente^G**.

3.5.1.1.2 Emissione dei Token di Sessione

Acquisita la conferma dal provider federato, Amazon Cognito processa il codice di autorizzazione e genera un set di token crittografici aderenti allo standard JWT (JSON Web Token). Questi token vengono trasmessi in modo sicuro all'applicazione mobile, concludendo la fase di autenticazione. Da questo momento, il client dispone di una prova di identità verificata e utilizzerà l'Access Token JWT per autorizzare le successive chiamate di rete dirette ai servizi di **Backend^G**.

3.5.1.1.3 Validazione Perimetrale e Instradamento Sicuro

Per accedere alle logiche applicative, l'app Flutter compone una richiesta HTTPS indirizzata all'Amazon API Gateway, includendo il token JWT nelle intestazioni di autorizzazione. L'API Gateway assume il ruolo di strato di sicurezza perimetrale (edge security). Prima di instradare il traffico, il Gateway utilizza un Cognito Authorizer nativo per eseguire un rigoroso controllo di validità sul token ricevuto. Tale procedura verifica l'autenticità della firma crittografica, l'integrità del **Payload^G** e la validità temporale della sessione direttamente contro le configurazioni dello User Pool di Cognito.

Questa specifica impostazione architetturale assicura che qualsiasi richiesta malevola, non autorizzata o provvista di credenziali scadute venga respinta al livello di rete. Solamente le richieste che superano con successo la validazione del token vengono instradate dall'API Gateway verso la specifica funzione AWS Lambda designata, garantendo che le risorse computazionali elaborino esclusivamente traffico certificato e sicuro.



3.5.2 Luoghi sicuri

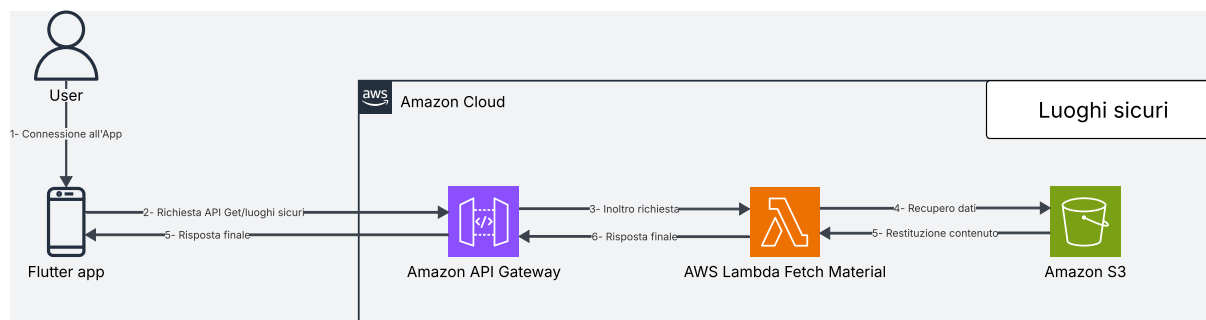


Figura 125: **Architettura^G** moduli Luoghi Sicuri

3.5.2.1 Architettura e Flusso di Elaborazione: Modulo Luoghi Sicuri

Il modulo "Luoghi Sicuri" è stato ingegnerizzato adottando una rigorosa separazione delle responsabilità (*Separation of Concerns^G*) tra l'elaborazione lato client (Frontend) e l'infrastruttura **Cloud^G** (**Backend^G**). L'obiettivo primario è stato quello di minimizzare il carico computazionale server-side ed eliminare del tutto i costi legati ai calcoli geospaziali.

Il flusso di elaborazione si articola nei seguenti passaggi:

- **Ingestione della Richiesta:** L'**Utente^G** accede alla sezione della mappa tramite l'applicazione. Il client Flutter genera una richiesta HTTPS (metodo `GET /safe_locations`).
- **Instradamento verso Lambda:** API Gateway intercetta la richiesta, ne valida il contesto e la inoltra alla funzione AWS Lambda dedicata alla gestione dei luoghi sicuri.
- **Accesso ai dati e logica applicativa:** la Lambda esegue la logica applicativa necessaria e recupera i dati dal bucket S3 dedicato. Al suo interno sono infatti memorizzati sia i metadati descrittivi, sia le coordinate geospaziali (latitudine e longitudine) per ciascun punto di interesse.
- **Elaborazione e serializzazione:** la funzione Lambda elabora i dati estratti e li converte in un **Payload^G** JSON standardizzato e ottimizzato per il consumo lato client.
- **Risposta al client:** API Gateway inoltra la risposta HTTP 200 OK al client Flutter contenente l'elenco dei luoghi sicuri.



3.5.2.1.1 Delega della Logica Geospaziale al Livello Client (Edge Computing)

Al fine di ottimizzare ulteriormente i costi, l'intera logica di rendering visivo e di routing è delegata all'applicazione sul dispositivo dell'**Utente^G**. Il **Backend^G** AWS si configura esclusivamente come Data Provider di coordinate statiche. Alla ricezione del **Payload^G** JSON, l'applicativo Flutter si fa carico di istanziare la mappa nativa e posizionare i marker grafici (pin). L'azione di navigazione guidata (es. pulsante "Portami lì") sfrutta le Intents (su Android) o le URL Schemes (su iOS) del sistema operativo per invocare applicazioni di navigazione di terze parti (come Google Maps o Apple Maps). Questo approccio demanda il tracciamento GPS e il calcolo dinamico dell'itinerario al dispositivo locale, garantendo la scalabilità infinita del sistema AWS anche in condizioni di massiccio utilizzo concorrente.

3.5.2.2 API associate

- **GET /safe-places/**

- **Descrizione:** Recupera la lista dei luoghi sicuri disponibili, includendo informazioni geografiche e tipologia della struttura.

- **Parametri:**

- * Nessun parametro richiesto.

- **Response:**

- * 200 OK: richiesta completata con successo.

- * Corpo della risposta:

```
[
  {
    "marker_id": "luogo-001",
    "name": "Ospedale Centrale",
    "address": "Via Roma 10, Milano",
    "latitude": 45.4642,
    "longitude": 9.1900,
    "category": "ospedale"
  },
  {
    "marker_id": "luogo-002",
    "name": "Centro Antiviolenza Milano",
    "address": "Via Torino 25, Milano",
    "latitude": 45.4658,
    "longitude": 9.1859,
    "category": "centro_antiviolenza"
```



```
    },  
    {  
      "marker_id": "luogo-003",  
      "name": "Comando Carabinieri Milano Centro",  
      "address": "Piazza Duomo 1, Milano",  
      "latitude": 45.4641,  
      "longitude": 9.1919,  
      "category": "carabinieri"  
    }  
  ]
```

– **Errori:**

- * 405 Method Not Allowed: metodo HTTP non consentito per l'endpoint.
- * 500 Internal Server Error: errore interno del server.

Nota di sicurezza: L'integrazione di un Authorizer Cognito per la verifica delle autorizzazioni è stata deliberatamente omessa per questa route. L'assenza dell'Authorizer in quest'area è difatti una scelta architetturale mirata a rimuovere complessità infrastrutturale non necessaria e ad abbattere i costi operativi associati al recupero di questi specifici dati.



3.5.2.3 Architettura logica: Modulo Luoghi Sicuri

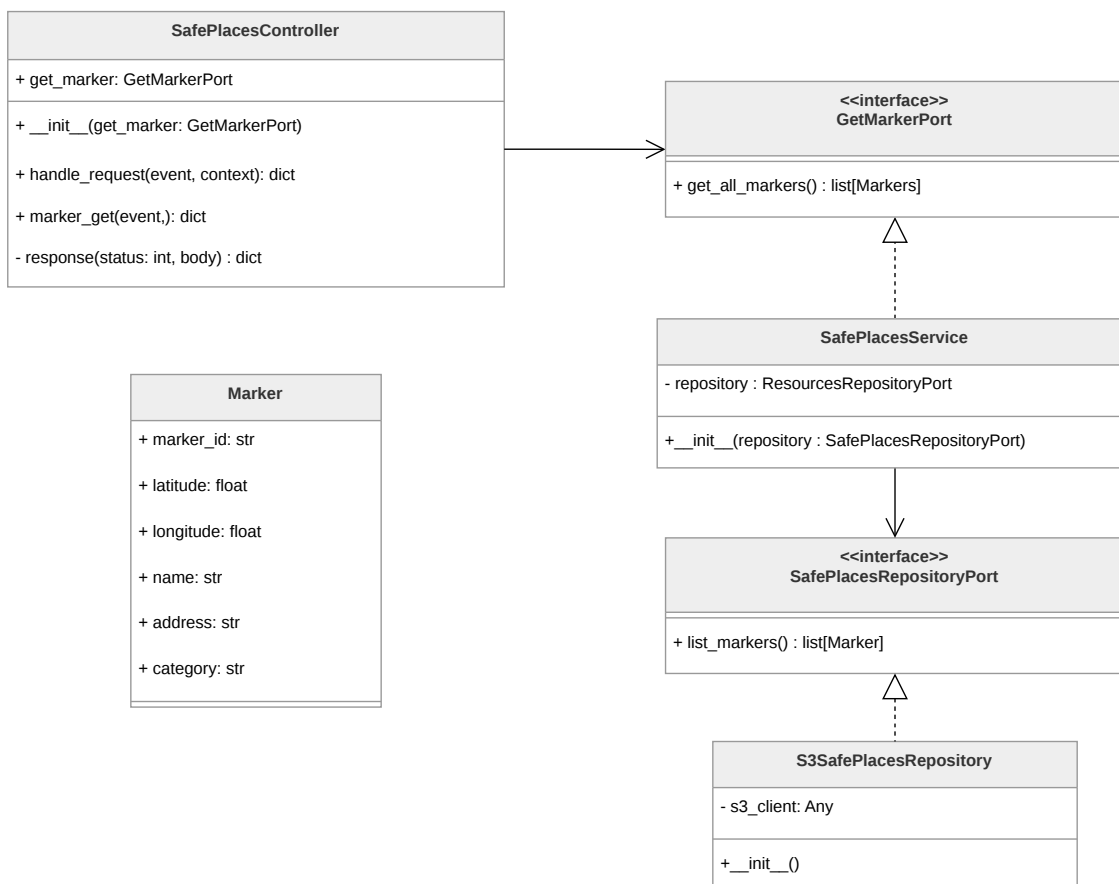


Figura 126: Componenti del microservizio luoghi sicuri



3.5.2.3.1 Marker

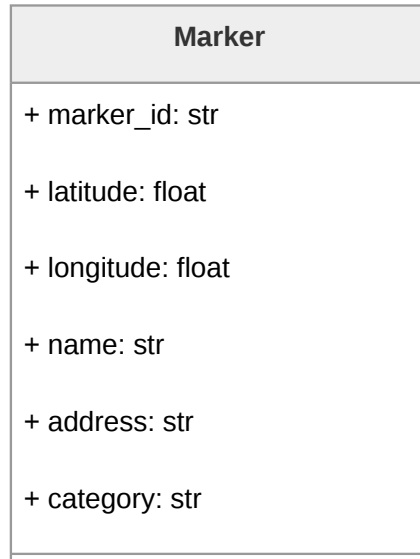


Figura 127: Diagramma della classe **Marker**

Rappresenta un punto geografico sicuro visualizzabile sulla mappa nel frontend dell'applicazione. Descrizione attributi:

- **marker_id**, che identifica univocamente il marker;
- **latitude** e **longitude** definiscono la posizione geografica;
- **name** indica il nome del marker;
- **address** indica l'indirizzo;
- **category** indica la categoria del marker;



3.5.2.3.2 SafePlacesController

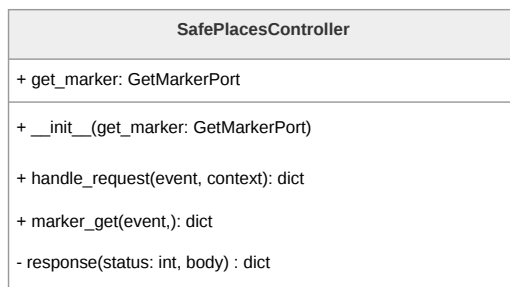


Figura 128: Diagramma della classe **SafePlacesController**

Controller del pattern esagonale. Riceve la richiesta grezza da API Gateway, la instrada verso la inbound port e costruisce il body per la risposta HTTP da restituire al client. Descrizione metodi:

- **marker_get(event)** esegue il parsing della richiesta e chiama la funzione appropriata della specifica port primaria;
- **handle_request(event)** chiama la funzione appropriata del controller in base alla route della richiesta;
- **response(status, body)** costruisce il dizionario di risposta nel formato atteso da API Gateway.

3.5.2.3.3 GetMarkerPort

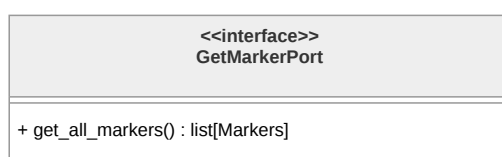


Figura 129: Diagramma dell'interfaccia **GetMarkerPort**

È la port primaria del pattern esagonale.

Il metodo **get_all_markers()** restituisce la lista completa dei marker disponibili senza esporre dettagli implementativi su come vengono recuperati.



3.5.2.3.4 SafePlacesService

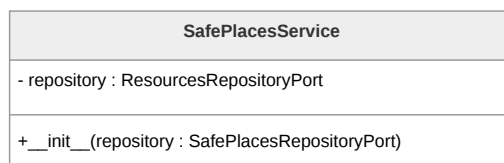


Figura 130: Diagramma della classe **SafePlacesService**

Corrisponde alla domain logic del microservizio. Contiene la logica per il recupero dei luoghi sicuri e orchestra la comunicazione con il layer di persistenza tramite la outbound port **SafePlacesRepositoryPort**.

3.5.2.3.5 SafePlacesRepositoryPort

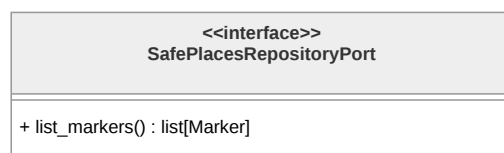


Figura 131: Diagramma dell'interfaccia **SafePlacesRepositoryPort**

Corrisponde alla outbound port del pattern esagonale. Definisce il metodo per il fetch della lista di tutti i marker disponibili, senza esporre dettagli implementativi di S3 o qualsiasi altro servizio sottostante.

3.5.2.3.6 S3SafePlacesRepository



Figura 132: Diagramma della classe **S3SafePlacesRepository**

Corrisponde all'aoutbound che implementa **SafePlacesRepositoryPort**. Contiene la logica per il fetch dei marker dall'istanza del bucket S3. Il costruttore **__init__()** inizializza il client S3.



3.5.2.3.7 Materiale informativo

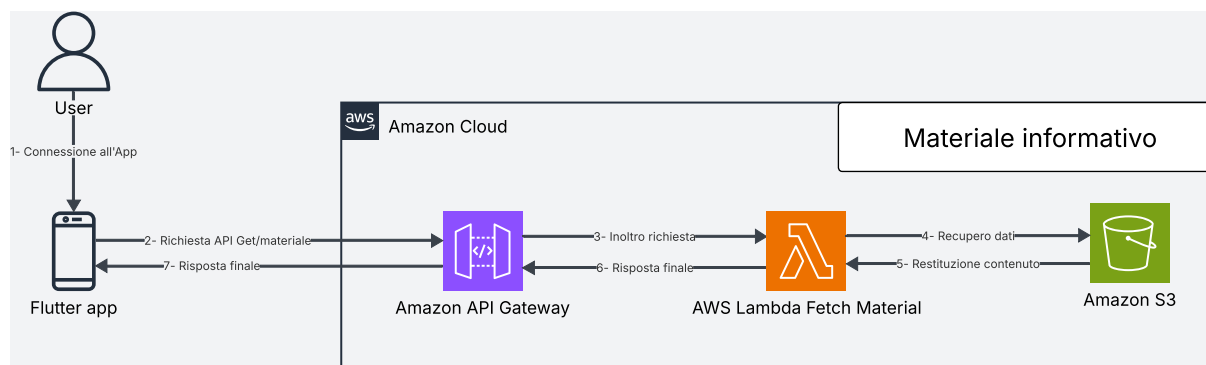


Figura 133: **Architettura^G** moduli Materiale Informativo

3.5.2.4 Architettura e Flusso di Elaborazione: Modulo Materiale Informativo

La sezione dedicata al "Materiale Informativo" è stata progettata privilegiando la massima efficienza e l'ottimizzazione dei costi operativi. Trattandosi di un modulo a prevalenza di lettura (Read-Heavy) e privo di logiche di business complesse per la manipolazione dei dati, l'**Architettura^G** prevede una funzione Lambda molto leggera, che si interfaccia con S3 esclusivamente per il fetch dei dati.

Il ciclo di elaborazione si innesca quando l'applicazione client necessita di recuperare il catalogo delle informazioni, generando una richiesta HTTPS (metodo **GET**) verso l'endpoint dedicato. L'API Gateway intercetta la chiamata e la inoltra alla funzione AWS Lambda responsabile del modulo.

La Lambda si occupa quindi di:

- recuperare i record relativi ai materiali informativi dal bucket S3;
- decodificare il contenuto per validarne l'integrità strutturale;
- serializzare il risultato aggiungendo gli header CORS necessari per renderlo consumabile dal frontend.

Il risultato elaborato viene restituito all'API Gateway, che lo inoltra al client Flutter tramite una risposta HTTP. Il ciclo sincrono si conclude con la consegna del catalogo informativo all'applicazione.

3.5.2.5 API associate

- **GET /materials/**
 - **Descrizione:** Recupera la lista delle risorse informative disponibili, tra cui riferimenti normativi, comunità di supporto e articoli informativi relativi alla violenza domestica e di genere.



– **Parametri:**

- * Nessun parametro richiesto.

– **Response:**

- * 200 OK: richiesta completata con successo.

- * Corpo della risposta:

```
[
  {
    "resource_id": "resource-123",
    "title": "Centro Antiviolenza Roma",
    "content": "Servizio di supporto per vittime.",
    "url": "https://www.centroantiviolenza.it/",
    "type": "COMMUNITY"
  },
  {
    "resource_id": "resource-456",
    "title": "Legge tutela vittime",
    "content": "Normativa nazionale violenza domestica.",
    "url": null,
    "type": "LAW"
  },
  {
    "resource_id": "resource-789",
    "title": "Riconoscere i segnali di abuso",
    "content": "Guida informativa.",
    "url": null,
    "type": "ARTICLE"
  }
]
```

– **Errori:**

- * 500 Internal Server Error: errore interno del server.

Nota di sicurezza: In maniera analoga a quanto previsto per i luoghi sicuri, anche per l'accesso al materiale informativo si è optato per non implementare l'Authorizer Cognito. Questa decisione è dettata dalla volontà di snellire l'**Architettura^G**, riducendo le complessità superflue ed evitando di incorrere nei costi operativi aggiuntivi dell'infrastruttura di autenticazione.



3.5.2.6 Architettura logica: Modulo Materiale Informativo

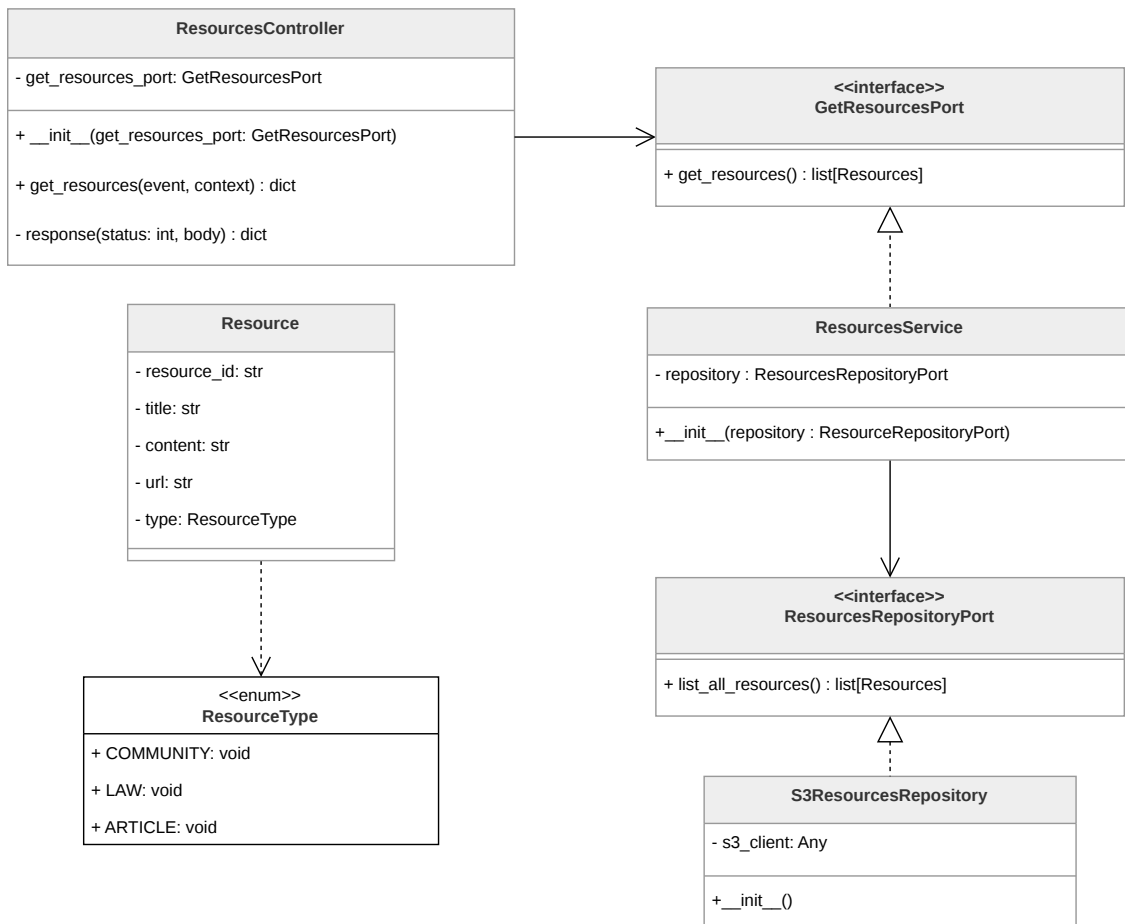


Figura 134: Componenti del microservizio materiale informativo



3.5.2.6.1 Resource

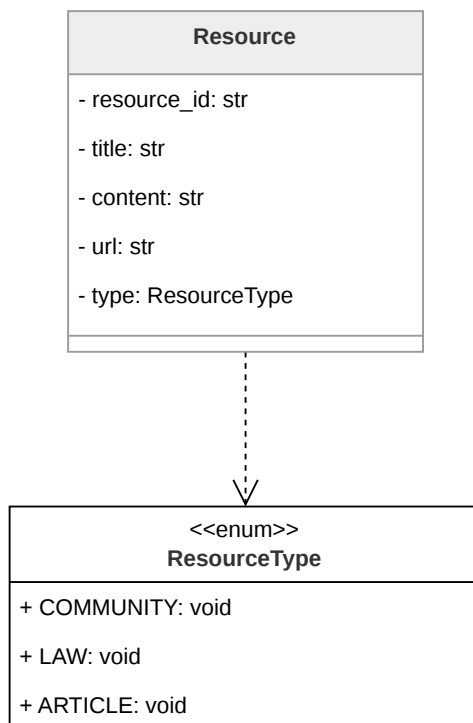


Figura 135: Diagramma della classe **Resource**

Rappresenta una risorsa informativa disponibile nell'applicazione.

Descrizione attributi:

- **resource_id**, identificativo della singola risorsa;
- **title**, nome della singola risorsa;
- **content**, testo descrittivo della singola risorsa;
- **url**, collegamento alla fonte esterna;
- **type**, corrisponde al tipo **ResourceType** e categorizza la risorsa secondo una delle tipologie previste dal dominio.

L'enumerazione **ResourceType** definisce le tre categorie disponibili: **COMMUNITY**, **LAW** e **ARTICLE**.



3.5.2.6.2 ResourcesController

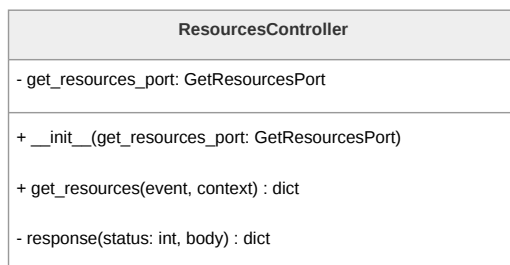


Figura 136: Diagramma della classe **ResourcesController**

Corrisponde al controller del pattern esagonale. Riceve la richiesta grezza di API Gateway, la instrada verso la inbound port e costruisce il body della risposta HTTP da restituire al client.

Descrizione metodi:

- **get_resources(event)**, costituisce il punto di ingresso del controller e riceve i parametri nativi della Lambda;
- **response(status, body)**, metodo di utilità che costruisce il dizionario di risposta nel formato atteso da API Gateway.

3.5.2.6.3 GetResourcesPort

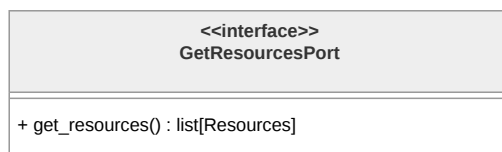


Figura 137: Diagramma della classe **GetResourcesPort**

È la inbound port del pattern esagonale. Descrive l'interfaccia per il fetch della lista completa delle risorse informative disponibili.

3.5.2.6.4 ResourcesService

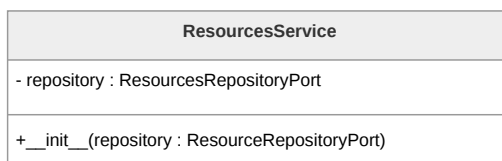


Figura 138: Diagramma della classe **ResourcesService**



Costituisce la domain logic del microservizio, implementa la inbound port **GetResourcesPort**. Contiene la logica per il recupero del materiale informativo e orchestra la comunicazione con il persistence layer tramite la outbound port **ResourcesRepositoryPort**.

3.5.2.6.5 ResourcesRepositoryPort

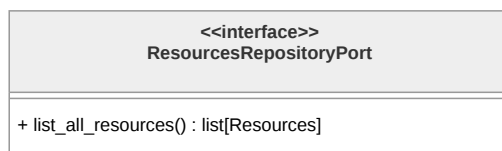


Figura 139: Diagramma della classe **ResourcesRepositoryPort**

Corrisponde all'outbound port del pattern esagonale. Definisce l'interfaccia per la restituzione della lista completa delle risorse disponibili, senza esporre dettagli implementativi relativi al servizio S3.

3.5.2.6.6 S3ResourcesRepository

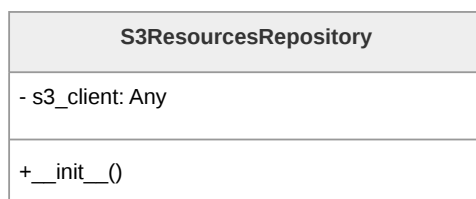


Figura 140: Diagramma della classe **S3ResourcesRepository**

Corrisponde all'outbound adapter, implementa **ResourcesRepositoryPort**. Contiene tutta la logica specifica per il recupero delle risorse informative da un bucket S3.



3.5.3 Diario criptato e fittizio

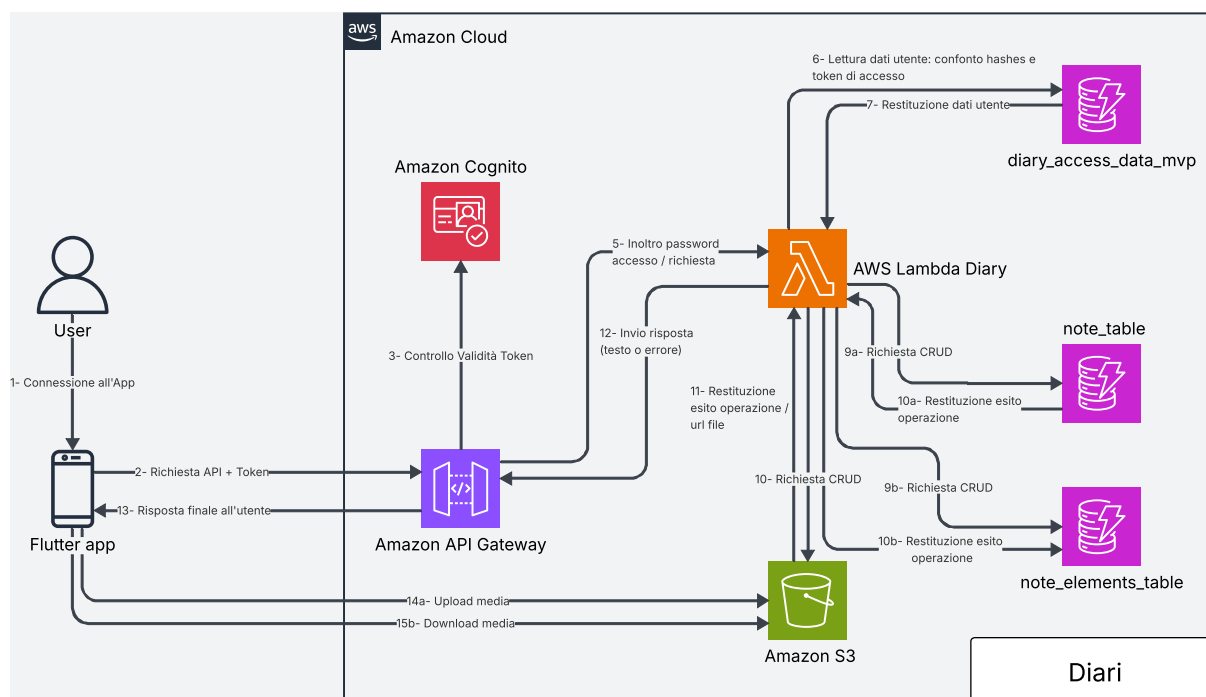


Figura 141: Architettura^G diari

3.5.3.1 Architettura e Flusso di Elaborazione: diario criptato e fittizio

La sezione dedicata al diario criptato e fittizio è progettata per garantire la sicurezza delle informazioni dell'**Utente^G** e la corretta gestione e archiviazione di diverse tipologie di dati, quali contenuti testuali, immagini e tracce audio. A differenza delle altre funzionalità, il diario è l'unica componente protetta da una password aggiuntiva, distinta da quella utilizzata per l'account principale.

Questo approccio consente all'applicazione di fornire accesso a risorse differenti in base alla password inserita, dato che i due diari sono protetti da credenziali separate. Poiché Amazon Cognito non supporta direttamente flussi di autenticazione di questo tipo, la gestione dell'accesso al diario avviene tramite la memorizzazione in DynamoDB degli hash delle due password. Quando l'**Utente^G** tenta l'accesso, viene generato l'hash della password inserita tramite HMAC-SHA256. Questo hash viene quindi confrontato con quello salvato nella tabella utenti. In caso di corrispondenza, l'accesso al diario viene garantito generando un token di accesso casuale, anch'esso salvato in DynamoDB. Tale token, comunicato al client in seguito al login, viene riportato nell'header di tutte le richieste a dati sensibili presenti nel diario.

L'archiviazione delle note avviene su due servizi AWS distinti. I contenuti testuali sono memorizzati in DynamoDB, che mantiene le informazioni per ciascun **Utente^G** e per ciascun diario. I file binari, come immagini e tracce audio, sono invece salvati in bucket



S3. Il recupero di tali risorse avviene tramite URL *presigned*, al fine di ridurre la quantità di dati trasferiti sulla rete. Poiché ogni nota è costituita da tanti blocchi informativi, le informazioni relative a tali blocchi sono memorizzate in una tabella DynamoDB separata rispetto a quella che tiene traccia dei metadati della nota che li contiene. Ciò garantisce maggiore modularità e rende più efficienti le operazioni di aggiornamento.

Il flusso di elaborazione è dunque il seguente:

- **Invio della Richiesta di Autenticazione:** L'Utente^G inserisce la password del diario tramite app. Il client genera una richiesta HTTPS (metodo POST /diary/auth) includendo la password nel body della richiesta.
- **verifica delle Credenziali:** API Gateway verifica che l'Utente^G sia autenticato all'applicazione. Dopodiché inoltra la richiesta alla funzione Lambda, che calcola l'hash della password ricevuta e lo confronta con gli hash memorizzati in DynamoDB. In caso di corrispondenza, viene identificato il tipo di diario associato.
- **Generazione token di accesso:** dopo la validazione della password, un token di accesso casuale viene generato, salvato su DynamoDB e passato in risposta al client. Tale token dovrà essere fornito nell'header di ogni richiesta successiva relativa a operazioni CRUD sui contenuti del diario.
- **Recupero delle Note:** Quando l'Utente^G richiede di recuperare la lista di note di uno dei diari (metodo GET /notes), API Gateway invia la richiesta alla Lambda, che recupera le *preview* dal database DynamoDB e le restituisce al client.
- **Recupero di una Nota:** Per ottenere il contenuto completo di una nota (metodo GET /notes/{note_id}), la Lambda recupera i dati da DynamoDB e gli eventuali riferimenti a file su S3, costruendo l'oggetto finale che viene restituito al client.
- **Gestione dei Contenuti Binari:** In presenza di immagini o tracce audio, la Lambda genera URL *presigned* per gli oggetti memorizzati su S3, permettendo al client di accedere direttamente alle risorse senza transitare attraverso il Backend^G.
- **Operazioni di Scrittura:** Per la creazione o modifica di note, la Lambda memorizza in DynamoDB i metadati principali e la struttura complessiva della nota, salvando eventuali file binari su S3.
- **Eliminazione delle Note:** In caso di richiesta di eliminazione (metodo DELETE /notes/{note_id}), la Lambda rimuove i dati associati dal database e, se presenti, elimina anche i file binari corrispondenti dai bucket S3.
- **Aggiunta elemento ad una Not:** In caso di richiesta di aggiunta di un elemento, che sia composto da un'immagine, un'audio o una stringa di testo, la Lambda crea una entry su DynamoDB per la memorizzazione dei metadati dell'elemento e tramite



il client di S3 crea un *URL presigned* per l'upload del file binario direttamente dal client.

- **Eliminazione elemento da una Nota:** In caso di richiesta di eliminazione di un'elemento di una nota, la Lambda rimuove i metadati da DynamoDB e, se presente, il file binario associato all'elemento presente nel bucket S3.
- **Logout:** quando l'**Utente^G** esce dal diario, il **Backend^G** viene notificato e la Lambda elimina la sessione precedentemente avviata.

3.5.3.2 API associate

Le API relative al diario criptato e fittizio sono suddivise in due categorie distinte:

- **API di autenticazione:** gestiscono login, logout, impostazione password e generazione e convalida dei token di accesso.
- **API per le operazioni CRUD:** gestiscono il recupero, la creazione, la modifica e l'eliminazione delle note. Le richieste agli endpoint che gestiscono queste operazioni devono avvenire specificando, nell'header della richiesta, il token di sessione ricevuto dopo il login al diario.
- **POST /diary/auth/login**
 - **Descrizione:** Autentica l'**Utente^G** rispetto a una tipologia di diario e restituisce un token di accesso se la password è corretta.
 - **Parametri:**
 - * **password:** password del diario.
 - **Response:**
 - * 200 OK: autenticazione completata con successo.
 - * Corpo della risposta:

```
{
  "diary_type": "REAL_DIARY",
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9"
```
 - **Errori:**
 - * 400 Bad Request: JSON malformato o campo **password** mancante.
 - * 500 Internal Server Error: Errore interno al server.
- **POST /diary/auth/logout**
 - **Descrizione:** Termina la sessione dell'**Utente^G** autenticato.
 - **Parametri:**



- * Nessuno.
- **Response:**
 - * 200 OK: logout completato con successo.
 - * Corpo della risposta: vuoto
- **Errori:**
 - * 400 Bad Request: **Utente^G** non trovato.
 - * 500 Internal Server Error: Errore interno al server.
- **POST /diary/auth/set_password**
 - **Descrizione:** Imposta o aggiorna la password di un diario per l'**Utente^G**, distinguendo tra diario reale e fittizio.
 - **Parametri:**
 - * **password:** nuova password.
 - * **previous_password:** password precedente (obbligatoria dopo la prima impostazione).
 - * **diary_type:** tipo di diario (REAL_DIARY o FAKE_DIARY).
 - **Response:**
 - * 200 OK: password impostata correttamente.
 - * Corpo della risposta:

```
{
  "message": "Password set successfully"
}
```
 - **Errori:**
 - * 400 Bad Request: **diary_type** non valido, password non conforme ai requisiti di sicurezza, password mancante o precedente errata, JSON malformato o campi mancanti.
 - * 500 Internal Server Error: errore di persistenza della password.
- **POST /diary/auth/status**
 - **Descrizione:** Restituisce lo stato di configurazione delle password dei diari dell'**Utente^G** autenticato.
 - **Parametri:**
 - * Nessuno.
 - **Response:**
 - * 200 OK: richiesta completata con successo.



```
* Corpo della risposta:
{
  "has_real_password": true,
  "has_fake_password": false
}
```

– **Errori:**

- * 400 Bad Request: JSON malformato o **Utente^G** non trovato.
- * 500 Internal Server Error: errore interno al server.

● **PUT /notes**

– **Descrizione:** Crea una nuova nota associata all'**Utente^G**, composta da metadati e una lista di elementi.

– **Parametri:**

- * **user_id:** identificativo dell'**Utente^G**;
- * **title:** titolo della nota;
- * **created_at:** timestamp di creazione;
- * **last_modified_at:** timestamp dell'ultima modifica;
- * **diary_type:** tipo di diario (REAL_DIARY o FAKE_DIARY);
- * **elements:** lista di elementi della nota:
 - **type:** tipo dell'elemento (text, image, audio);
 - **content:** obbligatorio solo nel caso venga inserito del testo, identifica il contenuto testuale inserito dall'**Utente^G**.

– **Response:**

```
* 200 OK: operazione completata con successo.
* Corpo della risposta:
{
  "note_id": "01KPX72J3Y5MFM9982J3TFTH7H",
  "user_id": "user1",
  "title": "Titolo",
  "created_at": "2026-02-22T15:22:27.615449+00:00",
  "last_modified_at": "2026-04-22T18:13:05.452786+00:00",
  "diary_type": "REAL_DIARY"
  "note_elements": [
    {
      "note_id": "01KPX72J3Y5MFM9982J3TFTH7H",
      "note_element_id": "01KQM5R2QQQ3GTV7026YGY7PD1",
      "type": "text",
```



```

        "content": "testo"
    },
    {
        "note_id": "01KPX72J3Y5MFM9982J3TFTH7H",
        "note_element_id": "01KQM5R2QRHD5Q38YEWSXNFJ2P",
        "type": "image",
        "content": "key_s3",
        "upload_url": "https://presigned-url"
    }
]
}

```

– **Errori:**

- * 400 Bad Request: Invalid diary_type (ValueError su enum DiaryType).
- * 500 Internal Server Error: errori di `service.add_note`, parsing JSON non valido, campi mancanti o errori runtime non gestiti.

• **GET /notes/{diary_type}**

– **Descrizione:** Recupera la lista delle note dell'**Utente^G** filtrate per tipo diario.

– **Parametri:**

- * `user_id`: identificativo dell'**Utente^G**;
- * `diary_type`: tipo di diario.

– **Response:**

- * 200 OK: liste delle note recuperata con successo.
- * Corpo della risposta:

```

{
    "notes": [
        {
            "note_id": "01KPX72J3Y5MFM9982J3TFTH7H",
            "user_id": "user1",
            "title": "Titolo",
            "created_at": "2026-02-22T15:22:27.615449+00:00",
            "last_modified_at": "2026-04-22T18:13:05.452786+00:00",
            "diary_type": "REAL_DIARY"
        },
        {
            "note_id": "01KQM5R2QQQ3GTV7026YGY7PD1",
            "user_id": "user1",

```



```

        "title": "Titolo",
        "created_at": "2026-02-22T15:22:27.615449+00:00",
        "last_modified_at": "2026-04-22T18:13:05.452786+00:00",
        "diary_type": "REAL_DIARY"
    }
]
}

```

– **Errori:**

- * 400 Bad Request: Invalid diary_type (ValueError su enum DiaryType).
- * 500 Internal Server Error: errore di service.list_notes.

• **GET /notes/{diary_type}/{note_id}**

– **Descrizione:** una nota completa con i suoi elementi recuperata con successo.

– **Parametri:**

- * user_id: identificativo dell'Utente^G;
- *
- * note_id: identificativo della nota;
- * diary_type: tipo di diario.

– **Response:**

- * 200 OK: : nota recuperata con successo.
- * Corpo della risposta:

```

{
    "note_id": "01KPX72J3Y5MFM9982J3TFTH7H",
    "user_id": "user1",
    "title": "Titolo",
    "created_at": "2026-02-22T15:22:27.615449+00:00",
    "last_modified_at": "2026-04-22T18:13:05.452786+00:00",
    "diary_type": "REAL_DIARY"
    "note_elements": [
        {
            "note_id": "01KPX72J3Y5MFM9982J3TFTH7H",
            "note_element_id": "01KQM5R2QQQ3GTV7026YGY7PD1",
            "type": "text",
            "content": "testo"
        },
        {
            "note_id": "01KPX72J3Y5MFM9982J3TFTH7H",

```



```

        "note_element_id": "01KQM5R2QRHD5Q38YEWSXNFJ2P",
        "type": "image",
        "content": "key_s3",
        "download_url": "https://presigned-url"
    }
]
}

```

– **Errori:**

- * 400 Bad Request: diary_type non valido.
- * 404 Not Found: nota non trovata.
- * 500 Internal Server Error: errore interno del server.

• **DELETE /notes/{diary_type}/{note_id}**

– **Descrizione:** Elimina una nota e di tutti i suoi elementi che la compongono.

– **Parametri:**

- * note_id: identificativo della nota;
- * user_id: identificativo dell'Utente^G;
- * diary_type: tipo di diario.

– **Response:**

- * 200 OK: eliminazione completata con successo.
- * Corpo della risposta:


```

{
    "message": "Note deleted successfully"
}

```

– **Errori:**

- * 400 Bad Request: campi obbligatori mancanti.
- * 404 Not Found: nota non trovata.
- * 500 Internal Server Error: errore interno del server.

• **PUT /notes/note_element**

– **Descrizione:** Aggiunge un elemento ad una nota esistente.

– **Parametri:**

- * note_id: identificativo della nota a cui si vuole aggiungere l'elemento;
- * type: tipo del contenuto inserito (text, image, audio)
- * content: obbligatorio solo nel caso venga inserito del testo, identifica il contenuto testuale inserito dall'Utente^G.



– **Response:**

- * 200 OK: elemento aggiunto con successo alla nota.
- * Corpo della risposta:

```
{
  "note_id": "01KPX72J3Y5MFM9982J3TFTH7H",
  "note_element_id": "01KQM5R2QRHD5Q38YEWSXNFJ2P",
  "type": "image",
  "content": "key_s3",
  "upload_url": "https://presigned-url"
}
```

– **Errori:**

- * 400 Bad Request: campi mancanti o incoerenti.
- * 500 Internal Server Error: errore interno del server.

• **DELETE /notes/note_element/{note_id}/{note_element_id}**

– **Descrizione:** Elimina un elemento da una nota (e il file associato se presente).

– **Parametri:**

- * **note_id:** identificativo della nota a cui si vuole eliminare l'elemento;
- * **note_element_id:** identificativo dell'elemento della note che si vuole eliminare;
- * **type:** tipo del contenuto che si vuole eliminare;
- * **content:** contenuto dell'elemento da eliminare.

– **Response:**

- * 200 OK:

```
{
  "message": "Note element deleted successfully"
}
```

– **Errori:**

- * 400 Bad Request: campi obbligatori mancanti.
- * 500 Internal Server Error: errore interno del server.

3.5.3.3 Architettura logica: microservizio diario

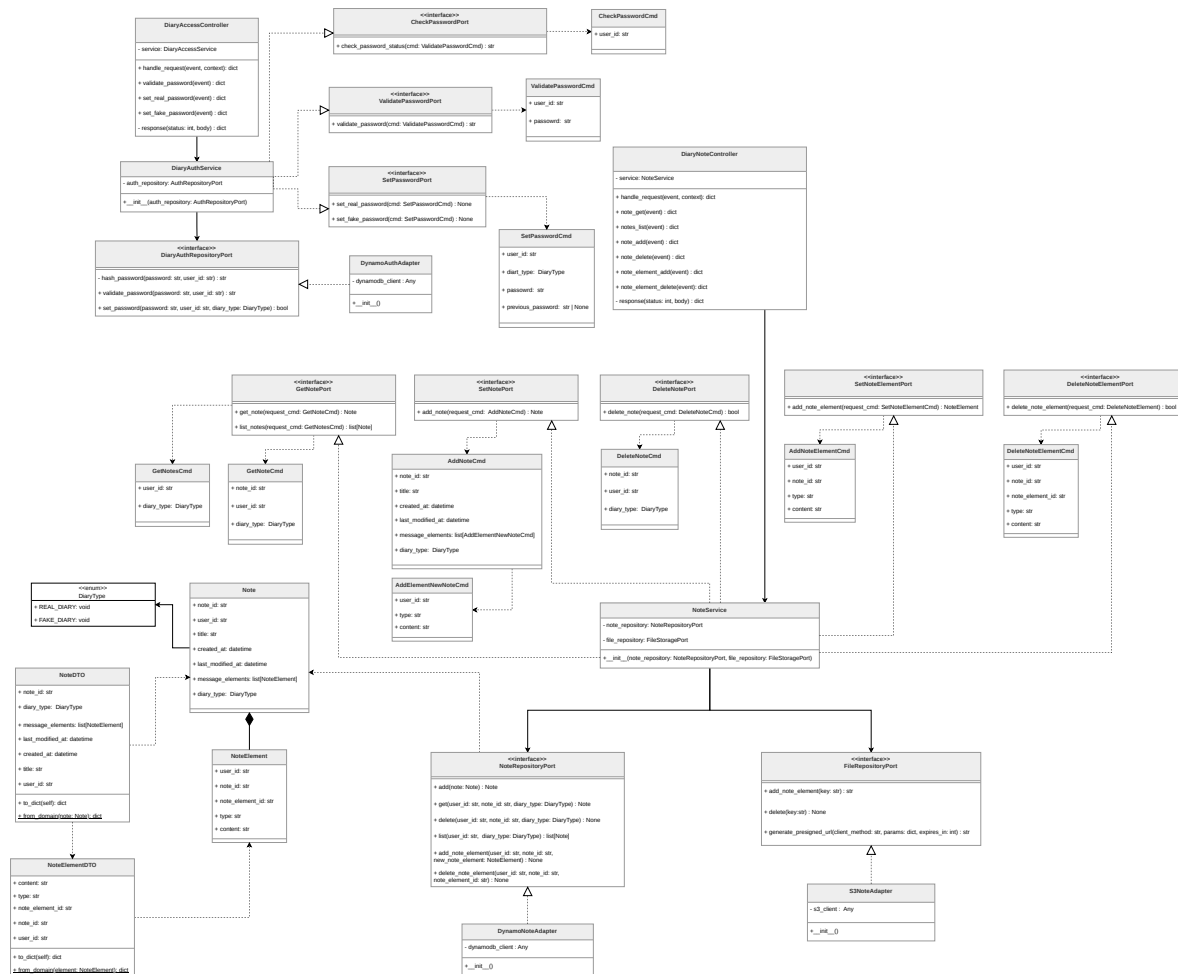


Figura 142: Componenti della Lambda per la gestione dei diari





3.5.3.3.1 DiaryType

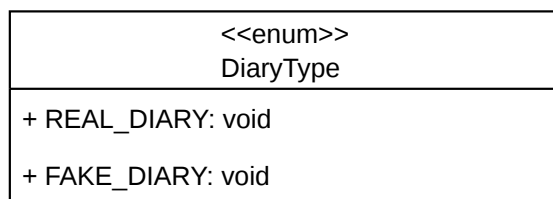


Figura 143: Diagramma dell'enumerazione **DiaryType**

L'enum **DiaryType** definisce i due tipi di diario supportati dal sistema: **REAL_DIARY** per il diario reale contenente i dati autentici dell'**Utente^G** e **FAKE_DIARY** per il diario fittizio.

3.5.3.3.2 Note

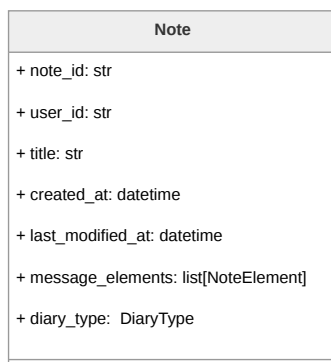


Figura 144: Diagramma della classe **Note**

La classe **Note** rappresenta l'entità di dominio principale del sistema, modellando una nota completa del diario. Include gli identificativi univoci (**note_id**, **user_id**), i metadati temporali (**created_at**, **last_modified_at**), il titolo della nota e il tipo di diario di appartenenza (**diary_type**). La nota è composta da una collezione di elementi di tipo **NoteElement**. La serializzazione della classe avviene tramite **NoteDTO**.



3.5.3.3.3 NoteElement

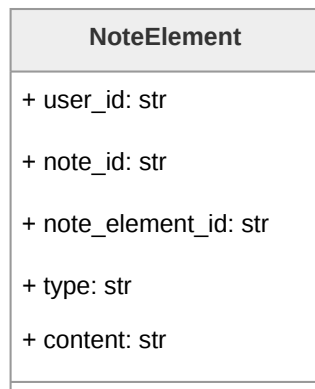


Figura 145: Diagramma della classe **NoteElement**

NoteElement modella un singolo blocco di contenuto all'interno di una nota. Ogni elemento è identificato univocamente tramite **note_element_id** e mantiene un riferimento alla nota di appartenenza (**note_id**) e all'**Utente^G** proprietario (**user_id**). Il campo **type** specifica la tipologia del contenuto (testo, immagine, audio), mentre **content** ne contiene il valore effettivo. Questa granularità permette di gestire note composite e strutturate, supportando l'aggiunta e rimozione incrementale di elementi. La classe viene serializzata tramite **NoteElementDTO**.

3.5.3.3.4 NoteElementDTO

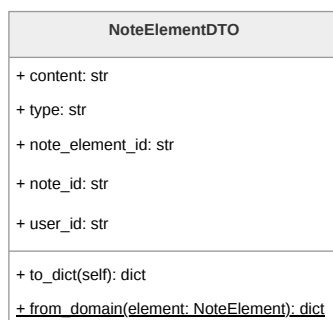


Figura 146: Diagramma della classe **NoteElementDTO**

La classe **NoteElementDTO** fornisce il mapping bidirezionale tra l'entità di dominio **NoteElement** e il formato di persistenza su DynamoDB.



3.5.3.3.5 NoteDTO

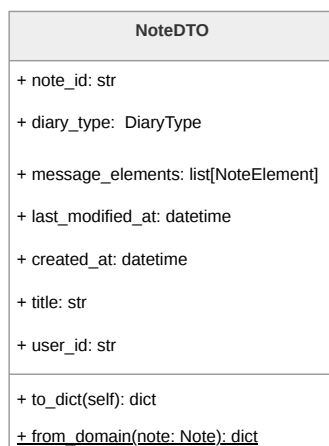


Figura 147: Diagramma della classe **NoteDTO**

La classe **NoteDTO** gestisce la conversione tra l'entità di dominio **Note** e il formato di persistenza su DynamoDB.

3.5.3.3.6 GetNoteCmd

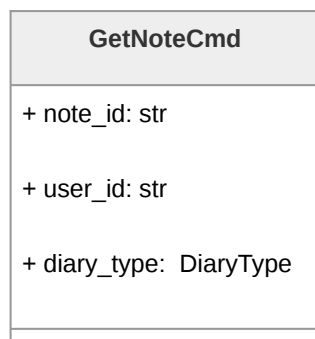


Figura 148: Diagramma della classe **GetNoteCmd**

Il command object **GetNoteCmd** incapsula i parametri necessari per richiedere il recupero di una singola nota. Contiene l'identificativo della nota (**note_id**), l'identificativo dell'**Utente^G** richiedente (**user_id**) e il tipo di diario da cui recuperare la nota (**diary_type**). Questo oggetto viene consumato dal metodo **get_note** dell'interfaccia **GetNotePort**, implementata da **NoteService**, che delega il recupero effettivo al repository di persistenza.



3.5.3.3.7 GetNotesCmd

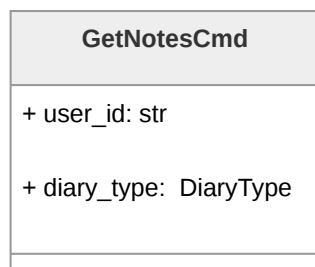


Figura 149: Diagramma della classe **GetNotesCmd**

GetNotesCmd rappresenta la richiesta dell'elenco delle note appartenenti a un **Utente^G**. Include l'identificativo dell'**Utente^G** (**user_id**) e il tipo di diario da interrogare (**diary_type**). Viene utilizzato dal metodo **list_notes** di **GetNotePort** per recuperare l'insieme completo delle note dell'**Utente^G**, mantenendo l'isolamento tra diario reale e fittizio.

3.5.3.3.8 AddElementNewNoteCmd

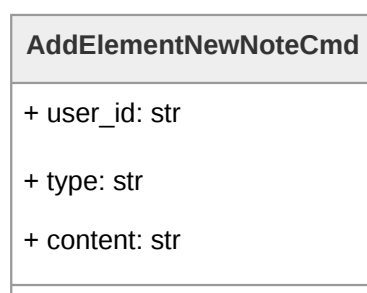


Figura 150: Diagramma della classe **AddElementNewNoteCmd**

Il command **AddElementNewNoteCmd** rappresenta la richiesta dell'aggiunta di un blocco di contenuto destinato a far parte di una nuova nota in fase di creazione. Contiene l'identificativo dell'**Utente^G** proprietario (**user_id**), la tipologia dell'elemento (**type**) e il suo contenuto (**content**). Questo oggetto viene utilizzato all'interno di **AddNoteCmd** come componente della lista **message_elements**, permettendo la creazione di note con elementi preimpostati.



3.5.3.3.9 AddNoteCmd

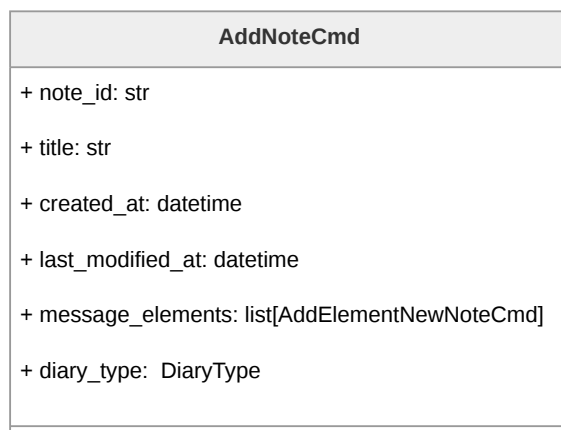


Figura 151: Diagramma della classe **AddNoteCmd**

AddNoteCmd incapsula tutti i dati necessari per la creazione di una nuova nota completa. Include l'identificativo della nota, il titolo, i timestamp di creazione e ultima modifica, il tipo di diario di destinazione e le richieste di creazione per i singoli elementi della nota. **AddNoteCmd** viene consumato dal metodo `add_note` dell'interfaccia **SetNotePort**, implementata da **NoteService**.

3.5.3.3.10 AddNoteElementCmd

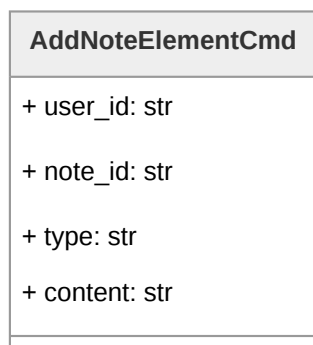


Figura 152: Diagramma della classe **AddNoteElementCmd**

Il command object **AddNoteElementCmd** rappresenta la richiesta di aggiunta di un singolo elemento a una nota esistente. Contiene l'identificativo dell'**Utente^G** e della nota target, la tipologia del nuovo elemento e il suo contenuto. Viene processato dal metodo `add_note_element` dell'interfaccia **SetNoteElementPort**, che delega a **NoteService** la creazione dell'entità **NoteElement** e la successiva persistenza tramite il repository.



3.5.3.3.11 DeleteNoteCmd

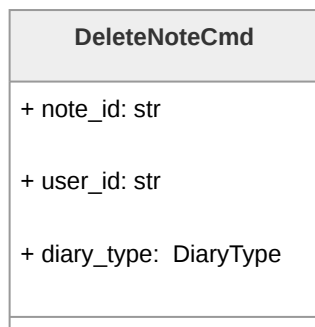


Figura 153: Diagramma della classe `DeleteNoteCmd`

`DeleteNoteCmd` incapsula i parametri per l'eliminazione di una nota. Include l'identificativo della nota da rimuovere, quello dell'**Utente^G** richiedente e il tipo di diario da cui eliminare la risorsa. Questo command viene consumato dal metodo `delete_note` di `DeleteNotePort`, che coordina sia la rimozione dal repository di persistenza che l'eventuale cleanup degli allegati file tramite `FileRepositoryPort`.

3.5.3.3.12 DeleteNoteElementCmd

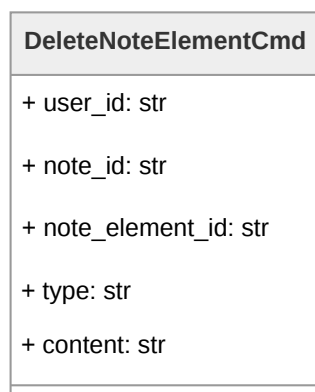


Figura 154: Diagramma della classe `DeleteNoteElementCmd`

Il command `DeleteNoteElementCmd` modella la richiesta di rimozione di un singolo elemento da una nota esistente. Contiene l'identificativo dell'**Utente^G**, della nota di riferimento e dell'elemento specifico (`note_element_id`). Viene processato dal metodo `delete_note_element` dell'interfaccia `DeleteNoteElementPort`, implementata da `NoteService`.



3.5.3.3.13 ValidatePasswordCmd

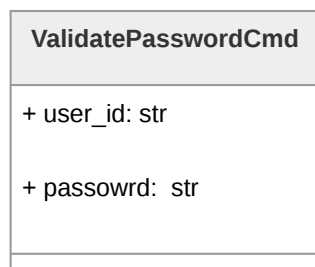


Figura 155: Diagramma della classe **ValidatePasswordCmd**

ValidatePasswordCmd rappresenta la richiesta di validazione delle credenziali di accesso al diario. Include l'identificativo dell'**Utente^G** e la password da verificare. Questo command viene consumato dal metodo `validate_password` dell'interfaccia **ValidationPort**, implementata da **DiaryAuthService**, che delega la verifica al repository di autenticazione e restituisce il tipo di diario corrispondente alla password fornita, permettendo così l'accesso selettivo al diario reale o fittizio.

3.5.3.3.14 ValidateTokenCmd

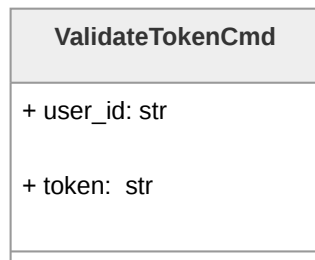


Figura 156: Diagramma della classe **ValidateTokenCmd**

ValidateTokenCmd rappresenta la richiesta di validazione del token associato alla sessione di accesso al diario corrente. Include l'identificativo dell'**Utente^G** e il token da verificare. Questo command viene consumato dal metodo `validate_token` dell'interfaccia **ValidationPort**, implementata da **DiaryAuthService**, che delega la verifica al repository di autenticazione e restituisce un booleano che indica se il token inserito corrisponde a quello presente nella tabella DynamoDB.



3.5.3.3.15 SetPasswordCmd

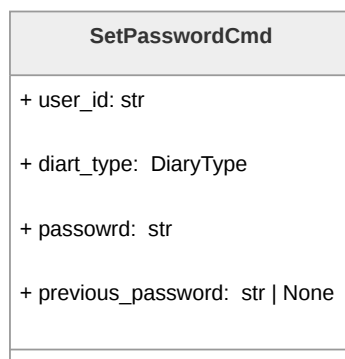


Figura 157: Diagramma della classe **SetPasswordCmd**

Il command object **SetPasswordCmd** incapsula i dati necessari per l'impostazione o l'aggiornamento delle password del diario criptato o fittizio. Include l'identificativo **Utente^G**, il tipo di diario di riferimento, la nuova password (**password**) e opzionalmente la password precedente (**previous_password**) per operazioni di modifica. Viene utilizzato dai metodi **set_real_password** e **set_fake_password** dell'interfaccia **SetPasswordPort**, permettendo la configurazione indipendente delle credenziali per ciascun tipo di diario.

3.5.3.3.16 CheckPasswordCmd

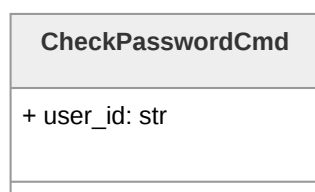


Figura 158: Diagramma della classe **CheckPasswordCmd**

CheckPasswordCmd modella la richiesta di verifica dello stato di configurazione delle password per un **Utente^G**. Contiene esclusivamente l'identificativo dell'**Utente^G** e viene consumato dal metodo **check_password_status** dell'interfaccia **CheckPasswordPort** per determinare quali credenziali di accesso sono state configurate. Ciò è necessario per permettere al frontend di mostrare una pagina di inserimento password dedicata nel caso in cui l'**Utente^G** non abbia mai impostato la password per il diario con cui vuole interagire.



3.5.3.3.17 GetNotePort

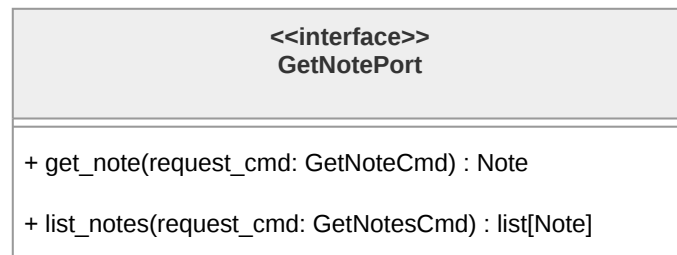


Figura 159: Diagramma dell'interfaccia **GetNotePort**

L'interfaccia **GetNotePort** definisce il contratto per le operazioni di lettura delle note. Tali operazioni di lettura includono il fetch di una singola nota e il fetch di tutte le note di un singolo **Utente^G**. **GetNotePort** viene implementata da **NoteService**, che orchestra il recupero dei dati tramite **NoteRepositoryPort**.

3.5.3.3.18 SetNotePort

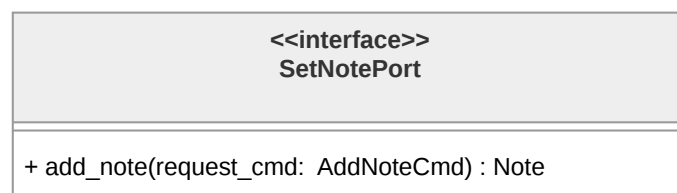


Figura 160: Diagramma dell'interfaccia **SetNotePort**

L'interfaccia **SetNotePort** definisce il contratto per la creazione di una nuova nota. È implementata da **NoteService**.

3.5.3.3.19 DeleteNotePort

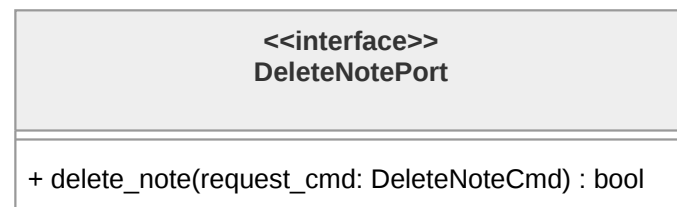


Figura 161: Diagramma dell'interfaccia **DeleteNotePort**

L'interfaccia **DeleteNotePort** definisce il contratto per l'eliminazione di un'intera nota. È implementata da **NoteService**, che orchestra sia la rimozione della nota dal repository che l'eventuale cleanup dei file allegati attraverso **FileRepositoryPort**.



3.5.3.3.20 SetNoteElementPort

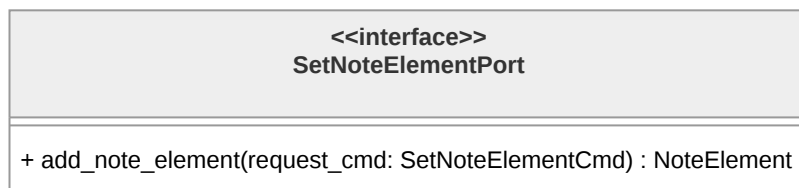


Figura 162: Diagramma dell'interfaccia **SetNoteElementPort**

L'interfaccia **SetNoteElementPort** definisce il contratto per l'aggiunta di un elemento di contenuto a una nota esistente. Il metodo **add_note_element** accetta un **AddNoteElementCmd** e restituisce l'entità **NoteElement** creata. È implementata da **NoteService**.

3.5.3.3.21 DeleteNoteElementPort



Figura 163: Diagramma dell'interfaccia **DeleteNoteElementPort**

L'interfaccia **DeleteNoteElementPort** definisce il contratto per la rimozione di un singolo elemento da una nota esistente. È implementata da **NoteService**.

3.5.3.3.22 SetPasswordPort

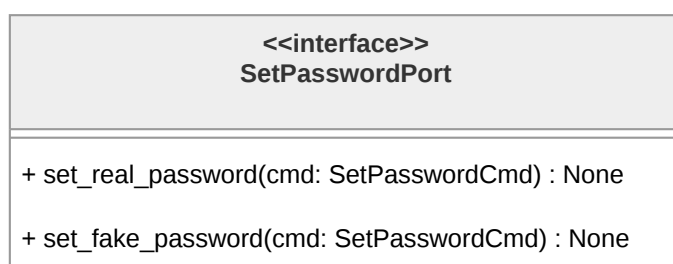


Figura 164: Diagramma dell'interfaccia **SetPasswordPort**

L'interfaccia **SetPasswordPort** definisce il contratto per la configurazione delle credenziali di accesso. Espone due metodi distinti: **set_real_password** per l'impostazione della password del diario reale e **set_fake_password** per quella del diario fittizio, entrambi accettando un **SetPasswordCmd**. È implementata da **DiaryAuthService**.



3.5.3.3.23 ValidationPort

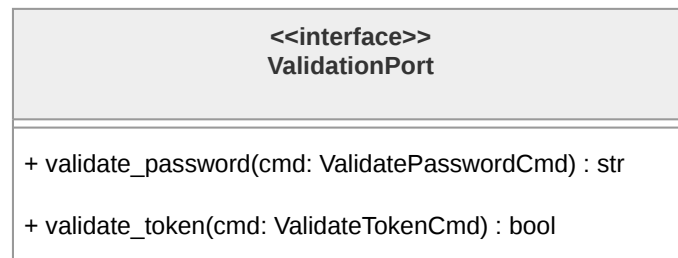


Figura 165: Diagramma dell'interfaccia **ValidationPort**

L'interfaccia **ValidationPort** definisce il contratto per la validazione delle credenziali di accesso al diario. Il metodo `validate_password` accetta un **ValidatePasswordCmd** e restituisce una stringa rappresentante il tipo di diario corrispondente alla password fornita se una corrispondenza viene effettivamente trovata. Il metodo `validate_token` accetta un **ValidateTokenCmd** e restituisce un booleano indicante la validità del token. È implementata da **DiaryAuthService**.

3.5.3.3.24 CheckPasswordPort

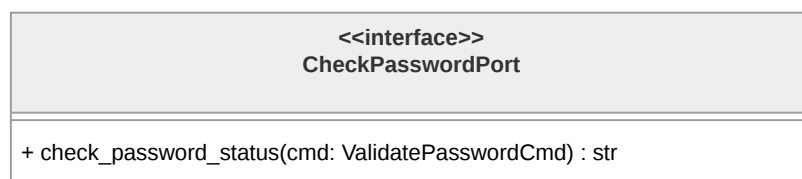


Figura 166: Diagramma dell'interfaccia **CheckPasswordPort**

L'interfaccia **CheckPasswordPort** definisce il contratto per la verifica dello stato di configurazione delle password. È implementata da **DiaryAuthService**.



3.5.3.3.25 NoteRepositoryPort

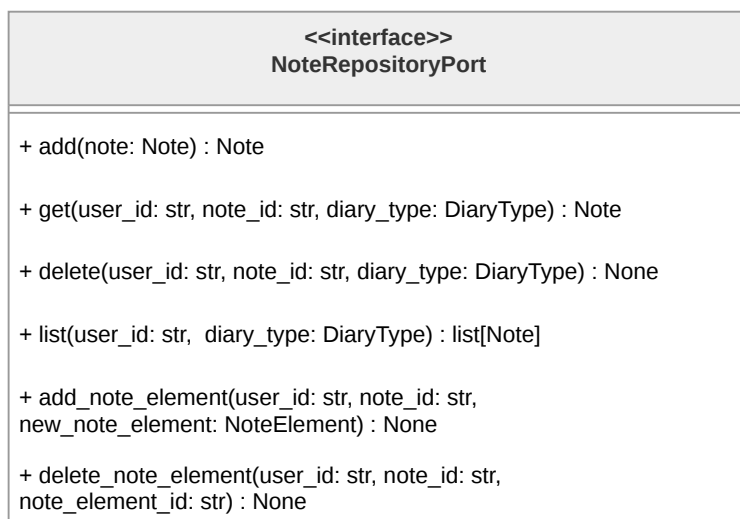


Figura 167: Diagramma dell'interfaccia **NoteRepositoryPort**

L'interfaccia **NoteRepositoryPort** definisce il contratto per la persistenza delle note e dei loro elementi. Espone i metodi **add** per la creazione di note, **get** per il recupero di una nota specifica, **delete** per la rimozione, **list** per l'elenco completo delle note di un **Utente^G**, **add_note_element** per l'inserimento di elementi in note esistenti e **delete_note_element** per la loro rimozione. Tutti i metodi accettano il parametro **diary_type** per garantire l'isolamento tra diario reale e fittizio. Questa porta di uscita viene implementata da **DynamoNoteAdapter**, che fornisce la persistenza concreta su DynamoDB.

3.5.3.3.26 FileRepositoryPort

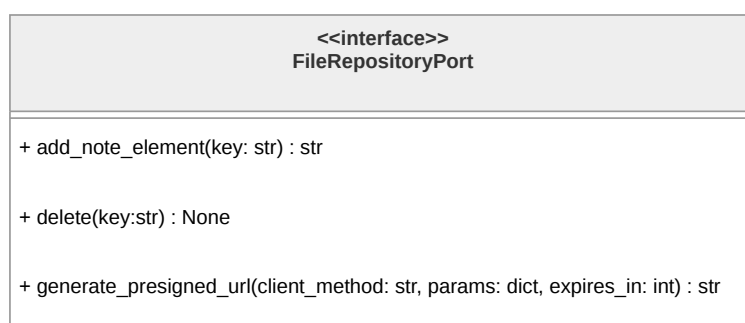


Figura 168: Diagramma dell'interfaccia **FileRepositoryPort**

Costituisce l'oubound adapter, definisce i metodi per la gestione degli file (immagini o audio) inseriti all'interno delle note del diario. Espone il metodo **add_note_element** che accetta una chiave e restituisce l'URL di accesso, **delete** per la rimozione di file tramite chiave e **generate_presigned_url** per la generazione di URL firmati con parametri e



scadenza configurabili. **FileRepositoryPort** è implementata da **S3NoteAdapter**, che fornisce l'integrazione concreta con Amazon S3.

3.5.3.3.27 DiaryAuthRepositoryPort

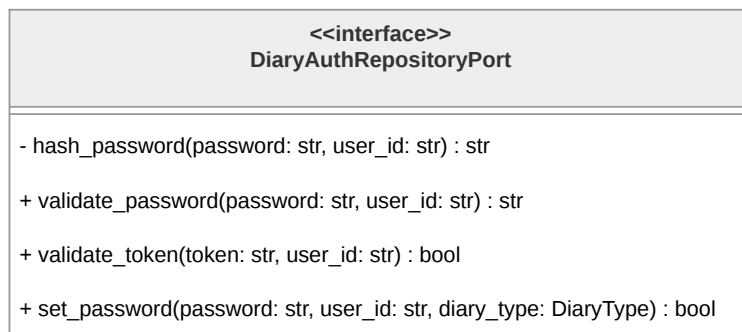


Figura 169: Diagramma dell'interfaccia **DiaryAuthRepositoryPort**

Costituisce l'outbound adapter, definisce i metodi per la persistenza e validazione delle credenziali di accesso. Espone il metodo **hash_password** per la generazione dell'hash di una password, **validate_password** per la verifica delle credenziali fornite restituendo il tipo di diario corrispondente, e **set_password** per la persistenza delle credenziali associate a uno specifico **diary_type**. Implementata da **DynamoAuthAdapter**, questa porta permette a **DiaryAuthService** di gestire l'autenticazione mantenendo separati gli hash delle password per diario reale e fittizio.

3.5.3.3.28 NoteService

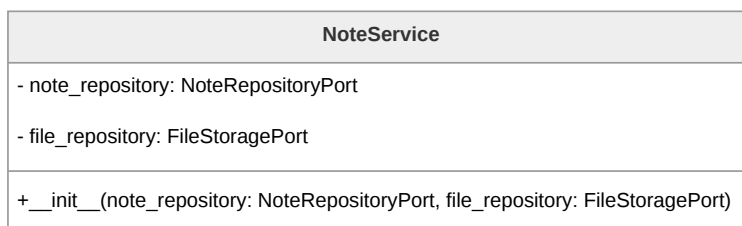


Figura 170: Diagramma della classe **NoteService**

Rappresenta la domain logic per la gestione delle note, implementando le interfacce **GetNotePort**, **SetNotePort**, **DeleteNotePort**, **SetNoteElementPort** e **DeleteNoteElementPort**. Coordina l'intera **Business Logic**^G delle note: gestisce le operazioni CRUD e la comunicazione con le porte per i servizi di persistenza. Tali porte vengono passate alla classe tramite dependency injection nel costruttore.



3.5.3.3.29 DiaryAuthService

DiaryAuthService
- auth_repository: AuthRepositoryPort
+ __init__(auth_repository: AuthRepositoryPort)

Figura 171: Diagramma della classe **DiaryAuthService**

Rappresenta la domain logic per la gestione dell'autenticazione al diario, che include l'impostazione della password di entrambi i diari e la loro verifica. Si occupa inoltre di verificare che la password impostata dall'**Utente^G** sia conforme a specifici criteri di sicurezza. **DiaryAuthService** riceve tramite costruttore la dipendenza **auth_repository** di tipo **DiaryAuthRepositoryPort** per l'orchestrazione delle operazioni relative alla persistenza dei dati di accesso degli utenti.

3.5.3.3.30 DynamoNoteAdapter

DynamoNoteAdapter
- dynamodb_client : Any
+ __init__()

Figura 172: Diagramma della classe **DynamoNoteAdapter**

Rappresenta l'outbound adapter, implementa l'interfaccia **NoteRepositoryPort** fornendo la persistenza concreta delle note su DynamoDB. È responsabile di effettuare le richieste concrete a DynamoDB e a ritornare i risultati ottenuti, implementando concretamente le varie operazioni CRUD. L'adapter utilizza **NoteDTO** e **NoteElementDTO** per la conversione bidirezionale tra le entità di dominio e il formato di persistenza su DynamoDB, isolando completamente il layer applicativo dai dettagli implementativi dello storage.



3.5.3.3.31 S3NoteAdapter

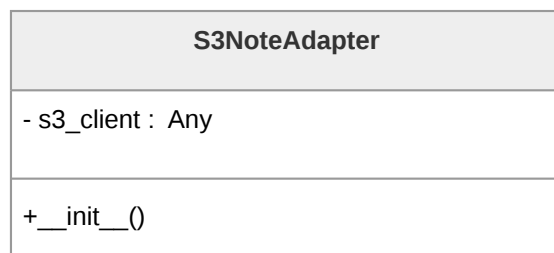


Figura 173: Diagramma della classe **S3NoteAdapter**

Rappresenta l'outbound adapter, implementa l'interfaccia **FileRepositoryPort** fornendo la gestione degli allegati file su Amazon S3. Gestisce la generazione dei `presigned_url` e le chiamate a S3 per l'archiviazione o il recupero dei file binari. Viene utilizzato da **NoteService** per gestire il ciclo di vita completo degli allegati associati alle note.

3.5.3.3.32 DynamoAuthAdapter

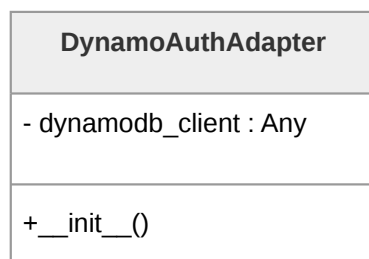


Figura 174: Diagramma della classe **DynamoAuthAdapter**

Rappresenta l'outbound adapter, implementa l'interfaccia **DiaryAuthRepositoryPort** fornendo la persistenza delle credenziali di autenticazione su DynamoDB. Mantiene un riferimento al client DynamoDB (**dynamodb_client**) inizializzato nel costruttore. Il metodo **hash_password** genera l'hash sicuro di una password per un **Utente^G** specifico tramite HMAC-SHA-256 applicato sulla concatenazione tra l'id dell'**Utente^G** e la password stessa. **validate_password** Verifica le credenziali fornite restituendo il tipo di diario corrispondente all'hash validato, e **set_password** memorizza o aggiorna le credenziali associate a uno specifico **diary_type**.



3.5.3.3.33 DiaryNoteController

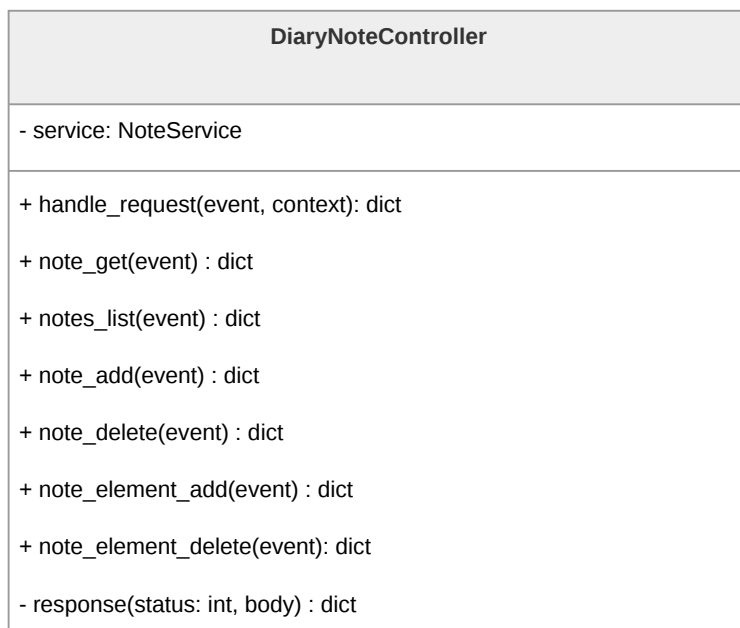


Figura 175: Diagramma della classe **DiaryNoteController**

La classe **DiaryNoteController** costituisce il punto di ingresso per le richieste relative alle operazioni sulle note. Mantiene un riferimento a **NoteService** che orchestra la **Business Logic**^G. Il metodo **handle_request** funge da router principale, interpretando la richiesta in arrivo dal Lambda handler e delegandola alle funzioni specifiche.



3.5.3.34 DiaryAccessController



Figura 176: Diagramma della classe **DiaryAccessController**

La classe **DiaryAccessController** costituisce il punto di ingresso per le operazioni di autenticazione. Mantiene un riferimento a **DiaryAuthService** che implementa la logica di gestione delle credenziali. Il metodo **handle_request** funge da router principale, interpretando la richiesta in arrivo dal Lambda handler e delegandola alle funzioni specifiche.

3.5.4 Chatbot

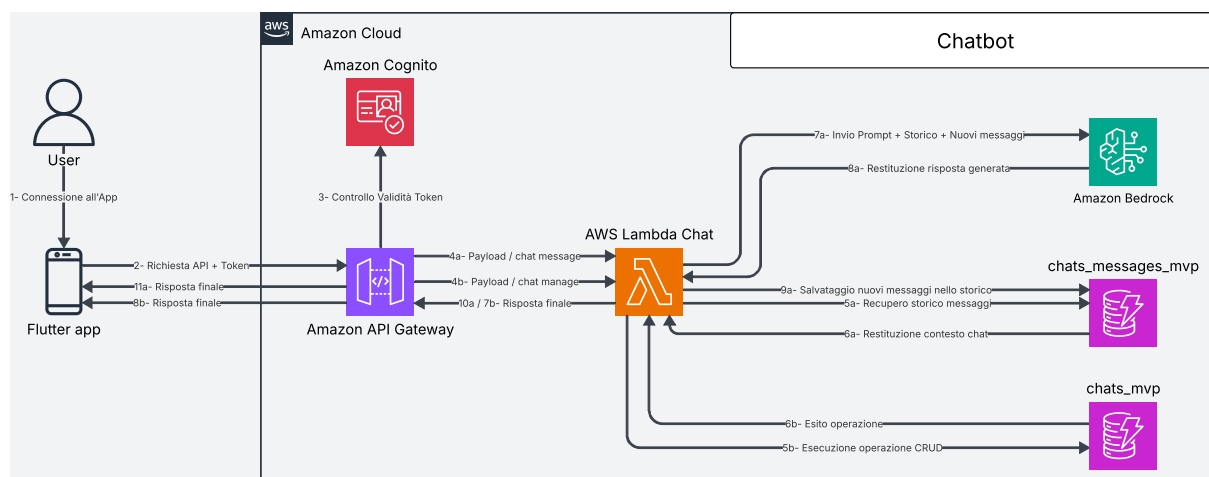


Figura 177: **Architettura^G chatbot**



3.5.4.1 Architettura e Flusso di Elaborazione: chatbot (detective delle relazioni e specchio intelligente)

L'**Architettura**^G di **Backend**^G dedicata al chatbot è progettata per gestire conversazioni persistenti tra **Utente**^G e modello linguistico, garantendo scalabilità, efficienza nell'accesso ai dati e qualità delle risposte generate.

API Gateway rappresenta il punto di ingresso per tutte le richieste client e si occupa della verifica dell'autenticazione dell'**Utente**^G. Una volta validata la richiesta, questa viene inoltrata a una funzione Lambda che implementa la logica applicativa del chatbot, inclusa la gestione delle chat e l'interazione con il modello linguistico.

Le conversazioni sono strutturate come entità persistenti (chat), ciascuna identificata da un `chat_id` univoco all'interno dell'intero sistema. Per garantire una maggiore modularità e facilitare il recupero delle informazioni, chat e messaggi sono memorizzati in tabelle DynamoDB separate. Ogni chat contiene una sequenza di messaggi (**Utente**^G e assistente artificiale), anch'essi memorizzati in DynamoDB. Per ottimizzare le prestazioni e ridurre il carico di rete, le operazioni di recupero dello storico restituiscono esclusivamente informazioni di sintesi (ad esempio titolo e metadata), mentre il contenuto completo dei messaggi viene fornito solo su richiesta esplicita.

Quando l'**Utente**^G invia un messaggio, il sistema non si limita a inoltrarlo direttamente al modello, ma recupera le informazioni più rilevanti dalla chat in corso. In particolare, la Lambda recupera il contesto rilevante della conversazione dalla base di dati (DynamoDB), costruendo un prompt arricchito che include la cronologia significativa dei messaggi. Per estrarre tali informazioni, la Lambda utilizza tecniche di ricerca lessicale basate su BM25, evitando di concatenare l'intera conversazione al testo inviato dall'**Utente**^G. Questo approccio consente di migliorare la coerenza e la pertinenza delle risposte generate dal modello.

Il prompt così costruito viene inviato a Bedrock, che restituisce la risposta generata dal LLM. La risposta, insieme al messaggio dell'**Utente**^G, viene quindi memorizzata in DynamoDB come parte della chat. Nel caso in cui il messaggio inviato sia il primo di una nuova chat, la Lambda imposta il titolo della chat alle prime tre parole del messaggio dell'**Utente**^G.

Il flusso di elaborazione è dunque il seguente:

- **Invio della Richiesta:** L'**Utente**^G interagisce con il chatbot tramite app, generando una richiesta HTTPS verso uno degli endpoint disponibili (ad esempio `POST /chats/{chat_id}/messages`).
- **Autenticazione:** API Gateway verifica che l'**Utente**^G sia autenticato all'applicazione. In caso positivo, la richiesta viene inoltrata alla funzione Lambda responsabile della logica del chatbot.



- **Gestione della Chat:** La Lambda identifica la chat di riferimento tramite `chat_id`. Se necessario, recupera i metadati o verifica l'esistenza della chat in DynamoDB.
- **Recupero del Contesto (RAG):** Prima di inviare il messaggio al modello, la Lambda esegue una query su DynamoDB per ottenere i messaggi precedenti rilevanti della conversazione, costruendo il contesto da includere nel prompt.
- **Costruzione del Prompt:** Il messaggio dell'**Utente^G** viene combinato con il contesto recuperato, formando un prompt strutturato da inviare al modello linguistico.
- **Invocazione del Modello:** La Lambda invia il prompt a Bedrock, che elabora la richiesta e restituisce una risposta generata.
- **Persistenza dei Messaggi:** La Lambda salva sia il messaggio dell'**Utente^G** sia la risposta del modello in DynamoDB, associandoli alla chat corrente.
- **Aggiornamento dei Metadati:** Se necessario, la Lambda aggiorna le informazioni della chat (ad esempio titolo o timestamp dell'ultima attività).
- **Restituzione della Risposta:** La risposta generata dal modello, insieme ai metadati aggiornati, viene restituita al client.
- **Recupero dello Storico:** Quando l'**Utente^G** richiede la lista delle chat (metodo GET `/chats/`), la Lambda restituisce solo le *preview* delle chat, senza includere l'intero contenuto dei messaggi.
- **Recupero di una Chat:** Per ottenere il contenuto completo di una chat (metodo GET `/chats/{chat_id}`), la Lambda recupera tutti i messaggi associati da DynamoDB e li restituisce al client.
- **Operazioni CRUD:** Le operazioni di creazione, aggiornamento ed eliminazione delle chat vengono gestite direttamente dalla Lambda tramite interazioni con DynamoDB.

3.5.4.2 API associate

- **GET `/chats/`**
 - **Descrizione:** Recupera la lista delle chat associate all'**Utente^G** autenticato, restituendo solo le informazioni principali, senza i messaggi.
 - **Parametri:**
 - * `user_id`: identificativo dell'**Utente^G**.



– **Response:**

* 200 OK: richiesta completata con successo.

* Corpo della risposta:

```
{
  "chats": [
    {
      "chat_id": "01KPX72J3Y5MFM9982J3TFTH7H",
      "title": "Supporto emotivo",
      "created_at": "2026-02-22T15:22:27.615449+00:00",
      "updated_at": "2026-04-22T18:13:05.452786+00:00",
      "messages": []
    },
    {
      "chat_id": "01KPX8AB4Z9QTR672L8VYH3D2C",
      "title": "Violenza domestica",
      "created_at": "2026-03-05T09:11:42.123456+00:00",
      "updated_at": "2026-04-24T16:47:19.987654+00:00",
      "messages": []
    }
  ]
}
```

– **Errori:**

* 400 Bad Request: parametri mancanti o non validi.

* 401 Unauthorized: **Utente^G** non autenticato.

* 500 Internal Server Error: errore interno del server.

• **POST /chats/**

– **Descrizione:** Crea una nuova chat associata all'**Utente^G** autenticato, iniziata senza messaggi.

– **Parametri:**

* **title:** titolo della chat.

– **Response:**

* 201 Created: chat creata con successo.

* Corpo della risposta:

```
{
  "user_id": "3f8a1c2e-7b4d-4f9a-9c21-5d7e6a8b2c90",
  "chat_id": "01KQ52JAEYD7ZDH894CN9HEH05",
  "title": "La mia nuovissima chat",
}
```



```

    "created_at": "2026-04-26T14:19:48.574747+00:00",
    "updated_at": "2026-04-26T14:19:48.574747+00:00",
    "messages": []
  }

```

– **Errori:**

- * 400 Bad Request: body mancante o non valido.
- * 500 Internal Server Error: errore interno del server.

• **GET /chats/{chat_id}**

– **Descrizione:** Recupera una chat specifica identificata da `chat_id`, includendo tutti i messaggi associati.

– **Parametri:**

- * `chat_id`: identificativo della chat.

– **Response:**

- * 200 OK: richiesta completata con successo.

- * Corpo della risposta:

```

{
  "user_id": "3f8a1c2e-7b4d-4f9a-9c21-5d7e6a8b2c90",
  "chat_id": "01KPTWJ5CZ9QM4VH8CSHV451ZV",
  "title": "La mia chat",
  "created_at": "2026-04-22T15:22:27.615449+00:00",
  "updated_at": "2026-04-25T17:26:12.748959+00:00",
  "messages": [
    {
      "chat_id": "01KPTWJ5CZ9QM4VH8CSHV451ZV",
      "message_id": "01MSG001",
      "text": "Ciao, come ti chiami?",
      "sender": "user",
      "created_at": "2026-04-25T17:25:00.000000+00:00"
    },
    {
      "chat_id": "01KPTWJ5CZ9QM4VH8CSHV451ZV",
      "message_id": "01MSG002",
      "text": "Sono Chatbot, come posso aiutarti?",
      "sender": "ai",
      "created_at": "2026-04-25T17:25:01.000000+00:00"
    }
  ]
}

```



```
}
```

– **Errori:**

- * 404 Not Found: chat non trovata.
- * 400 Bad Request: parametri non validi.
- * 500 Internal Server Error: errore interno del server.

• **PUT /chats/{chat_id}**

– **Descrizione:** Aggiorna le informazioni di una chat esistente identificata da `chat_id`.

– **Parametri:**

- * `chat_id`: identificativo della chat.
- * `title`: nuovo titolo della chat.
- * `user_id`: identificativo dell'**Utente**^G.

– **Response:**

- * 200 OK: aggiornamento completato con successo.
- * Corpo della risposta:

```
{
  "user_id": "3f8a1c2e-7b4d-4f9a-9c21-5d7e6a8b2c90",
  "chat_id": "01KPTWJ5CZ9QM4VH8CSHV451ZV",
  "title": "Nuovo titolo aggiornato",
  "created_at": "2026-04-22T15:22:27.615449+00:00",
  "updated_at": "2026-04-26T14:26:38.684047+00:00",
  "messages": [...]
}
```

– **Errori:**

- * 400 Bad Request: body non valido o parametri mancanti.
- * 404 Not Found: chat non trovata.
- * 500 Internal Server Error: errore interno del server.

• **DELETE /chats/{chat_id}**

– **Descrizione:** Elimina una chat esistente identificata da `chat_id` insieme ai relativi messaggi.

– **Parametri:**

- * `chat_id`: identificativo della chat.

– **Response:**



- * 204 No Content: eliminazione completata con successo.
- * Corpo della risposta: vuoto.
- **Errori:**
 - * 404 Not Found: chat non trovata.
 - * 400 Bad Request: parametri non validi.
 - * 500 Internal Server Error: errore interno del server.
- **POST /chats/{chat_id}/messages**
 - **Descrizione:** Inserisce un nuovo messaggio all'interno di una chat esistente e attiva la generazione della risposta da parte del modello LLM, secondo la modalità specificata.
 - **Parametri:**
 - * chat_id: identificativo della chat.
 - * message: contenuto del messaggio dell'**Utente^G**.
 - * response_mode (body, opzionale): modalità di generazione della risposta del chatbot: "mirror" o "detective".
 - **Response:**
 - * 200 OK: messaggio elaborato con successo e risposta generata.
 - * Corpo della risposta:

```

{
  "response": "questa è la risposta del chatbot",
  "input_message_id": "01KQ536RE4QX5XSCZVC42XQJ49",
  "response_message_id": "01KQ536RGM0Q86WPV8TNGZ5J1V"
}

```
 - **Errori:**
 - * 400 Bad Request: messaggio mancante o non valido.
 - * 404 Not Found: chat non trovata.
 - * 500 Internal Server Error: errore interno del servizio LLM o del **Bac-kend^G**.

3.5.4.3 Architettura logica del chatbot

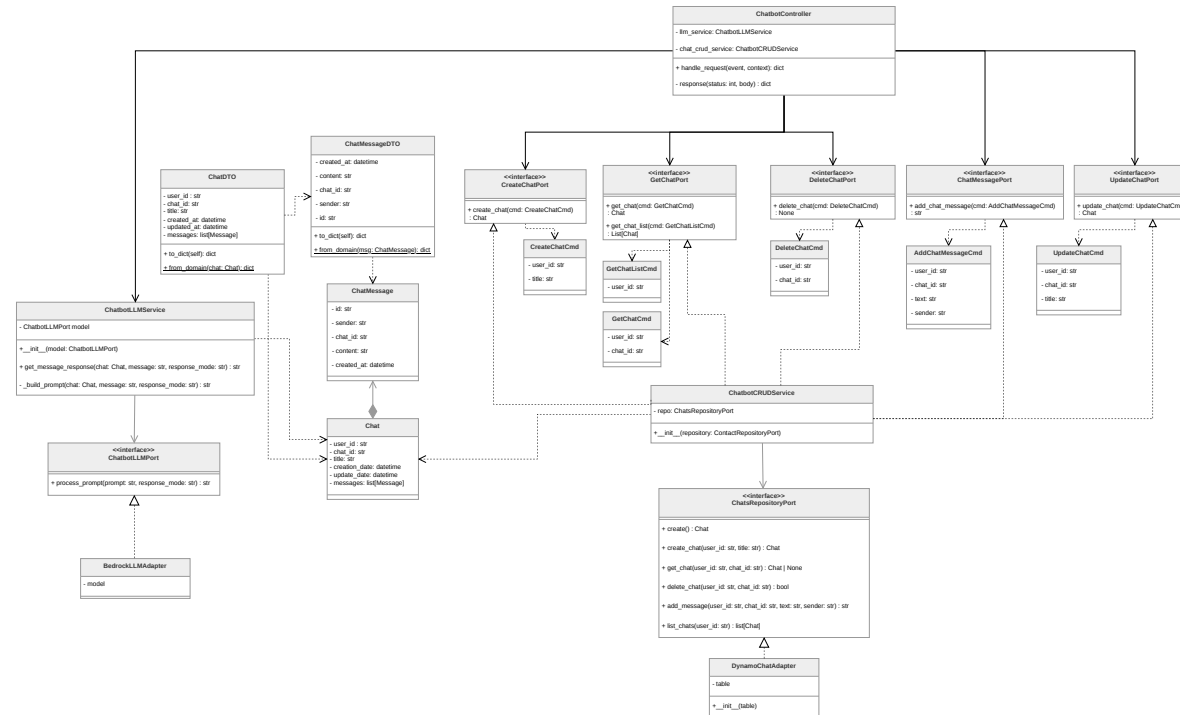


Figura 178: Componenti del microservizio Chatbot



3.5.4.3.1 Chat

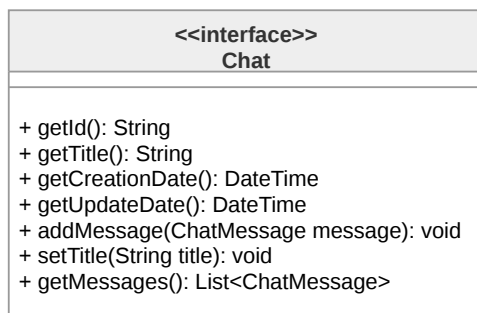


Figura 179: Diagramma della classe **Chat**

La classe **Chat** rappresenta il modello di dominio di una conversazione tra **Utente^G** e chatbot. Contiene i metadati identificativi (**user_id**, **chat_id**), il titolo della conversazione, le date di creazione e aggiornamento e l'elenco completo dei messaggi scambiati. Questa classe costituisce l'entità centrale del dominio applicativo e viene utilizzata sia dai servizi di persistenza che da quelli di elaborazione tramite LLM.

3.5.4.3.2 ChatMessage

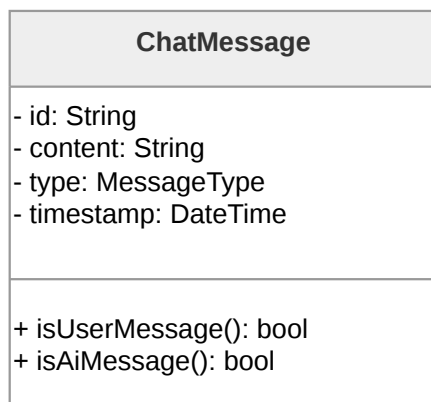


Figura 180: Diagramma della classe **ChatMessage**

ChatMessage modella un singolo messaggio all'interno di una conversazione. Include l'identificativo univoco del messaggio (**id**), il riferimento alla chat di appartenenza (**chat_id**), il contenuto testuale (**content**), il mittente (**sender**, che può assumere il valore **"user"** o **"ai"**) e la data di creazione. Questa classe permette di mantenere traccia completa dello storico conversazionale necessario per il funzionamento del chatbot.



3.5.4.3.3 ChatbotLLMPort



Figura 181: Diagramma dell'interfaccia **ChatbotLLMPort**

L'interfaccia **ChatbotLLMPort** definisce il contratto per gli adapter che si interfacciano con LLM. Espone il metodo **process_prompt**, che accetta un prompt testuale e una modalità di risposta, restituendo la stringa generata dal modello. Questa astrazione permette di disaccoppiare la logica applicativa dall'implementazione specifica del provider LLM utilizzato.

3.5.4.3.4 BedrockLLMAdapter

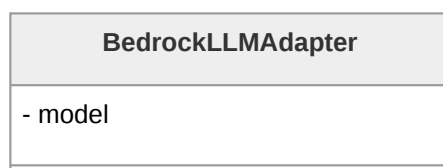


Figura 182: Diagramma della classe **BedrockLLMAdapter**

BedrockLLMAdapter implementa l'interfaccia **ChatbotLLMPort** fornendo l'integrazione concreta con Amazon Bedrock. Questa classe incapsula la logica di comunicazione con il servizio **CloudG**, traducendo le richieste dell'applicazione in chiamate API specifiche per il modello di linguaggio selezionato.

3.5.4.3.5 ChatbotLLMService

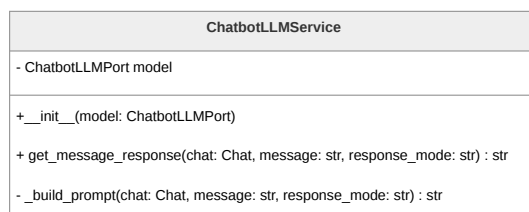


Figura 183: Diagramma della classe **ChatbotLLMService**

Rappresenta la domain logic, coordina la generazione delle risposte del chatbot interagendo con un'implementazione di **ChatbotLLMPort**. Il metodo **get_message_response** orchestra l'intero processo: recupera i messaggi rilevanti dalla cronologia della conversazione



tramite l'algoritmo BM25, costruisce il prompt contestualizzato attraverso `_build_prompt` combinando il messaggio `UtenteG` con il contesto recuperato e il prompt di sistema appropriato alla modalità selezionata, e infine delega al modello la generazione della risposta. Questa classe è quindi responsabile di implementare i diversi meccanismi di risposta sulla base di `response_mode`, realizzando così le funzionalità di risposta "Detective delle relazioni" e "Specchio intelligente".

3.5.4.3.6 ChatsRepositoryPort

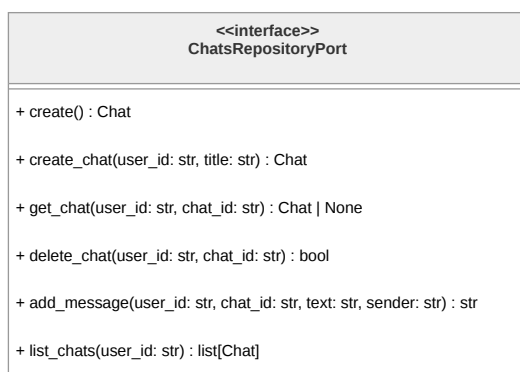


Figura 184: Diagramma dell'interfaccia `ChatsRepositoryPort`

Definisce il contratto per la persistenza delle conversazioni. Espone metodi per tutte le operazioni CRUD: creazione di nuove chat (`create_chat`), recupero singolo (`get_chat`) e multiplo (`list_chats`), eliminazione (`delete_chat`) e aggiunta di messaggi (`add_message`). Questa astrazione garantisce l'indipendenza del dominio dall'implementazione specifica del layer di persistenza.

3.5.4.3.7 DynamoChatAdapter

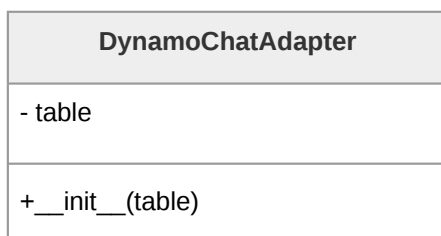


Figura 185: Diagramma della classe `DynamoChatAdapter`

`DynamoChatAdapter` implementa `ChatsRepositoryPort` fornendo la persistenza concreta su DynamoDB. Questa classe gestisce l'interazione con due tabelle: `chats_mvp` per i



metadati delle conversazioni e `chats_messages_mvp` per i messaggi. Implementa la logica di traduzione tra gli oggetti di dominio e le strutture dati di DynamoDB, inclusa la generazione di identificativi ULID e la gestione delle operazioni batch per l'eliminazione dei messaggi. L'adapter incapsula completamente i dettagli tecnici di AWS, esponendo al dominio un'interfaccia pulita e tecnologicamente agnostica.

3.5.4.3.8 CreateChatPort

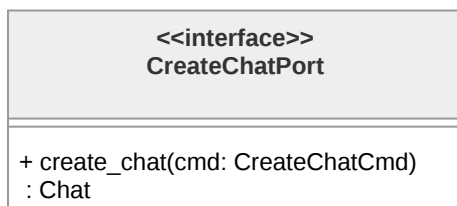


Figura 186: Diagramma dell'interfaccia `CreateChatPort`

L'interfaccia `CreateChatPort` definisce la porta applicativa per la creazione di nuove conversazioni. Espone il metodo `create_chat`, che accetta un comando `CreateChatCmd` e restituisce l'oggetto `Chat` creato. Questa interfaccia rappresenta uno degli use case primari del sistema, separando l'intento dell'operazione dalla sua implementazione.

3.5.4.3.9 GetChatPort

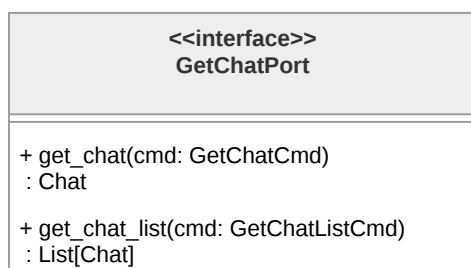


Figura 187: Diagramma dell'interfaccia `GetChatPort`

`GetChatPort` definisce le operazioni di lettura delle conversazioni. Include il metodo `get_chat` per il recupero di una singola chat dato il suo identificativo e `get_chat_list` per ottenere l'elenco di tutte le conversazioni appartenenti a un `Utente`^G. Entrambi i metodi operano attraverso comandi dedicati, mantenendo la separazione tra richiesta ed esecuzione.



3.5.4.3.10 UpdateChatPort

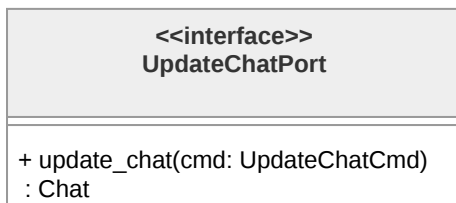


Figura 188: Diagramma dell'interfaccia **UpdateChatPort**

L'interfaccia **UpdateChatPort** gestisce le operazioni di modifica delle conversazioni esistenti. Il metodo `update_chat` accetta un **UpdateChatCmd** e restituisce la chat aggiornata, permettendo la modifica di metadati quali il titolo della conversazione.

3.5.4.3.11 DeleteChatPort

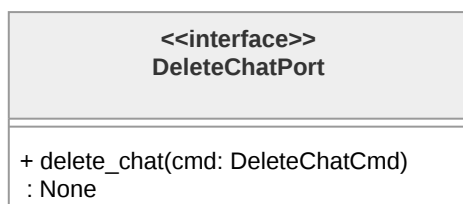


Figura 189: Diagramma dell'interfaccia **DeleteChatPort**

DeleteChatPort definisce l'operazione di eliminazione di una conversazione. Il metodo `delete_chat` riceve un **DeleteChatCmd** contenente gli identificativi necessari e procede con la rimozione della chat e di tutti i messaggi associati dalle rispettive tabelle DynamoDB.

3.5.4.3.12 ChatMessagePort

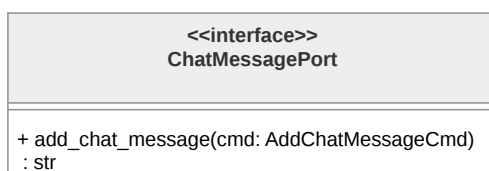


Figura 190: Diagramma dell'interfaccia **ChatMessagePort**

L'interfaccia **ChatMessagePort** astrae l'operazione di aggiunta di messaggi a una conversazione esistente. Il metodo `add_chat_message` accetta un **AddChatMessageCmd** e resti-



tuisce l'identificativo del messaggio appena creato, permettendo al chiamante di tracciare il risultato dell'operazione.

3.5.4.3.13 ChatbotCRUDService

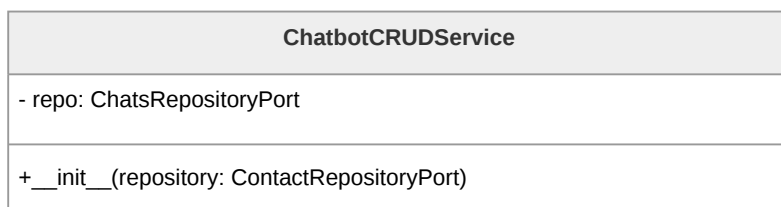


Figura 191: Diagramma della classe **ChatbotCRUDService**

Costituisce la domain logic per le operazione relative alle operazioni CRUD sulle conversazioni, implementando le classi **CreateChatPort**, **GetChatPort**, **UpdateChatPort**, **DeleteChatPort** e **ChatMessagePort**. Questo servizio funge da facade unificato per tutte le operazioni di gestione delle chat, delegando al repository la persistenza effettiva dei dati. Mantiene un riferimento a **ChatsRepositoryPort**, rispettando il principio di inversione delle dipendenze dell'**Architettura^G** esagonale.

3.5.4.3.14 CreateChatCmd

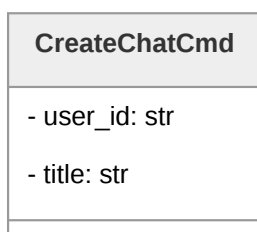


Figura 192: Diagramma della classe **CreateChatCmd**

CreateChatCmd incapsula i parametri necessari per la creazione di una nuova conversazione: l'identificativo dell'**Utente^G** (**user_id**) e il titolo della chat (**title**).



3.5.4.3.15 GetChatCmd

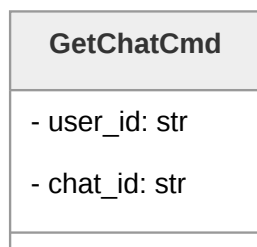


Figura 193: Diagramma della classe **GetChatCmd**

GetChatCmd trasporta i parametri per il recupero di una specifica conversazione: l'identificativo dell'**Utente^G** e quello della chat.

3.5.4.3.16 GetChatListCmd



Figura 194: Diagramma della classe **GetChatListCmd**

GetChatListCmd contiene l'identificativo dell'**Utente^G** per il quale si richiede l'elenco completo delle conversazioni. Questo comando rappresenta l'operazione di recupero multiplo, fondamentale per la visualizzazione dello storico delle chat nell'interfaccia **Utente^G**.

3.5.4.3.17 UpdateChatCmd

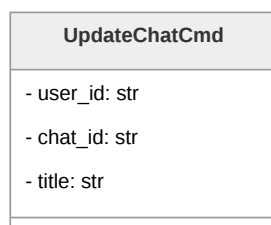


Figura 195: Diagramma della classe **UpdateChatCmd**

UpdateChatCmd incapsula i parametri per la modifica di una conversazione esistente: gli identificativi di **Utente^G** e chat, e il nuovo titolo da assegnare.



3.5.4.3.18 DeleteChatCmd

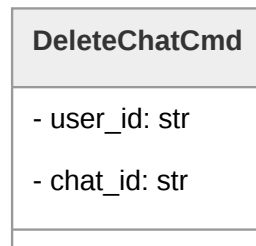


Figura 196: Diagramma della classe **DeleteChatCmd**

DeleteChatCmd trasporta gli identificativi necessari per l'eliminazione di una conversazione e di tutti i messaggi associati.

3.5.4.3.19 AddChatMessageCmd

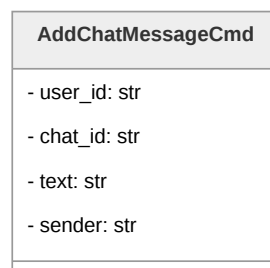


Figura 197: Diagramma della classe **AddChatMessageCmd**

AddChatMessageCmd contiene tutti i parametri necessari per aggiungere un messaggio a una conversazione: identificativi di **Utente^G** e chat, contenuto testuale (**text**) e tipologia del mittente (**sender**). Questo comando viene utilizzato sia per memorizzare i messaggi inviati dall'**Utente^G** che le risposte generate dal chatbot, mantenendo la coerenza dello storico conversazionale.



3.5.4.3.20 ChatDTO

ChatDTO
- user_id : str - chat_id: str - title: str - created_at: datetime - updated_at: datetime - messages: list[Message]
+ to_dict(self): dict + <u>from_domain(chat: Chat): dict</u>

Figura 198: Diagramma della classe **ChatDTO**

ChatDTO è il Data Transfer Object utilizzato per la serializzazione di una chat in un dizionario utilizzato per la risposta della lambda. Contiene tutti i campi dell'entità **Chat** di dominio e fornisce i metodi **to_dict** per la conversione in dizionario Python (successivamente serializzato in JSON) e **from_domain** per la creazione a partire da un oggetto di dominio.

3.5.4.3.21 ChatMessageDTO

ChatMessageDTO
- created_at: datetime - content: str - chat_id: str - sender: str - id: str
+ to_dict(self): dict + <u>from_domain(msg: ChatMessage): dict</u>

Figura 199: Diagramma della classe **ChatMessageDTO**



ChatMessageDTO rappresenta il Data Transfer Object per i singoli messaggi. Analogamente a **ChatDTO**, fornisce metodi per la conversione bidirezionale tra oggetti di dominio e strutture dati serializzabili.

3.5.4.3.22 ChatBotController

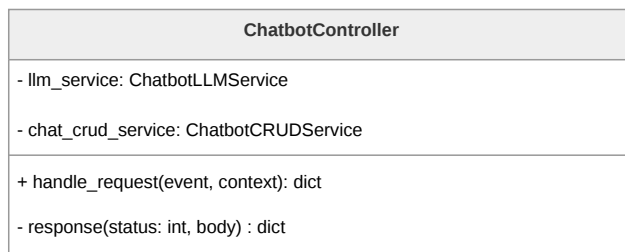


Figura 200: Diagramma della classe **ChatBotController**

Rappresenta l'inbound adapter del microservizio. Esegue il parsing delle richieste inviate da API Gateway in oggetti di dominio. il punto di ingresso della funzione Lambda, li instrada ai gestori appropriati, coordina l'interazione tra **ChatbotCRUDService** e **ChatbotLLMService**, e costruisce le risposte HTTP conformi alle specifiche dell'API.

3.5.5 Contatti fidati, invio SOS e Dead man's switch

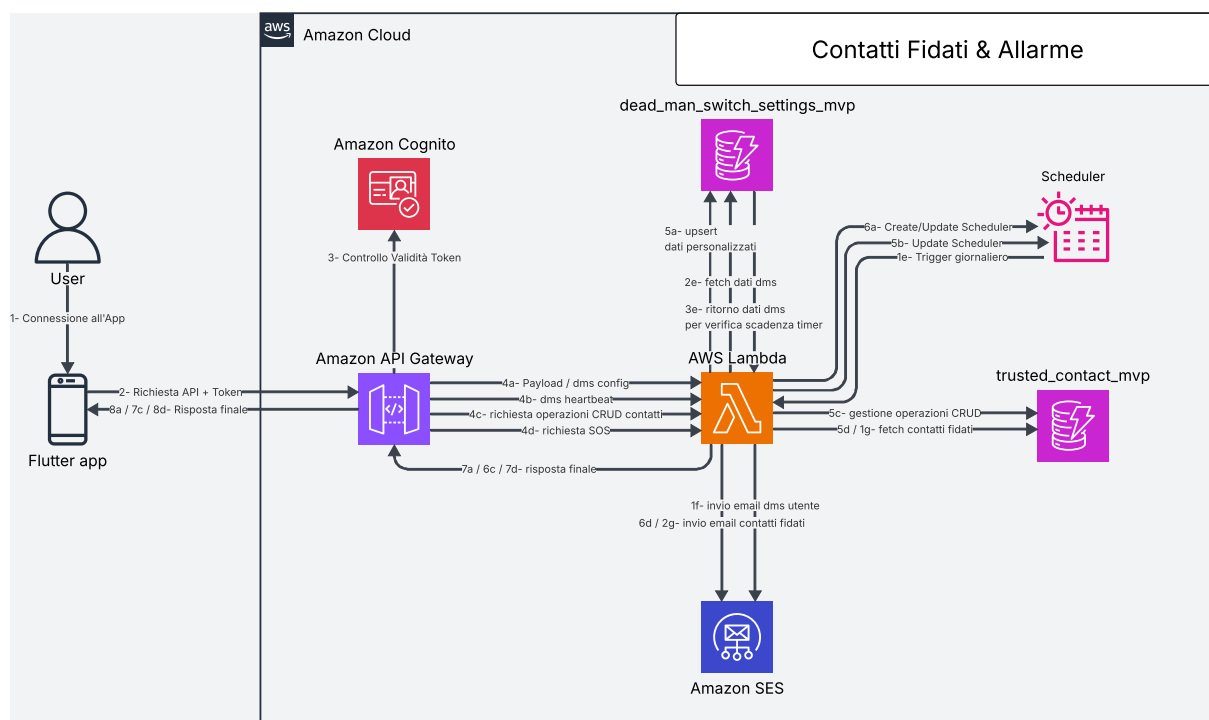


Figura 201: **Architettura^G** del sistema di contatti fidati, SOS e Dead man's switch



3.5.5.1 Architettura e Flusso di Elaborazione: contatti fidati, SOS e Dead man's switch

La sezione relativa ai contatti fidati, all'invio di messaggi di emergenza e al meccanismo di Dead man's switch è progettata per garantire la gestione automatizzata dello stato di attività dell'**Utente^G** e la possibilità di attivare procedure di emergenza sia manualmente che in modo automatico.

API Gateway gestisce l'esposizione delle API e verifica, per ogni richiesta, che l'**Utente^G** sia correttamente autenticato all'applicazione. Le richieste valide vengono inoltrate a una funzione Lambda centrale, responsabile della gestione della logica applicativa relativa ai contatti fidati, alla configurazione del Dead man's switch e all'orchestrazione dell'invio delle email tramite Amazon SES.

I dati sono memorizzati su due database DynamoDB distinti:

- **dead_man_switch_settings_mvp**: contiene la configurazione del Dead man's switch, includendo il titolo e il corpo dell'email di inattività, il primo e secondo timer di inattività, l'ultimo accesso dell'**Utente^G** e lo stato corrente del sistema (attivo, prima inattività, seconda inattività).
- **trusted_contact_mvp**: memorizza le informazioni relative ai contatti fidati associati a ciascun **Utente^G**, inclusi nome, email e numero di telefono.

Il sistema implementa inoltre un meccanismo di monitoraggio automatico basato su AWS EventBridge. Uno scheduler giornaliero attiva la Lambda centrale che verifica lo stato di attività degli utenti e determina eventuali condizioni di inattività prolungata.

Il flusso di elaborazione del Dead man's switch è il seguente:

- **Trigger programmato**: EventBridge attiva quotidianamente la Lambda, che avvia il controllo dello stato degli utenti.
- **Recupero configurazioni**: la Lambda effettua il fetch dei dati presenti in **dead_man_switch_settings_mvp** ottenendo i timer di inattività, l'ultimo accesso registrato e lo stato corrente del Dead man's switch.
- **Valutazione dello stato di inattività**: la funzione confronta il tempo corrente con l'ultimo accesso dell'**Utente^G**. Se viene superata la prima soglia di inattività, il sistema entra nello stato di "prima inattività". Al raggiungimento della seconda soglia di inattività, viene attivato lo stato di "seconda inattività".
- **Invio email di prima inattività**: al superamento della prima soglia, la Lambda invia tramite Amazon SES un'email all'**Utente^G** utilizzando il titolo e il corpo configurati, notificando lo stato di inattività.



- **Attivazione SOS (seconda inattività):** al superamento della seconda soglia, la Lambda recupera l'elenco dei contatti fidati da `trusted_contact_mvp` e invia, tramite SES, un'email di SOS predefinita a ciascun contatto.
- **Aggiornamento stato:** lo stato del sistema viene aggiornato in DynamoDB per evitare invii ripetuti non necessari.

Oltre al flusso automatico, il sistema supporta l'invio manuale di SOS e la gestione in tempo reale dello stato di attività dell'**Utente^G** tramite heartbeat.

Il flusso di heartbeat è il seguente:

- **Invio heartbeat:** All'avvio dell'app, il client invia una richiesta all'API `POST /heartbeat` per notificare l'utilizzo attivo dell'applicazione.
- **Aggiornamento stato Backend^G:** la Lambda aggiorna il campo "ultimo accesso" nella tabella `dead_man_switch_settings_mvp` e, se necessario, resetta lo stato di inattività.

L'invio SOS **manuale** segue invece il seguente flusso:

- **Richiesta di emergenza:** il client invia una richiesta HTTPS all'API `POST /alert`.
- **Recupero contatti:** la Lambda recupera tutti i contatti fidati associati all'**Utente^G** da `trusted_contact_mvp`.
- **Invio email SES:** la funzione Lambda invia immediatamente un'email di emergenza a tutti i contatti fidati tramite Amazon SES.

La gestione dei contatti fidati è completamente CRUD e permette all'**Utente^G** di mantenere aggiornato il proprio elenco di destinatari di emergenza.

3.5.5.2 API associate

- **POST `/dms_settings`**
 - **Descrizione:** Crea la configurazione del Dead Man's Switch per l'**Utente^G** autenticato.
 - **Parametri:**
 - * `user_id`: identificativo dell'**Utente^G**;
 - * `user_email`: email dell'**Utente^G**;
 - * `user_name`: nome dell'**Utente^G**.
 - **Response:**
 - * 200 OK: configurazione creata con successo.
 - * Corpo della risposta:



```
{
  "user_id": "user1",
  "is_active": true,
  "first_timer": 60,
  "second_timer": 120,
  "email_subject": "Messaggio di emergenza",
  "email_body": "Corpo del messaggio"
}
```

– **Errori:**

- * 400 Bad Request: campi non validi.
- * 500 Internal Server Error: errore interno del server.

● **GET /dms_settings**

– **Descrizione:** Recupera la configurazione del Dead Man's Switch dell'**Utente^G**.

– **Parametri:**

- * **user_id:** identificativo dell'**Utente^G**.

– **Response:**

- * 200 OK: configurazione recuperata con successo.
- * Corpo della risposta:

```
{
  "user_id": "user1",
  "is_active": true,
  "first_timer": 60,
  "second_timer": 120,
  "email_subject": "Messaggio di emergenza",
  "email_body": "Corpo del messaggio"
}
```

– **Errori:**

- * 500 Internal Server Error: errore interno del server.

● **PUT /dms_settings**

– **Descrizione:** Aggiorna le impostazioni del Dead Man's Switch.

– **Parametri:**

- * **user_id:** identificativo dell'**Utente^G**;
- * **is_active:** valore booleano
- * **first_timer:** primo timer del dead man's switch;
- * **second_timer:** secondo timer del dead man's switch;



- * **email_subject**: oggetto della mail per la notifica del dead man's switch all'**Utente^G**;
- * **email_body**: corpo della mail per la notifica del dead man's switch all'**Utente^G**.
- **Response**:
 - * 204 OK: aggiornamento della configurazione avvenuto con successo.
 - * Corpo della risposta:

```
{  
    "user_id": "user1",  
    "is_active": true,  
    "first_timer": 60,  
    "second_timer": 120,  
    "email_subject": "Messaggio di emergenza",  
    "email_body": "Corpo del messaggio"  
}
```
- **Errori**:
 - * 500 Internal Server Error: errore interno del server.
- **PUT /dms_settings/heartbeat**
 - **Descrizione**: registra il segnale di attività dell'**Utente^G** aggiornando l'ultimo accesso.
 - **Parametri**:
 - * **user_id**: identificativo dell'**Utente^G**.
 - **Response**:
 - * 200: dati aggiornati con successo.
 - * Corpo della risposta:

```
{  
    "message": "Heartbeat processed with success"  
}
```
- **POST /trusted_contact**
 - **Descrizione**: Crea un nuovo contatto fidato associato all'**Utente^G** autenticato.
 - **Parametri**:
 - * **user_id**: identificativo dell'**Utente^G**;
 - * **contact_name**: nome del contatto fidato;
 - * **contact_email** email del contatto fidato;



- * `contact_phone_number`: numero di telefono del contatto fidato.

– **Response:**

- * 200 OK: creazione contatti completata con successo.

- * Corpo della risposta:

```
{
  "user_id": "user1",
  "contact_id": "contact123",
  "contact_name": "Mario Rossi",
  "contact_email": "mario@gmail.com",
  "contact_phone_number": "333 1111 111"
}
```

– **Errori:**

- * 400 Bad Request: errore nella **Validazione^G** dei campi.

- * 500 Internal Server Error: errore interno del server.

- **GET /trusted_contact**

– **Descrizione:** Recupera tutti i contatti fidati associati all'**Utente^G** autenticato.

– **Parametri:**

- * `user_id`: identificativo dell'**Utente^G**.

– **Response:**

- * 200 OK: contatti recuperati con successo.

- * Corpo della risposta:

```
{
  "trusted_contacts": [
    {
      "user_id": "user1",
      "contact_id": "contact1",
      "contact_name": "Mario Rossi",
      "contact_email": "mario@gmail.com",
      "contact_phone_number": "111 1111 111"
    },
    {
      "user_id": "user1",
      "contact_id": "contact2",
      "contact_name": "Luigi Verdi",
      "contact_email": "luigi@gmail.com",
      "contact_phone_number": "333 2222 222"
    }
  ]
}
```



```
    }
  ]
}
```

- **GET /trusted_contact/{contact_id}**

- **Descrizione:** Recupera un contatto specifico associato all'**Utente^G** autenticato.

- **Parametri:**

- * **user_id:** identificativo dell'**Utente^G**.

- **Response:**

- * 200 OK: contatto recuperato con successo.

- * Corpo della risposta:

```
{
  "user_id": "user1",
  "contact_id": "contact1",
  "contact_name": "Mario Rossi",
  "contact_email": "mario@gmail.com",
  "contact_phone_number": "111 1111 111"
}
```

- **Errori:**

- * 500 Internal Server Error: errore interno del server.

- **PUT /trusted_contact**

- **Descrizione:** Aggiorna un contatto fidato esistente associato all'**Utente^G** autenticato.

- **Parametri:**

- * **user_id:** identificativo dell'**Utente^G**;
 - * **contact_id:** identificativo del contatto fidato;
 - * **contact_name:** nome del contatto fidato;
 - * **contact_email** email del contatto fidato;
 - * **contact_phone_number:** numero di telefono del contatto fidato.

- **Response:**

- * 200 OK: contatto aggiornato con successo.

- * Corpo della risposta:

```
{
  "user_id": "user1",
```



```
"contact_id": "contact1",  
"contact_name": "Mario Rossi",  
"contact_email": "mario@gmail.com",  
"contact_phone_number": "111 1111 111"  
}
```

– **Errori:**

- * 400 Bad Request: errore nella **Validazione^G** dei campi.
- * 500 Internal Server Error: errore interno del server.

● **DELETE /trusted_contact/{contact_id}**

– **Descrizione:** Elimina un contatto fidato esistente associato all'**Utente^G** autenticato.

– **Parametri:**

- * **user_id:** identificativo dell'**Utente^G**.

– **Response:**

- * 200 OK: eliminazione contatto avvenuta con successo.
{
 "message": "Contact contact1 deleted"
}

● **PUT /alert**

– **Descrizione:** Invia un alert ai contatti fidati con posizione opzionale.

– **Parametri:**

- * **user_id:** identificativo dell'**Utente^G**;
- * **latitude** (opzionale);
- * **longitude** (opzionale).

– **Response:**

- * 200 OK: invio dell'alert avvenuto con successo.
{
 "message": "Alert send processed with success"
}

– **Errori:**

- * 400 Bad Request: **Verifica^G** presenza contatti fidati fallita.
- * 500 Internal Server Error: errore interno del server.

3.5.5.3 Architettura logica: Modulo contatti fidati, SOS e Dead man's switch

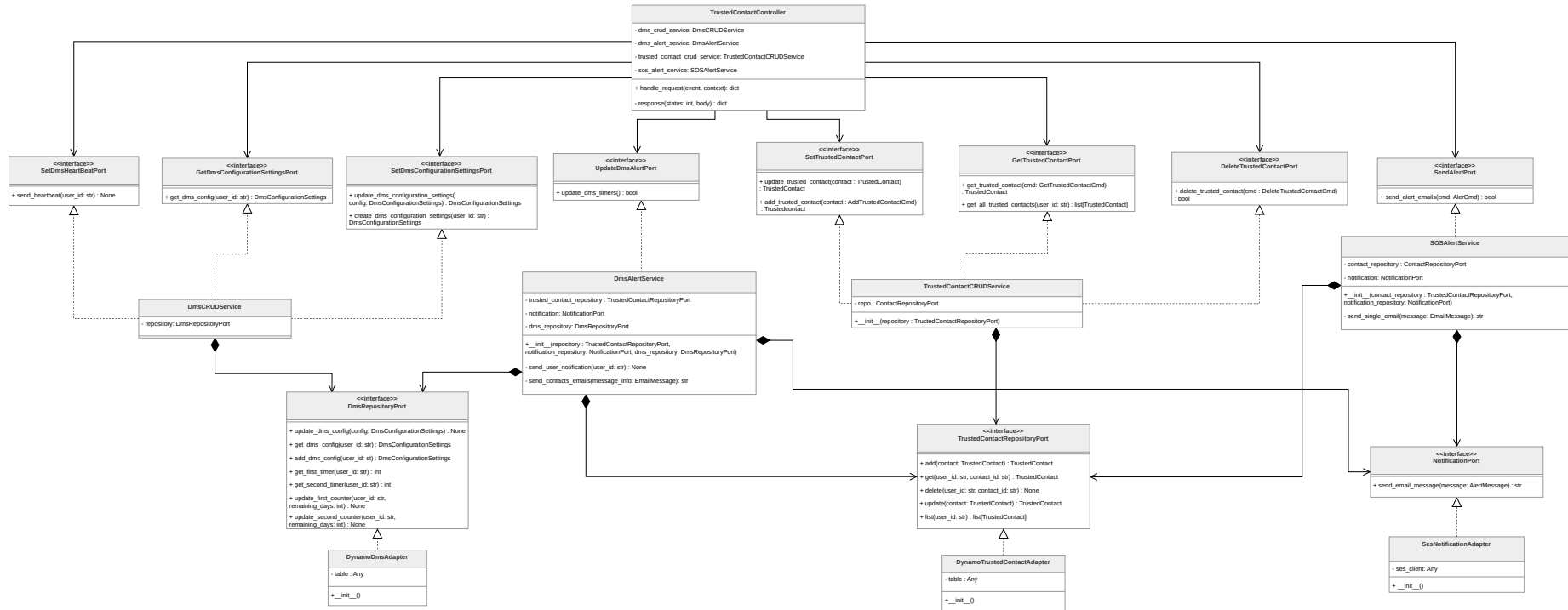


Figura 202: Componenti del microservizio contatti fidati, SOS e Dead man's switch





3.5.5.3.1 TrustedContact

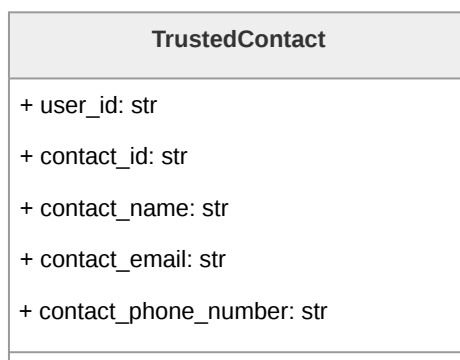


Figura 203: Diagramma della classe **TrustedContact**

Rappresenta un contatto fidato associato a un **Utente^G**.

Descrizione attributi:

- **user_id** identifica l'**Utente^G** proprietario del contatto;
- **contact_id** identifica univocamente il contatto;
- **contact_name**, **contact_email** e **contact_phone_number** ne rappresentano i dati anagrafici.

3.5.5.3.2 TrustedContactDTO

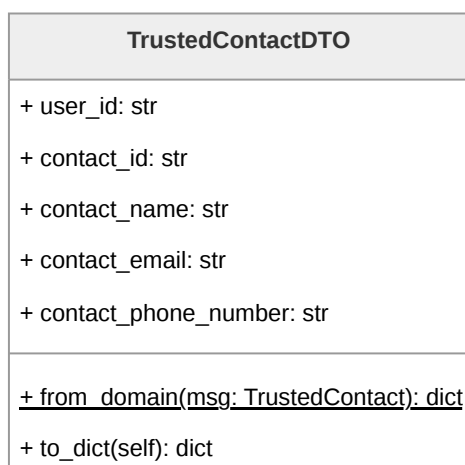


Figura 204: Diagramma della classe **TrustedContactDTO**



Rappresenta il Data Transfer Object utilizzato per la serializzazione di un contatto fidato in un dizionario utilizzato per la risposta della lambda. Contiene tutti i campi dell'entità di dominio **TrustedContact** e fornisce i metodi **to_dict** per la conversione in dizionario Python (successivamente serializzato in JSON) e **from_domain** per la creazione a partire da un oggetto di dominio.

3.5.5.3.3 AddTrustedContactCmd



Figura 205: Diagramma della classe **AddTrustedContactCmd**

Command Object utilizzato dal controller per trasportare i dati necessari all'aggiunta di un contatto fidato.

3.5.5.3.4 GetTrustedContactCmd

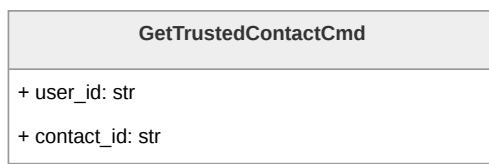


Figura 206: Diagramma della classe **GetTrustedContactCmd**

Command Object utilizzato dal controller per trasportare i dati necessari per il fetch di un contatto fidato.

3.5.5.3.5 DeleteTrustedContactCmd

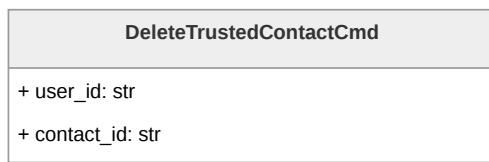


Figura 207: Diagramma della classe **DeleteTrustedContactCmd**

Command Object utilizzato dal controller per trasportare i dati necessari per la cancellazione di un contatto fidato.



3.5.5.3.6 DmsConfigurationSettings

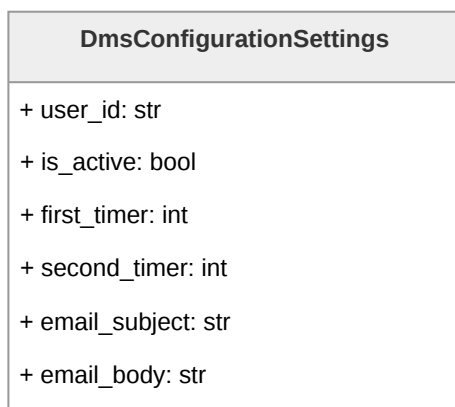


Figura 208: Diagramma della classe **DmsConfigurationSettings**

Rappresenta la configurazione del Dead Man's Switch associata ad un **Utente^G**.
Descrizione attributi:

- **user_id** identifica l'**Utente^G** a cui è associata la configurazione del dead man's switch;
- **is_active** indica se il dead man's switch è attivo;
- **first_timer** e **second_timer** definiscono la durata in giorni dei due timer;
- **email_body** contiene il corpo della mail riferita al dead man's switch;
- **email_subject** contiene l'oggetto della mail riferita al dead man's switch.

3.5.5.3.7 DmsConfigurationSettingsDTO

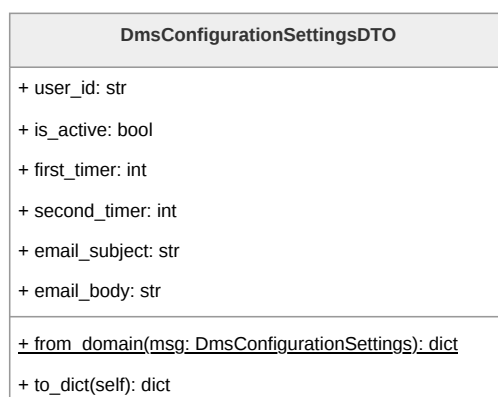


Figura 209: Diagramma della classe **DmsConfigurationSettingsDTO**



Data Transfer Object utilizzato per la serializzazione delle impostazioni di un **Utente^G** in un dizionario utilizzato per la risposta della lambda. . Contiene tutti i campi dell'entità di dominio **DmsConfigurationSettings** e fornisce i metodi **to_dict** per la conversione in dizionario Python (successivamente serializzato in JSON) e **from_domain** per la creazione a partire da un oggetto di dominio.

3.5.5.3.8 AlertCmd

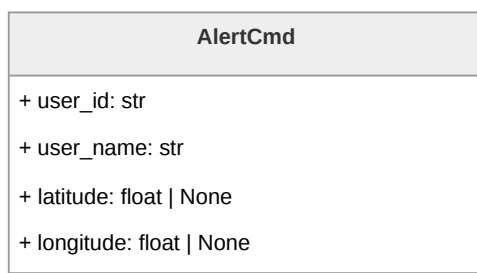


Figura 210: Diagramma della classe **AlertCmd**

Command Object utilizzato dal controller per trasportare i dati necessari all'invio dell'alert SOS.

Descrizione attributi:

- **user_id**, identificativo dell'**Utente^G**;
- **user_name**, nome dell'**Utente^G** che ha azionato l'alert;
- **latitude** e **longitude** indicano le coordinate geografiche in cui si trova l'**Utente^G** al momento dell'invio dell'alert.

3.5.5.3.9 EmailMessage

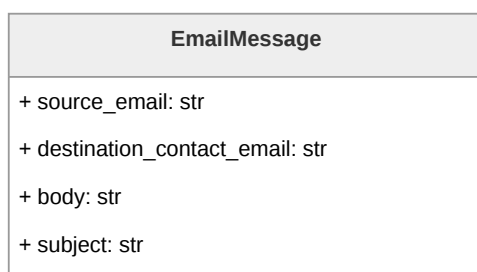


Figura 211: Diagramma della classe **EmailMessage**

Rappresenta la struttura delle email inviabili dal sistema.

Descrizione attributi:



- **source_email**, indirizzo con cui viene mandata l'email di alert;
- **destination_contact_email**, indirizzo email a cui viene inviato l'alert;
- **body**, corpo della mail di alert,
- **subject**, oggetto della mail di alert.

3.5.5.3.10 TrustedContactController

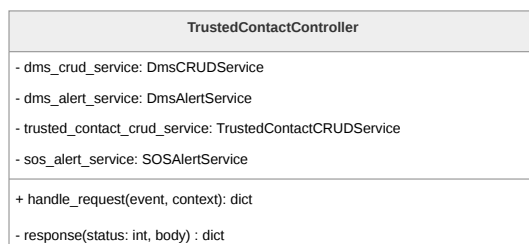


Figura 212: Diagramma della classe **TrustedContactController**

Rappresenta l'inbound adapter del microservizio. Riceve la richiesta grezza di API Gateway e la instrada verso la port primaria appropriata in base alla route.

Descrizione metodi:

- **handle_request(event, context)**, punto di ingresso del controller, legge la **routeKey** dall'evento e chiama la port primaria corrispondente;
- **response(status, body)**, metodo privato di utilità che costruisce la risposta nel formato atteso da API Gateway.

3.5.5.3.11 SetDmsHeartBeatPort

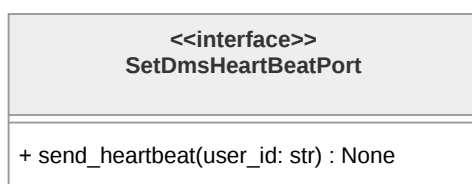


Figura 213: Diagramma della classe **SetDmsHeartBeatPort**

Espone **send_heartbeat(user_id: str)** per il reset dei timer del Dead Man's Switch al rientro dell'**Utente^G** nell'applicazione.



3.5.5.3.12 GetDmsConfigurationSettingsPort

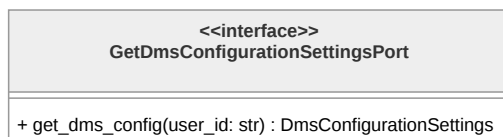


Figura 214: Diagramma della classe **GetDmsConfigurationSettingsPort**

Espone **get_dms_config(user_id: str)** per il recupero della configurazione del Dead Man's Switch.

3.5.5.3.13 SetDmsConfigurationSettingsPort

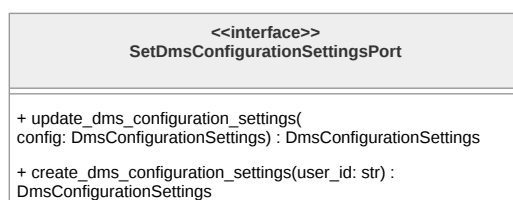


Figura 215: Diagramma della classe **SetDmsConfigurationSettingsPort**

Descrizione metodi:

- **update_dms_configuration_settings(config: DmsConfigurationSettings)**, per l'aggiornamento delle impostazioni del dead man's switch;
- **create_dms_configuration_settings(user_id: str)**, per la creazione dei valori di default per le impostazioni del dead man's switch.

3.5.5.3.14 UpdateDmsAlertPort

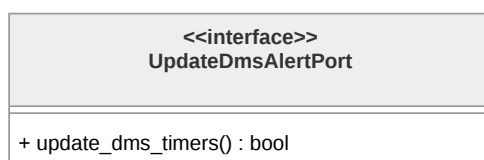


Figura 216: Diagramma della classe **UpdateDmsAlertPort**

Espone **update_dms_timers()** per l'aggiornamento dei timer lato **Backend**^G.



3.5.5.3.15 SetTrustedContactPort

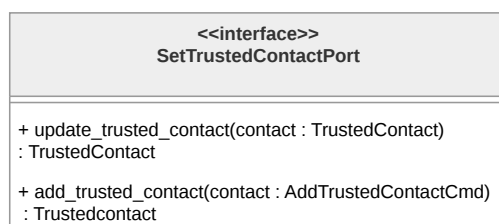


Figura 217: Diagramma della classe **SetTrustedContactPort**

Descrizione dei metodi:

- **add_trusted_contact(contact: AddTrustedContactCmd)** per la creazione di un nuovo contatto;
- **update_trusted_contact(request: TrustedContact)** per la modifica di uno esistente.

3.5.5.3.16 GetTrustedContactPort



Figura 218: Diagramma della classe **GetTrustedContactPort**

Descrizione dei metodi:

- **get_trusted_contact(cmd: GetTrustedContactCmd)** per il recupero di un singolo contatto;
- **get_all_trusted_contacts(user_id: str)** per il recupero della lista completa.

3.5.5.3.17 DeleteTrustedContactPort

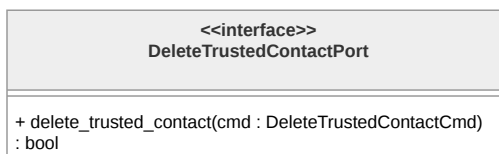


Figura 219: Diagramma della classe **DeleteTrustedContactPort**

Espone **delete_trusted_contact(cmd: DeleteTrustedContactCmd)** per l'eliminazione di un contatto.



3.5.5.3.18 SendAlertPort

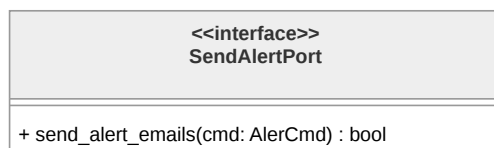


Figura 220: Diagramma della classe **SendAlertPort**

Espone `send_alert_emails(alert_info: AlertCmd)` per l'invio delle email di soccorso ai contatti fidati.

3.5.5.3.19 DmsCRUDService

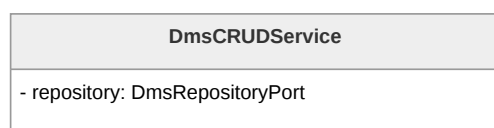


Figura 221: Diagramma della classe **DmsCRUDService**

Rappresenta la domain logic del microservizio, è responsabile delle operazioni di lettura e scrittura sui valori delle impostazioni del Dead Man's Switch. Implementa le inbound port che definiscono le operazioni crud possibili sui dati.

3.5.5.3.20 DmsAlertService

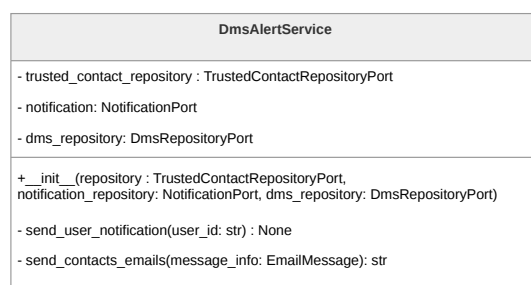


Figura 222: Diagramma della classe **DmsAlertService**

Rappresenta la domain logic del microservizio, è responsabile dell'invio delle notifiche del Dead Man's Switch. Si occupa del recupero dei contatti fidati, la configurazione del DMS e la costruzione dei messaggi email.

Descrizione metodi:

- `__init__(trusted_contact_repository, notification_repository, dms_repository)`, costruttore che riceve le tre port secondarie tramite dependency injection;



- `send_user_notification(user_id: str)`, metodo privato che recupera la configurazione DMS dell'**Utente^G** e costruisce e invia la notifica all'**Utente^G** alla scadenza del primo timer;
- `send_contacts_emails(message_info: EmailMessage)`, metodo privato che costruisce e invia le email ai contatti fidati dell'**Utente^G** alla scadenza del secondo timer.

3.5.5.3.21 TrustedContactCRUDService

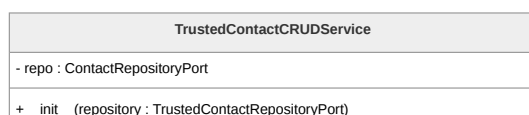


Figura 223: Diagramma della classe **TrustedContactCRUDService**

Rappresenta la domain logic, è responsabile delle operazioni CRUD sui contatti fidati.
Descrizione metodi:

- `__init__(repository: TrustedContactRepositoryPort)`, costruttore che riceve la port secondaria tramite dependency injection.

3.5.5.3.22 SOSAlertService

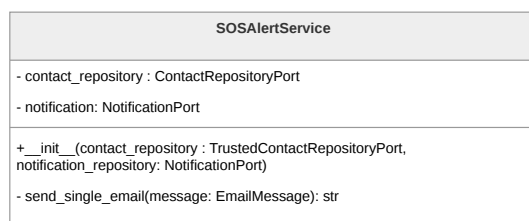


Figura 224: Diagramma della classe **SOSAlertService**

Rappresenta la domain logic, è responsabile dell'invio immediato delle email di soccorso al momento della pressione del pulsante SOS da parte dell'**Utente^G**. Implementa la port primaria **SendAlertPort**.

Descrizione metodi:

- `__init__(contact_repository, notification_repository)`, costruttore che riceve le due port secondarie tramite dependency injection;
- `send_single_email(message: EmailMessage)`, metodo privato che costruisce e invia una singola email a partire da un oggetto **EmailMessage**, delegando l'invio alla **NotificationPort**.



3.5.5.3.23 DmsRepositoryPort

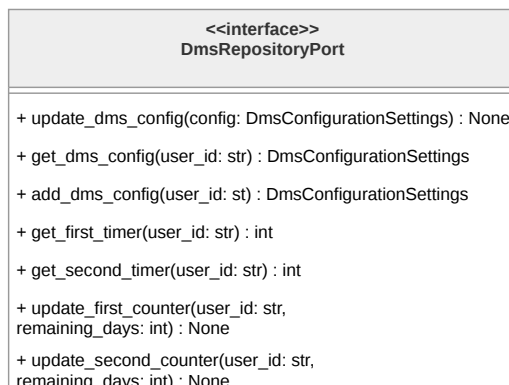


Figura 225: Diagramma della classe **DmsRepositoryPort**

Costituisce l'outbound port, fornisce il contratto per la persistenza delle impostazioni del Dead man's Switch. Espone i metodi utilizzati dalla domain logic. Descrizione metodi:

- `update_dms_config(config: DmsConfigurationSettings)`, aggiorna la configurazione del dead man's switch dell'**Utente^G**;
- `get_dms_config(user_id: str)`, recupera la configurazione del dead man's switch associata all'**Utente^G**;
- `add_dms_config(user_id: str)`, crea una nuova configurazione del dead man's switch per l'**Utente^G**;
- `get_first_timer(user_id: str)`, recupera il valore corrente del primo timer;
- `get_second_timer(user_id: str)`, recupera il valore corrente del secondo timer;
- `update_first_counter(user_id: str, remaining_days: int)`, aggiorna il contatore residuo del primo timer;
- `update_second_counter(user_id: str, remaining_days: int)`, aggiorna il contatore residuo del secondo timer.



3.5.5.3.24 TrustedContactRepositoryPort

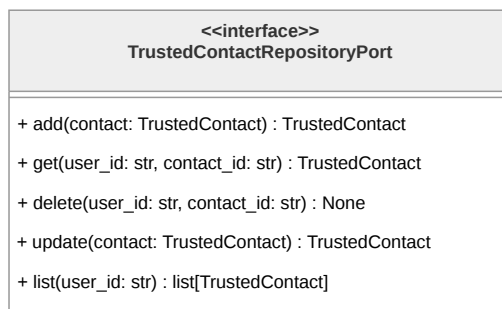


Figura 226: Diagramma della classe **TrustedContactRepositoryPort**

Definisce il contratto per la persistenza dei contatti fidati. Espone i metodi per le operazioni CRUD. Descrizione metodi:

- **add(contact: TrustedContact)**, aggiunge un nuovo contatto fidato e restituisce l'entità creata;
- **get(user_id: str, contact_id: str)**, recupera un singolo contatto fidato tramite le chiavi identificative, restituendo **None** se non trovato;
- **delete(user_id: str, contact_id: str)**, elimina il contatto fidato corrispondente alle chiavi fornite;
- **update_trusted_contact(contact: TrustedContact)**, aggiorna i dati anagrafici di un contatto fidato esistente;
- **list(user_id: str)**, restituisce la lista completa dei contatti fidati associati all'**Utente^G**.

3.5.5.3.25 NotificationPort

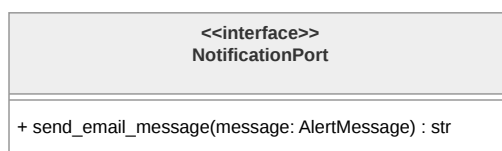


Figura 227: Diagramma della classe **NotificationPort**

È la inbound port del pattern esagonale per l'invio delle notifiche email. Descrizione metodi:

- **send_email_message(message: EmailMessage)**, invia un messaggio email a partire da un oggetto **AlertMessage**.



3.5.5.3.26 DynamoDmsAdapter

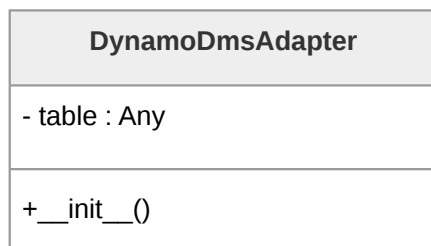


Figura 228: Diagramma della classe **DynamoDmsAdapter**

Contiene la logica specifica per la persistenza della configurazione e dei contatori del Dead Man's Switch su DynamoDB.

3.5.5.3.27 DynamoTrustedContactAdapter

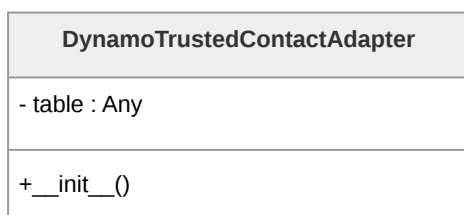


Figura 229: Diagramma della classe **DynamoTrustedContactAdapter**

Contiene tutta la logica specifica per la persistenza dei contatti fidati su DynamoDB.

3.5.5.3.28 SesNotificationAdapter

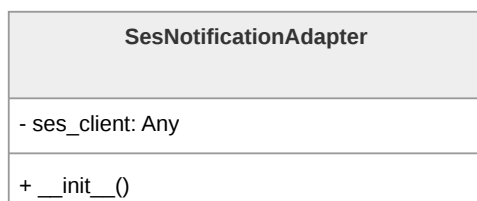


Figura 230: Diagramma della classe **SesNotificationAdapter**

Contiene tutta la logica specifica per l'invio delle email tramite il servizio Amazon SES.



4. Stato dei requisiti funzionali

4.1. Requisiti Funzionali obbligatori

Codice	Descrizione	Stato
RF-OB-1	L' Utente^G deve poter autenticarsi all'applicazione tramite l'utilizzo di Google Auth	Non soddisfatto
RF-OB-2	Il sistema deve attivare automaticamente la funzionalità di Dead Man's Switch a seguito del login dell' Utente^G	Non soddisfatto
RF-OB-3	L' Utente^G deve poter visualizzare un messaggio di errore esplicativo nel caso in cui l'autenticazione tramite Google Auth fallisca, venga annullata o restituisca parametri non validi	Non soddisfatto
RF-OB-5	L' Utente^G deve poter visualizzare la lista dei contatti fidati	Non soddisfatto
RF-OB-6	L' Utente^G deve poter visualizzare un messaggio di errore esplicativo nel caso in cui il recupero dei dati dal database fallisca	Non soddisfatto
RF-OB-7	L' Utente^G deve poter ricaricare la sezione dei contatti fidati dopo la visualizzazione dell'errore di caricamento dei dati dal database	Non soddisfatto
RF-OB-8	Il sistema deve permettere la visualizzazione di un singolo contatto fidato dalla lista dei contatti fidati	Non soddisfatto
RF-OB-9	L' Utente^G deve poter visualizzare la preview delle informazioni (il nome) del contatto fidato che sta visualizzando dalla lista dei contatti fidati	Non soddisfatto
RF-OB-10	L' Utente^G deve poter visualizzare i dettagli di un contatto fidato selezionato dalla lista dei contatti fidati	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-11	L'Utente ^G deve poter visualizzare il nome del contatto fidato selezionato dalla lista dei contatti fidati	Non soddisfatto
RF-OB-12	L'Utente ^G deve poter visualizzare il numero di telefono del contatto fidato selezionato dalla lista dei contatti fidati	Non soddisfatto
RF-OB-13	L'Utente ^G deve poter visualizzare l'indirizzo email del contatto fidato selezionato dalla lista dei contatti fidati	Non soddisfatto
RF-OB-14	L'Utente ^G deve poter aggiungere un nuovo contatto fidato alla lista dei contatti fidati	Non soddisfatto
RF-OB-15	L'Utente ^G deve poter inserire il nome del nuovo contatto fidato durante l'aggiunta di esso alla lista dei contatti fidati	Non soddisfatto
RF-OB-16	L'Utente ^G deve poter inserire il numero di telefono del nuovo contatto fidato durante l'aggiunta di esso alla lista dei contatti fidati	Non soddisfatto
RF-OB-17	L'Utente ^G deve poter inserire l'indirizzo email del nuovo contatto fidato durante l'aggiunta di esso alla lista dei contatti fidati	Non soddisfatto
RF-OB-18	Il sistema deve riconoscere e gestire le informazioni di numero di telefono e email già presenti nel sistema	Non soddisfatto
RF-OB-19	L'Utente ^G deve poter visualizzare un messaggio di errore esplicativo nel caso in cui le informazioni di numero di telefono e email siano già presenti nel sistema	Non soddisfatto
RF-OB-20	L'Utente ^G deve poter reinserire le informazioni di numero di telefono e email dopo la visualizzazione del messaggio di errore esplicativo	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-21	Il sistema deve validare la conformità del numero di telefono inserito o modificato	Non soddisfatto
RF-OB-22	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui il numero di telefono inserito o modificato non sia conforme ai criteri prestabiliti	Non soddisfatto
RF-OB-23	Il sistema deve validare la conformità dell'indirizzo email inserito o modificato	Non soddisfatto
RF-OB-24	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui l'indirizzo email inserito o modificato non sia conforme ai criteri prestabiliti	Non soddisfatto
RF-OB-25	L' Utente^G deve poter modificare i dati di un contatto fidato esistente	Non soddisfatto
RF-OB-26	L' Utente^G deve poter modificare il nome di un contatto fidato esistente	Non soddisfatto
RF-OB-27	L' Utente^G deve poter modificare il numero di telefono di un contatto fidato esistente	Non soddisfatto
RF-OB-28	L' Utente^G deve poter modificare l'indirizzo email di un contatto fidato esistente	Non soddisfatto
RF-OB-29	L' Utente^G deve poter eliminare un contatto fidato	Non soddisfatto
RF-OB-30	Il sistema deve richiedere conferma prima dell'eliminazione di un contatto fidato	Non soddisfatto
RF-OB-31	L' Utente^G deve poter confermare, con un'apposita voce, l'eliminazione di un contatto fidato quando il sistema richiede la conferma	Non soddisfatto
RF-OB-32	L' Utente^G deve poter effettuare il logout dopo essersi autenticato all'applicazione	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-34	L'Utente ^G deve poter modificare le informazioni della mail di avviso	Non soddisfatto
RF-OB-35	L'Utente ^G deve poter modificare il titolo della mail di avviso	Non soddisfatto
RF-OB-36	L'Utente ^G deve poter modificare il corpo della mail di avviso	Non soddisfatto
RF-OB-37	L'Utente ^G deve poter visualizzare un'anteprima della mail di avviso	Non soddisfatto
RF-OB-38	L'Utente ^G deve poter visualizzare l'anteprima del corpo della mail di avviso	Non soddisfatto
RF-OB-39	L'Utente ^G deve poter visualizzare un'anteprima del titolo della mail di avviso	Non soddisfatto
RF-OB-40	L'Utente ^G deve poter attivare il Dead Man's Switch	Non soddisfatto
RF-OB-41	L'Utente ^G deve poter disattivare la funzionalità di Dead Man's Switch	Non soddisfatto
RF-OB-42	L'Utente ^G deve poter impostare una password per il diario criptato	Non soddisfatto
RF-OB-43	Il sistema deve validare la conformità della password del diario criptato secondo i criteri pre-stabiliti	Non soddisfatto
RF-OB-44	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui la password del diario criptato non sia conforme ai criteri pre-stabiliti	Non soddisfatto
RF-OB-45	L'Utente ^G deve poter reinserire la password del diario criptato dopo la visualizzazione del messaggio di errore esplicativo	Non soddisfatto
RF-OB-46	L'Utente ^G deve poter impostare una password per il diario fittizio	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-47	Il sistema deve validare la conformità e l'unicità della password del diario fittizio rispetto a quella del diario criptato	Non soddisfatto
RF-OB-48	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui la password del diario fittizio non sia conforme ai criteri prestabiliti	Non soddisfatto
RF-OB-49	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui la password del diario fittizio sia uguale a quella del diario criptato	Non soddisfatto
RF-OB-50	L' Utente^G deve poter reinserire la password del diario fittizio dopo la visualizzazione del messaggio di errore esplicativo	Non soddisfatto
RF-OB-51	L' Utente^G deve poter inserire la password per accedere al diario desiderato	Non soddisfatto
RF-OB-52	L' Utente^G deve poter inserire la password per accedere al diario criptato	Non soddisfatto
RF-OB-53	L' Utente^G deve poter inserire la password per accedere al diario fittizio	Non soddisfatto
RF-OB-54	Il sistema deve visualizzare un messaggio di errore esplicativo nel caso in cui la password inserita per l'accesso al diario risulti errata	Non soddisfatto
RF-OB-55	L' Utente^G deve poter reinserire la password di accesso al diario dopo la visualizzazione del messaggio di errore esplicativo	Non soddisfatto
RF-OB-57	L' Utente^G deve poter visualizzare il contenuto del diario criptato	Non soddisfatto
RF-OB-58	L' Utente^G deve poter visualizzare la lista delle note presenti nel diario criptato	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-59	L'Utente ^G deve poter visualizzare l'anteprima della singola nota all'interno della lista delle note del diario criptato	Non soddisfatto
RF-OB-60	L'Utente ^G deve poter ricaricare la sezione delle note del diario criptato dopo la visualizzazione dell'errore di caricamento dei dati dal database	Non soddisfatto
RF-OB-61	L'Utente ^G deve poter creare una nuova nota vuota all'interno del diario criptato	Non soddisfatto
RF-OB-62	L'Utente ^G deve poter eliminare una nota esistente dal diario criptato	Non soddisfatto
RF-OB-63	Il sistema deve richiedere conferma prima dell'eliminazione definitiva di una nota del diario criptato	Non soddisfatto
RF-OB-64	L'Utente ^G deve poter confermare, con un'apposita voce, l'eliminazione della nota dal diario criptato quando il sistema ne richiede la conferma	Non soddisfatto
RF-OB-65	L'Utente ^G deve poter visualizzare il contenuto completo di una nota selezionata dalla lista delle note del diario criptato	Non soddisfatto
RF-OB-66	L'Utente ^G deve poter visualizzare il contenuto testuale della nota del diario criptato	Non soddisfatto
RF-OB-67	L'Utente ^G deve poter visualizzare il contenuto multimediale della nota del diario criptato	Non soddisfatto
RF-OB-68	L'Utente ^G deve poter visualizzare il contenuto audio presente all'interno della nota del diario criptato	Non soddisfatto
RF-OB-69	L'Utente ^G deve poter visualizzare le immagini presenti all'interno della nota del diario criptato	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-70	L'Utente ^G deve poter inserire del contenuto testuale all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-71	L'Utente ^G deve poter inserire del contenuto multimediale all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-72	L'Utente ^G deve poter inserire un'immagine all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-73	L'Utente ^G deve poter inserire una traccia audio all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-74	L'Utente ^G deve poter modificare il contenuto testuale presente all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-75	L'Utente ^G deve poter rimuovere del contenuto multimediale presente all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-76	L'Utente ^G deve poter rimuovere un'immagine presente all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-77	L'Utente ^G deve poter rimuovere una traccia audio presente all'interno di una nota del diario criptato	Non soddisfatto
RF-OB-78	L'Utente ^G deve poter visualizzare la schermata del diario fittizio	Non soddisfatto
RF-OB-79	L'Utente ^G deve poter visualizzare la lista delle note presenti nel diario fittizio	Non soddisfatto
RF-OB-80	L'Utente ^G deve poter visualizzare l'anteprima della singola nota all'interno della lista delle note del diario fittizio	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-81	L'Utente ^G deve poter ricaricare la sezione delle note del diario fittizio dopo la visualizzazione dell'errore di caricamento dei dati dal database	Non soddisfatto
RF-OB-82	L'Utente ^G deve poter eliminare una nota esistente dal diario fittizio	Non soddisfatto
RF-OB-83	Il sistema deve richiedere conferma prima dell'eliminazione definitiva di una nota del diario fittizio	Non soddisfatto
RF-OB-84	L'Utente ^G deve poter confermare, con un'apposita voce, l'eliminazione della nota dal diario fittizio quando il sistema ne richiede la conferma	Non soddisfatto
RF-OB-85	L'Utente ^G deve poter creare una nuova nota vuota all'interno del diario fittizio	Non soddisfatto
RF-OB-86	L'Utente ^G deve poter visualizzare il contenuto completo di una nota selezionata dalla lista delle note del diario fittizio	Non soddisfatto
RF-OB-87	L'Utente ^G deve poter visualizzare il contenuto testuale della nota del diario fittizio	Non soddisfatto
RF-OB-88	L'Utente ^G deve poter visualizzare il contenuto multimediale della nota del diario fittizio	Non soddisfatto
RF-OB-89	L'Utente ^G deve poter visualizzare il contenuto audio presente all'interno della nota del diario fittizio	Non soddisfatto
RF-OB-90	L'Utente ^G deve poter visualizzare le immagini presenti all'interno della nota del diario fittizio	Non soddisfatto
RF-OB-91	L'Utente ^G deve poter inserire del contenuto testuale all'interno di una nota del diario fittizio	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-92	L' Utente^G deve poter inserire del contenuto multimediale all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-93	L' Utente^G deve poter inserire un'immagine all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-94	L' Utente^G deve poter inserire una traccia audio all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-95	L' Utente^G deve poter rimuovere del contenuto multimediale presente all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-96	L' Utente^G deve poter rimuovere un'immagine presente all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-97	L' Utente^G deve poter rimuovere una traccia audio presente all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-98	L' Utente^G deve poter modificare il contenuto testuale presente all'interno di una nota del diario fittizio	Non soddisfatto
RF-OB-99	L' Utente^G deve poter inviare un messaggio di SOS ai contatti fidati tramite un'apposita voce	Non soddisfatto
RF-OB-100	Il sistema deve notificare l' Utente^G dell'assenza di connessione internet durante il tentativo di invio del messaggio SOS e proporre l'invio di un segnale acustico	Non soddisfatto
RF-OB-101	Il sistema deve notificare l' Utente^G della mancanza di contatti fidati configurati durante il tentativo di invio del messaggio SOS e proporre l'invio di un segnale acustico	Non soddisfatto
RF-OB-102	L' Utente^G deve poter avviare un segnale acustico di emergenza indipendente dalle impostazioni di volume del dispositivo	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-103	L'Utente ^G deve poter inviare un messaggio al chatbot e ricevere una risposta generata dall'LLM in base al contesto scelto	Non soddisfatto
RF-OB-105	Il sistema deve notificare l'Utente ^G della mancanza di connessione internet durante la consultazione del chatbot	Non soddisfatto
RF-OB-106	L'Utente ^G deve poter ripetere l'invio del messaggio o scriverne uno nuovo dopo la notifica di assenza di connessione internet	Non soddisfatto
RF-OB-107	L'Utente ^G deve poter cambiare il contesto della chat a "Detective delle Relazioni"	Non soddisfatto
RF-OB-108	L'Utente ^G deve poter cambiare il contesto della chat a "Specchio Intelligente"	Non soddisfatto
RF-OB-111	L'Utente ^G deve poter creare una nuova sessione di chat con il chatbot	Non soddisfatto
RF-OB-112	L'Utente ^G deve poter eliminare una chat dalla cronologia delle chat	Non soddisfatto
RF-OB-113	Il sistema deve richiedere conferma esplicita prima dell'eliminazione definitiva di una chat	Non soddisfatto
RF-OB-114	L'Utente ^G deve poter confermare, con un'apposita voce, l'eliminazione di una chat quando il sistema ne richiede la conferma	Non soddisfatto
RF-OB-115	L'Utente ^G deve poter visualizzare il contenuto di una chat selezionata, inclusa la lista dei messaggi scambiati	Non soddisfatto
RF-OB-116	Il sistema deve notificare l'Utente ^G di un errore nel caricamento del contenuto della chat	Non soddisfatto
RF-OB-117	Il sistema deve visualizzare la lista dei messaggi scambiati tra l'Utente ^G e il chatbot all'interno di una chat	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-118	Il sistema deve distinguere visivamente i messaggi inviati dall' Utente^G da quelli generati dal chatbot	Non soddisfatto
RF-OB-119	Il sistema deve visualizzare i messaggi inviati dall' Utente^G con la formattazione grafica allineata a destra	Non soddisfatto
RF-OB-120	Il sistema deve visualizzare i messaggi generati dal chatbot con la formattazione grafica allineata a sinistra	Non soddisfatto
RF-OB-121	L' Utente^G deve poter visualizzare la cronologia delle chat con il chatbot	Non soddisfatto
RF-OB-122	L' Utente^G deve poter selezionare e visualizzare una singola chat dalla cronologia	Non soddisfatto
RF-OB-123	Il sistema deve mostrare il titolo identificativo di ogni chat presente nella cronologia	Non soddisfatto
RF-OB-124	L' Utente^G deve poter visualizzare la lista delle community disponibili	Non soddisfatto
RF-OB-125	Il sistema deve notificare l' Utente^G di un errore nel caricamento della lista delle community	Non soddisfatto
RF-OB-126	Il sistema deve mostrare i dati riassuntivi di ogni singola community nella lista	Non soddisfatto
RF-OB-127	Il sistema deve mostrare il nome di ogni singola community nella lista	Non soddisfatto
RF-OB-128	Il sistema deve mostrare il link di ogni singola community nella lista	Non soddisfatto
RF-OB-129	L' Utente^G deve poter aprire il link di una community per visitarne il sito web esterno	Non soddisfatto
RF-OB-130	L' Utente^G deve poter visualizzare la lista delle leggi e normative disponibili	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-131	Il sistema deve mostrare il titolo di ogni singola legge nella lista	Non soddisfatto
RF-OB-132	L'Utente ^G deve poter visualizzare i dettagli completi di una legge selezionata dalla lista	Non soddisfatto
RF-OB-133	Il sistema deve notificare l'Utente ^G di un errore nel caricamento dei dati di una legge	Non soddisfatto
RF-OB-134	L'Utente ^G deve poter visualizzare la lista dei contatti e degli enti d'emergenza	Non soddisfatto
RF-OB-135	Il sistema deve mostrare i dati riassuntivi di ogni singolo contatto d'emergenza nella lista	Non soddisfatto
RF-OB-136	Il sistema deve mostrare il nome identificativo di ogni contatto d'emergenza	Non soddisfatto
RF-OB-137	Il sistema deve mostrare il numero di telefono associato ad ogni contatto d'emergenza	Non soddisfatto
RF-OB-139	L'Utente ^G deve poter visualizzare la lista del materiale informativo disponibile	Non soddisfatto
RF-OB-140	Il sistema deve notificare l'Utente ^G di un errore nel caricamento della lista del materiale informativo	Non soddisfatto
RF-OB-141	Il sistema deve mostrare il titolo e il formato di ogni singolo materiale informativo nella lista	Non soddisfatto
RF-OB-142	L'Utente ^G deve poter visualizzare il contenuto di un materiale informativo selezionato dalla lista	Non soddisfatto
RF-OB-143	Il sistema deve notificare l'Utente ^G di un errore nel recupero dei dati del materiale informativo	Non soddisfatto
RF-OB-144	L'Utente ^G deve poter visualizzare la mappa interattiva con i marker dei luoghi sicuri nelle vicinanze	Non soddisfatto



Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori (continua)

Codice	Descrizione	Stato
RF-OB-145	L' Utente^G deve poter aprire un luogo sicuro nella mappa esterna Google Maps	Non soddisfatto
RF-OB-146	Il sistema deve notificare l' Utente^G di un errore nell'apertura dell'applicazione Google Maps e mantenere la visualizzazione della mappa interna	Non soddisfatto

Tabella 2: Tabella dello stato dei requisiti funzionali obbligatori

4.2. Requisiti Funzionali desiderabili

Codice	Descrizione	Stato
RF-DE-1	Il sistema deve mostrare all' Utente^G una spiegazione della differenza tra "per me" e "per altri"	Non Soddisfatto
RF-DE-2	Il sistema mostra all' Utente^G una breve descrizione del funzionamento del Dead Man's Switch	Non Soddisfatto
RF-DE-3	Il sistema deve permettere all' Utente^G di inserire più contenuti multimediali con una singola azione nel diario criptato	Non Soddisfatto
RF-DE-4	Il sistema deve permettere all' Utente^G di inserire più immagini con una singola azione nel diario criptato	Non Soddisfatto
RF-DE-5	Il sistema deve permettere all' Utente^G di inserire più tracce audio con una singola azione nel diario criptato	Non Soddisfatto
RF-DE-6	Il sistema deve permettere all' Utente^G di inserire più contenuti multimediali con una singola azione nel diario fittizio	Non Soddisfatto



Tabella 3: Tabella Requisiti Funzionali desiderabili (continua)

Codice	Descrizione	Stato
RF-DE-7	Il sistema deve permettere all' Utente^G di inserire più immagini con una singola azione nel diario fittizio	Non Soddisfatto
RF-DE-8	Il sistema deve permettere all' Utente^G di inserire più tracce audio con una singola azione nel diario fittizio	Non Soddisfatto
RF-DE-9	Il sistema deve permettere all' Utente^G di riprodurre le tracce audio inserite nel diario criptato	Non Soddisfatto
RF-DE-10	Il sistema deve permettere all' Utente^G di riprodurre le tracce audio inserite nel diario fittizio	Non Soddisfatto
RF-DE-11	Il sistema deve indicare all' Utente^G che il messaggio del chatbot è in fase di caricamento	Non Soddisfatto
RF-DE-12	Il sistema deve utilizzare i contenuti del diario criptato per personalizzare le risposte del chatbot	Non Soddisfatto
RF-DE-13	Il sistema deve mostrare all' Utente^G una spiegazione del tipo di contesto a cui può passare (Detective delle relazioni)	Non Soddisfatto
RF-DE-14	Il sistema deve mostrare all' Utente^G una spiegazione del tipo di contesto a cui può passare (Specchio intelligente)	Non Soddisfatto
RF-DE-15	L' Utente^G deve poter comporre automaticamente il numero di un contatto d'emergenza tramite l'apposita voce	Non Soddisfatto

Tabella 3: Tabella Requisiti Funzionali Desiderabili



4.3. Requisiti Funzionali opzionali

Codice	Descrizione	Riferimento
RF-OP-1	L' Utente^G deve poter impostare il timer di prima inattività (per la funzionalità di Dead Man's Switch)	Non Soddisfatto
RF-OP-2	L' Utente^G deve poter impostare il timer di seconda inattività (per la funzionalità di Dead Man's Switch)	Non Soddisfatto
RF-OP-3	Il sistema deve inviare un SMS ai numeri di telefono dei contatti fidati all'invio del segnale SOS	Non Soddisfatto
RF-OP-4	Il sistema deve indicare all' Utente^G se le risorse sono autorevoli e verificate	Non Soddisfatto
RF-OP-5	Il sistema deve mettere a disposizione leggi estere	Non Soddisfatto
RF-OP-6	Il sistema deve mostrare leggi aggiornate, giuridicamente valide e complete	Non Soddisfatto
RF-OP-7	Il sistema deve indicare all' Utente^G se i luoghi sicuri visualizzati nella mappa sono autorevoli e verificati	Non Soddisfatto
RF-OP-8	L' Utente^G deve poter selezionare il profilo di utilizzo ("Per me" o "Per altri")	Non soddisfatto
RF-OP-9	Il sistema deve bloccare temporaneamente l'accesso al diario in seguito al superamento del limite massimo di tentativi errati di inserimento password	Non soddisfatto
RF-OP-10	L' Utente^G deve poter interrompere la generazione della risposta dell'LLM durante la consultazione del chatbot	Non soddisfatto
RF-OP-11	Il sistema deve notificare l' Utente^G di un errore durante il cambio di contesto del chatbot e mantenere il contesto precedente	Non soddisfatto



Tabella 4: Tabella dello stato dei requisiti funzionali opzionali (continua)

Codice	Descrizione	Riferimento
RF-OP-12	L'Utente ^G deve poter ritentare il cambio di contesto del chatbot dopo la notifica di errore	Non soddisfatto

Tabella 4: Tabella dello stato dei requisiti funzionali opzionali